

posed first source matrix, perform a matrix multiplication using the transposed first source matrix and the second source matrix to generate a result, and store the result in a destination matrix location.

20 Claims, 39 Drawing Sheets

- (52) **U.S. Cl.**
CPC **G06F 9/30038** (2023.08); **G06F 9/3016**
(2013.01); **G06F 17/16** (2013.01)

(56) References Cited

U.S. PATENT DOCUMENTS

5,892,962	A	4/1999	Cloutier
6,161,219	A	12/2000	Ramkumar et al.
6,212,112	B1	4/2001	Naura et al.
6,332,186	B1	12/2001	Elwood et al.
6,877,020	B1	4/2005	Bratt et al.
7,003,542	B2	2/2006	Devir
7,209,939	B2	4/2007	Castrapel et al.
7,725,521	B2	5/2010	Chen et al.
7,792,895	B1	9/2010	Juffa et al.
7,873,812	B1	1/2011	Mimar
7,912,889	B1	3/2011	Juffa et al.
7,932,910	B2	4/2011	Hansen et al.
8,392,487	B1	3/2013	Mesh et al.
8,984,043	B2	3/2015	Ginzburg et al.
9,442,723	B2	9/2016	Yang et al.
9,906,359	B2	2/2018	Gueron
9,960,907	B2	5/2018	Gueron
10,535,114	B2	1/2020	Bolz
11,972,230	B2*	4/2024	Adelman G06F 9/30038
2003/0126176	A1	7/2003	Devir
2004/0111587	A1	6/2004	Nair et al.
2005/0193050	A1	9/2005	Sazegari
2006/0101245	A1	5/2006	Nair et al.
2006/0190517	A1	8/2006	Guerrero
2007/0186210	A1	8/2007	Hussain et al.
2008/0071851	A1	3/2008	Zohar et al.
2008/0140994	A1	6/2008	Khailany et al.
2008/0208942	A1	8/2008	Won et al.
2009/0043836	A1	2/2009	Dupaquis et al.
2009/0248778	A1	10/2009	Magerlein
2009/0292758	A1	11/2009	Brokenshire et al.
2009/0300091	A1	12/2009	Brokenshire et al.
2009/0300249	A1	12/2009	Moyer et al.
2010/0180100	A1	7/2010	Lu et al.
2010/0325187	A1	12/2010	Juffa et al.
2012/0079252	A1	3/2012	Sprangle
2012/0113133	A1	5/2012	Shpigelblat
2012/0137074	A1	5/2012	Kim et al.
2012/0254588	A1	10/2012	Adrian et al.
2012/0314774	A1	12/2012	Yang et al.
2013/0305020	A1	11/2013	Valentine et al.
2014/0149480	A1	5/2014	Catanzaro et al.
2015/0067302	A1	3/2015	Gueron
2015/0199266	A1	7/2015	Franchetti et al.
2015/0378734	A1	12/2015	Hansen et al.

2018/0113708	A1	4/2018	Corbal et al.
2019/0042202	A1	2/2019	Sade et al.
2019/0042254	A1	2/2019	Sade et al.
2019/0377573	A1	12/2019	Magklis et al.

FOREIGN PATENT DOCUMENTS

WO	2004/053841	A2	6/2004
WO	2016/003740	A1	1/2016
WO	2016/105727	A1	6/2016
WO	2018/125250	A1	7/2018
WO	2018/126073	A1	7/2018

OTHER PUBLICATIONS

Corrected Notice of Allowability, U.S. Appl. No. 15/201,442, filed Mar. 11, 2019, 2 pages.
European Search Report and Search Opinion, EP App. No. 20209948.7, May 31, 2021, 08 pages.
Intention to Grant, EP App. No. 20209948.7, May 2, 2024, 6 pages.
International Preliminary Report on Patentability, PCT App. No. PCT/US2017/036038, Jan. 17, 2019, 14 pages.
International Preliminary Report on Patentability, PCT App. No. PCT/US2017/040546, Oct. 3, 2019, 10 pages.
International Search Report and Written Opinion, PCT App. No. PCT/US2017/036038, Sep. 5, 2017, 15 pages.
International Search Report and Written Opinion, PCT App. No. PCT/US2017/040534, Jan. 3, 2018, 11 pages.
International Search Report and Written Opinion, PCT App. No. PCT/US2017/040536, Dec. 20, 2017, 11 pages.
International Search Report and Written Opinion, PCT App. No. PCT/US2017/040537, Dec. 20, 2017, 11 pages.
International Search Report and Written Opinion, PCT App. No. PCT/US2017/040540, Jan. 3, 2018, 14 pages.
International Search Report and Written Opinion, PCT App. No. PCT/US2017/040546, Jan. 24, 2018, 15 pages.
Lahr, David L., "Distractions: Timing Matrix Multiplication in SciDB and Setting the Number of Worker Instances in SciDB and Running Matrix Multiplication Piecemeal", Available Online at <<http://dllaahr.blogspot.com/2012/11/timing-matrix-multiplication-in-scldb.html>>, Nov. 13, 2012, 8 pages.
Non-Final Office Action, U.S. Appl. No. 15/201,442, May 4, 2018, 11 pages.
Non-Final Office Action, U.S. Appl. No. 16/398,200, Jul. 28, 2020, 17 pages.
Non-Final Office Action, U.S. Appl. No. 16/914,318, Aug. 16, 2023, 8 pages.
Notice of Allowance, U.S. Appl. No. 15/201,442, Dec. 14, 2018, 5 pages.
Notice of Allowance, U.S. Appl. No. 16/914,318, Dec. 22, 2023, 8 pages.
Office Action, EP App. No. 20209948.7, Apr. 17, 2023, 9 pages.
Office Action, EP App. No. 20209948.7, Jun. 14, 2022, 5 pages.
Summons to attend oral proceedings, EP App. No. 20209948.7, Sep. 27, 2023, 20 pages.
Decision to Grant, EP App. No. 20209948.7, Sep. 12, 2024, 2 pages.
Extended European search report and Search Opinion, EP App. No. 24203555.8, Jan. 21, 2025, 10 pages.
Extended European search report and Search Opinion, EP App. No. 24205150.6, Jan. 21, 2025, 36 pages.

* cited by examiner

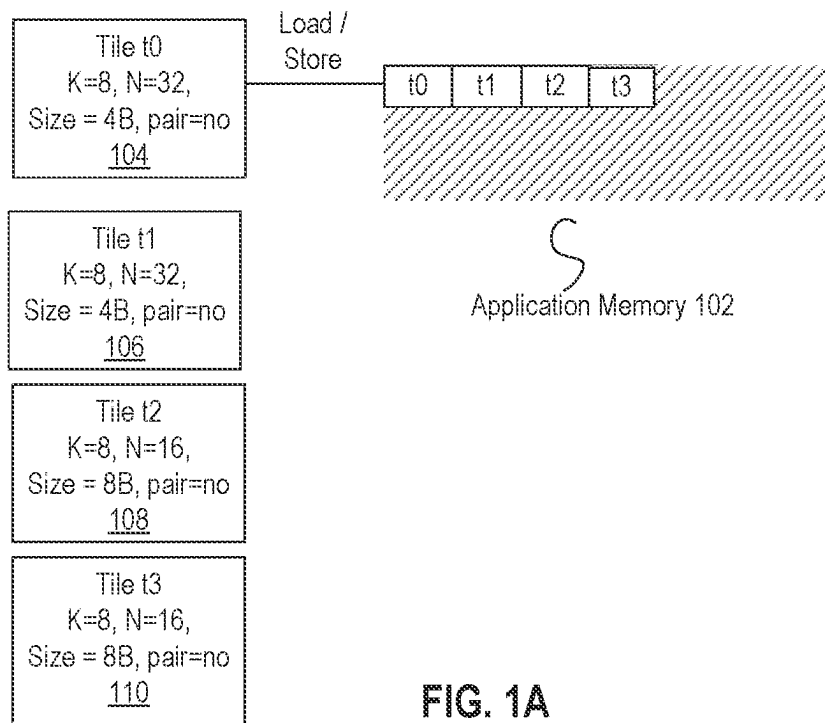


FIG. 1A

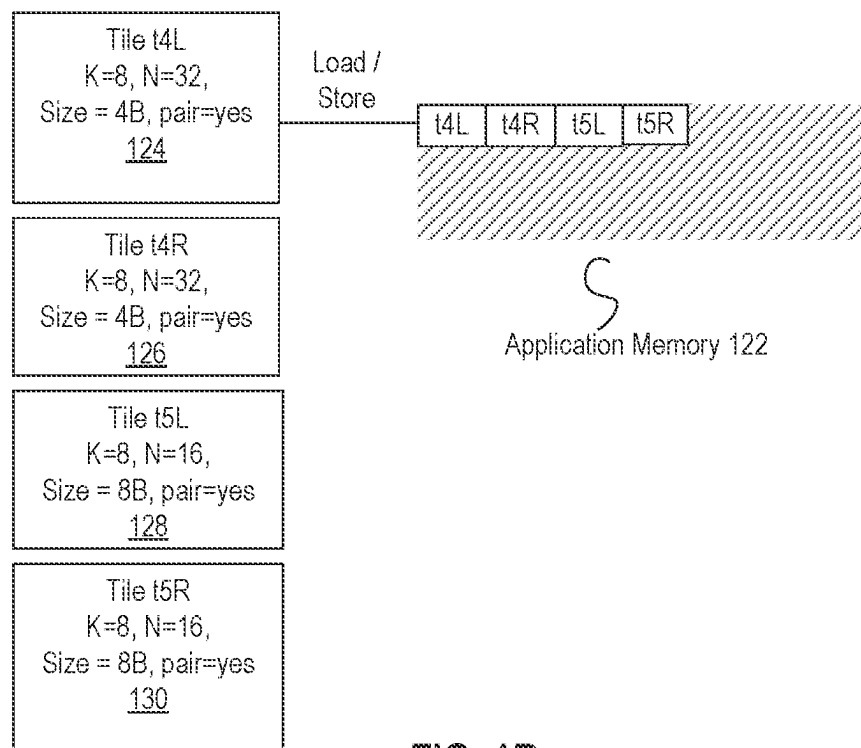


FIG. 1B

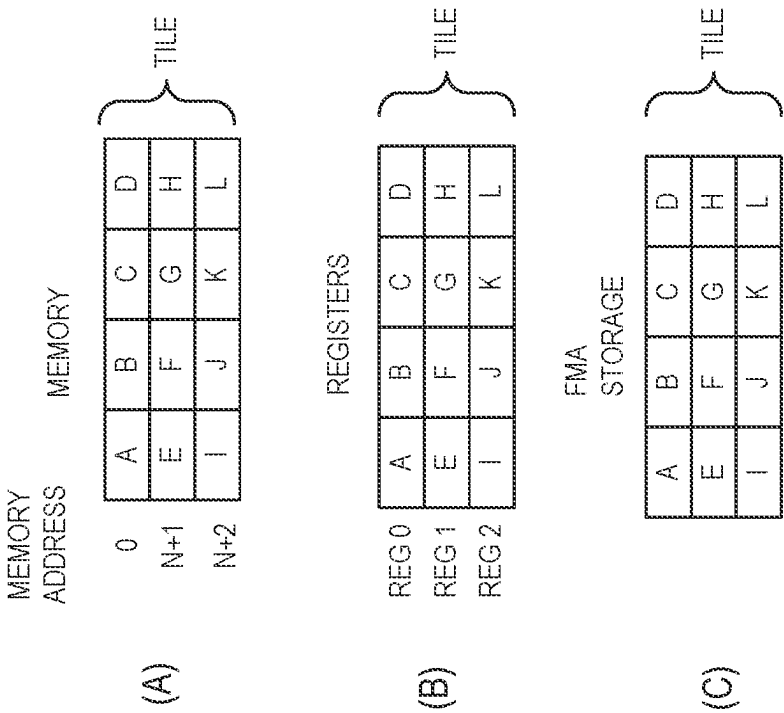


FIG. 2

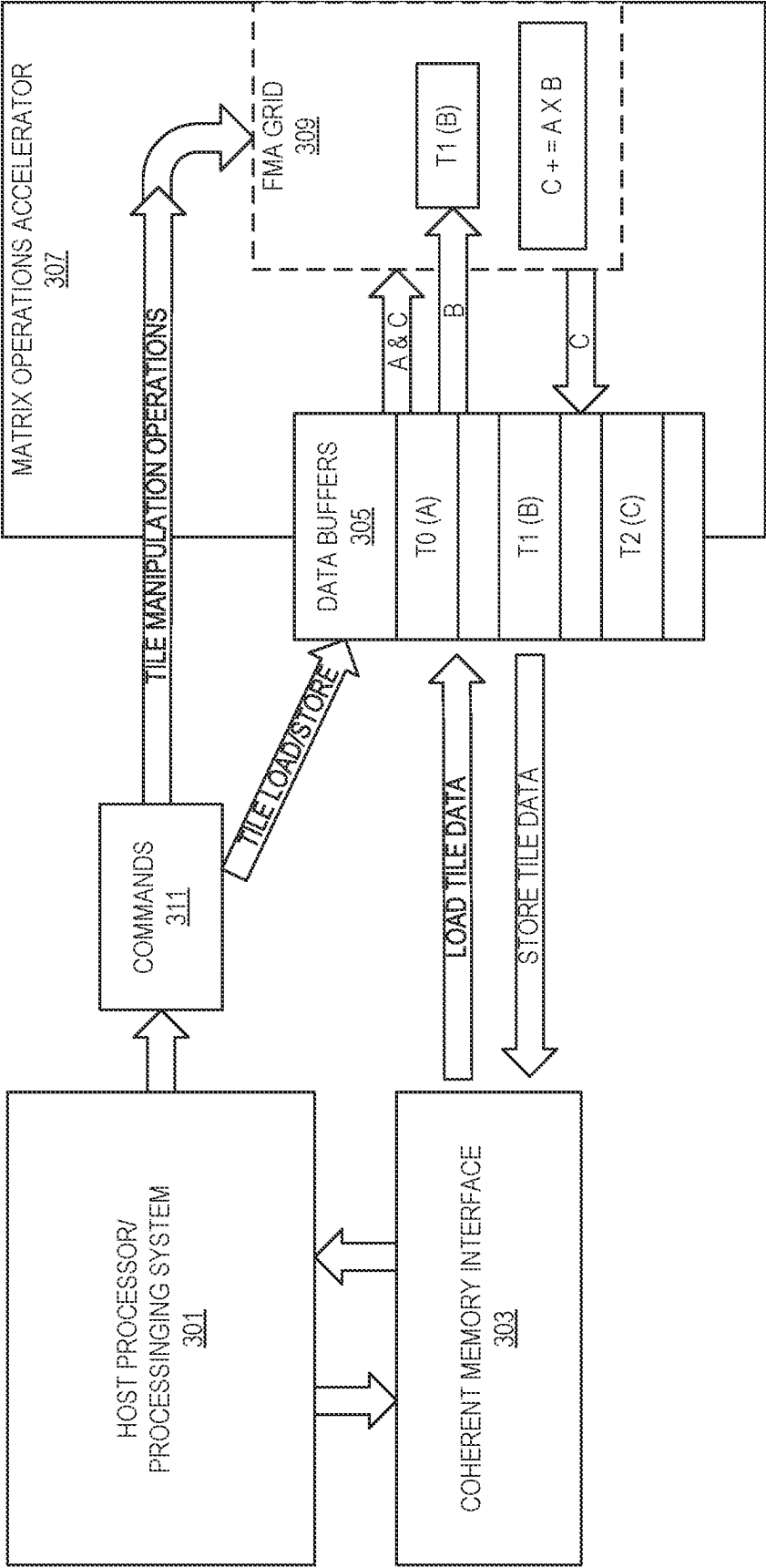


FIG. 3

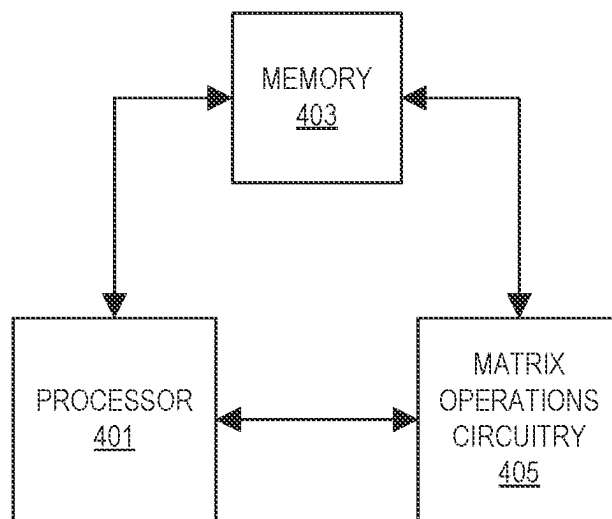


FIG. 4

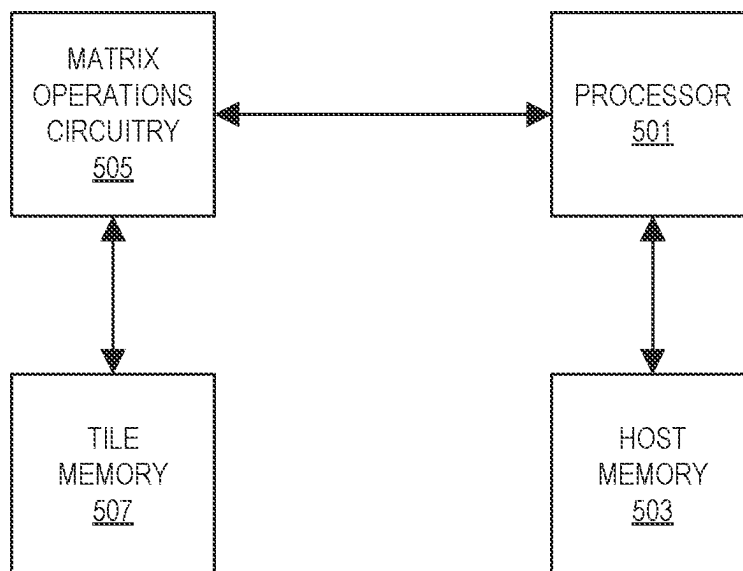


FIG. 5

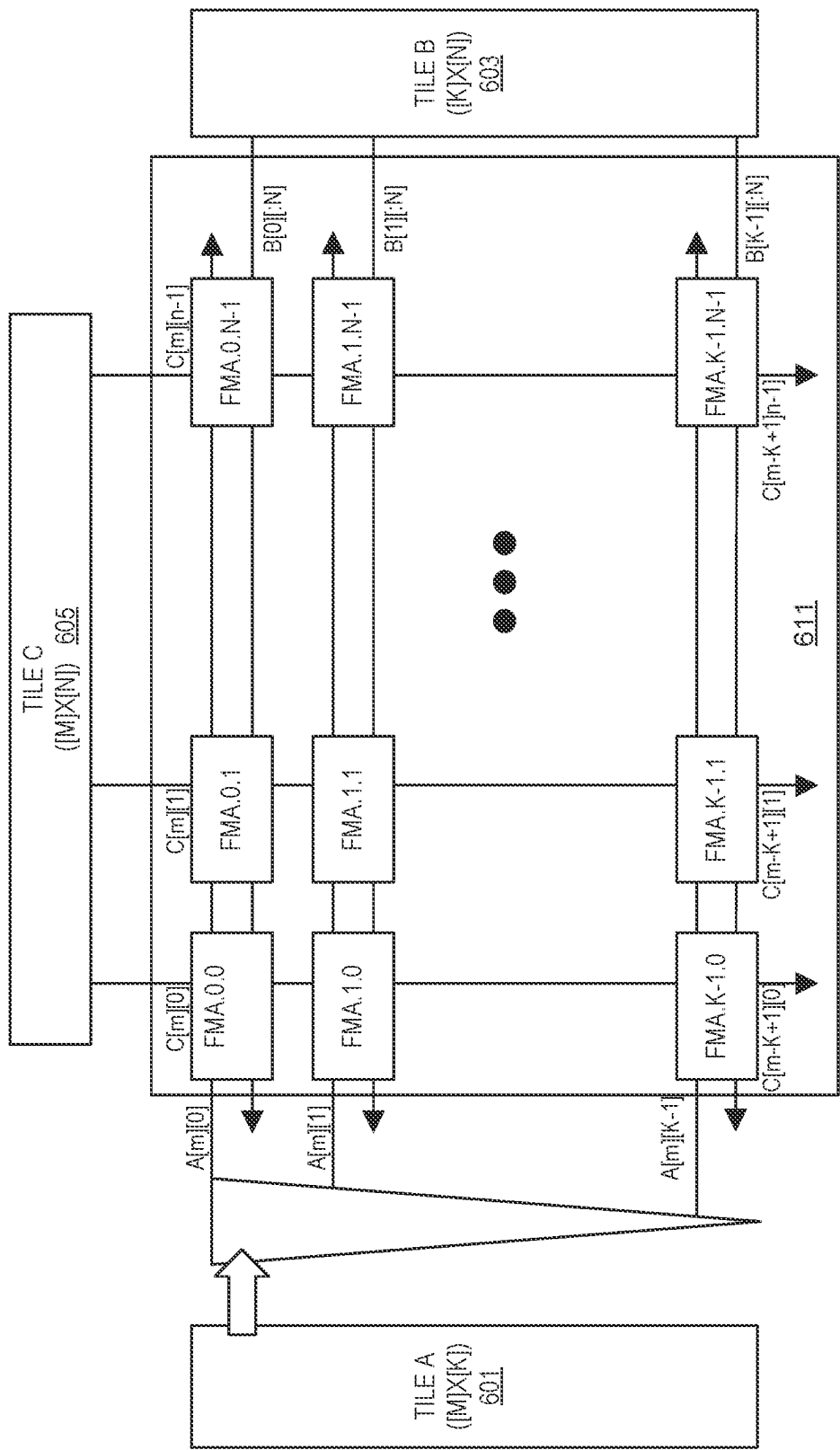


FIG. 6

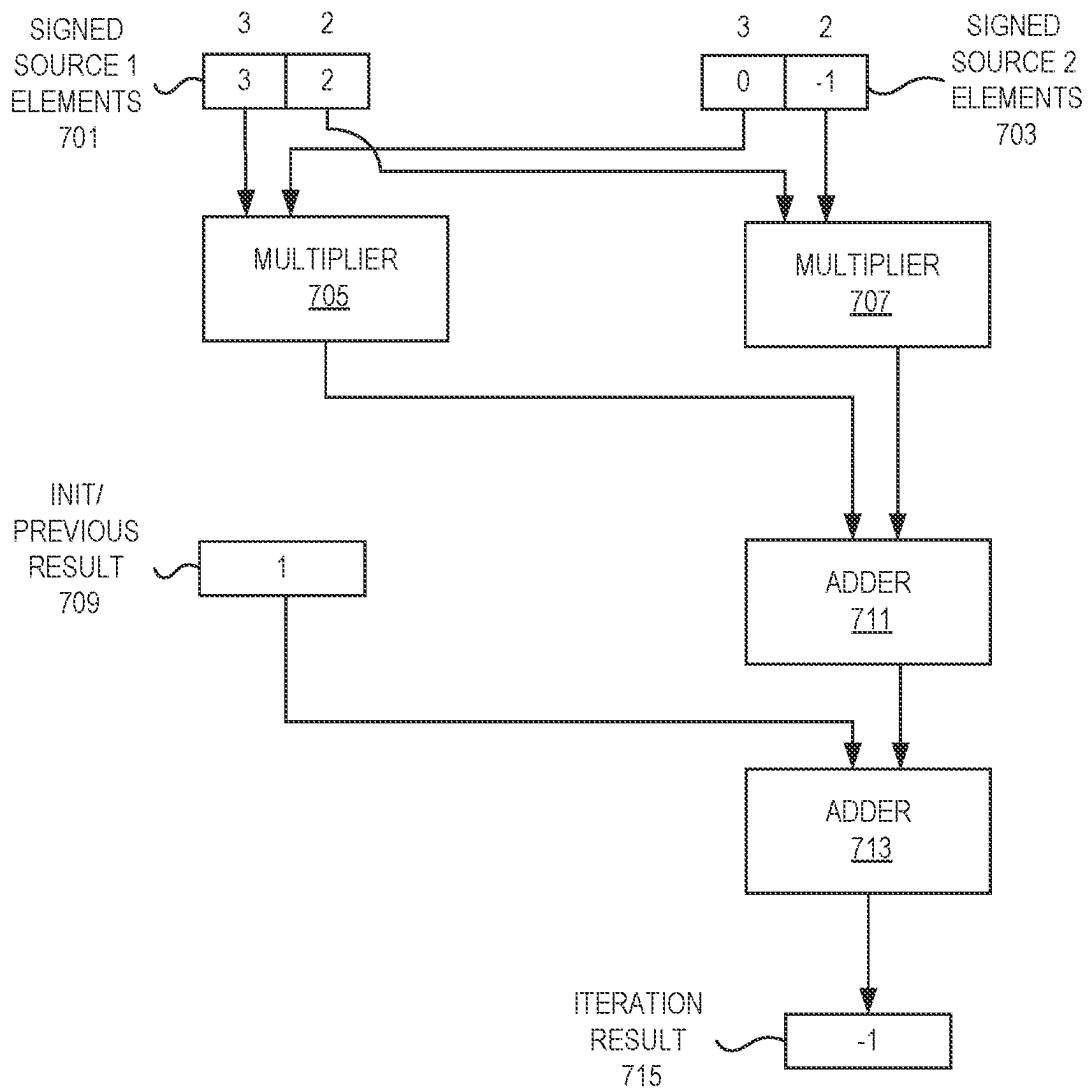


FIG. 7

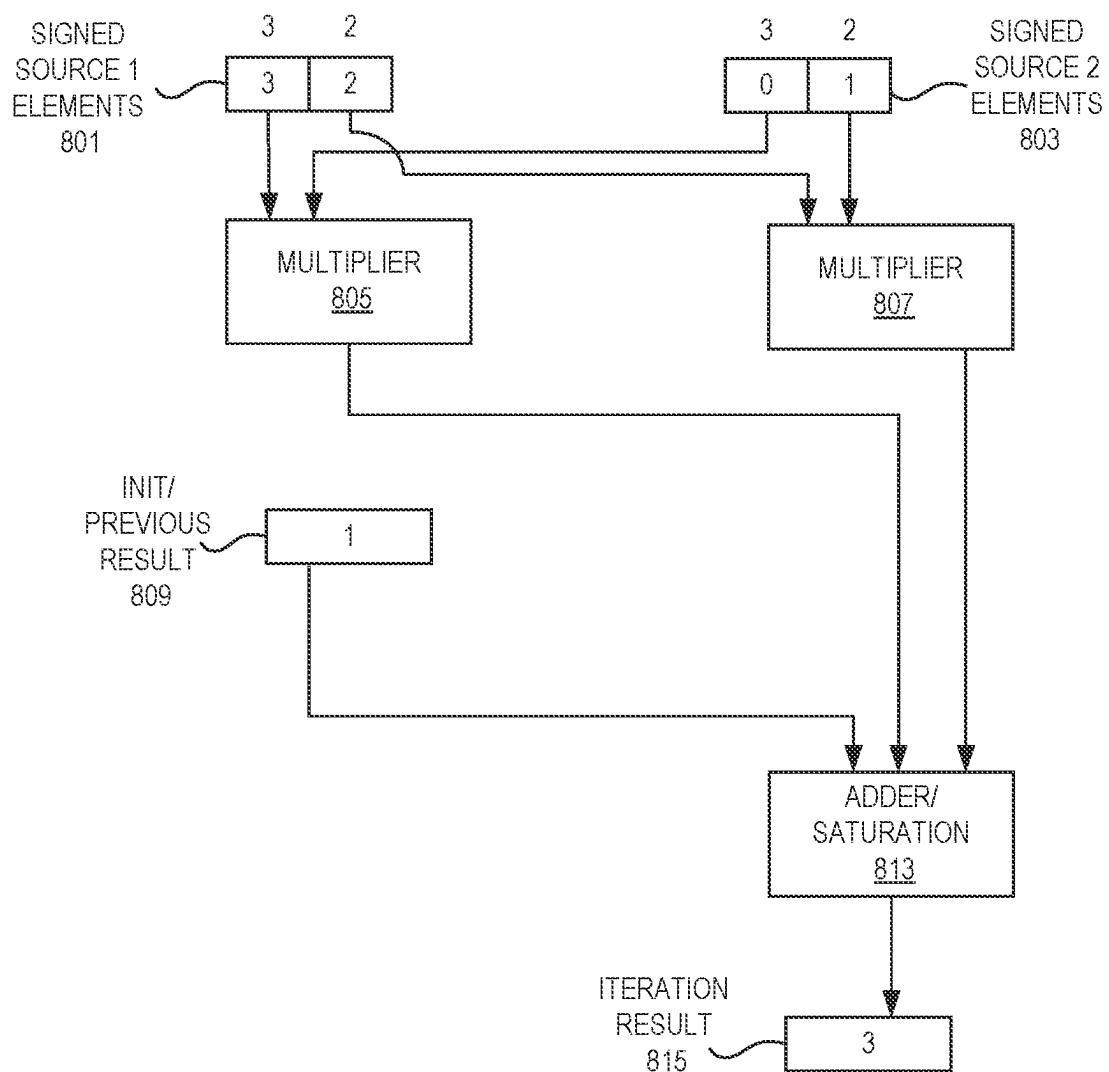


FIG. 8

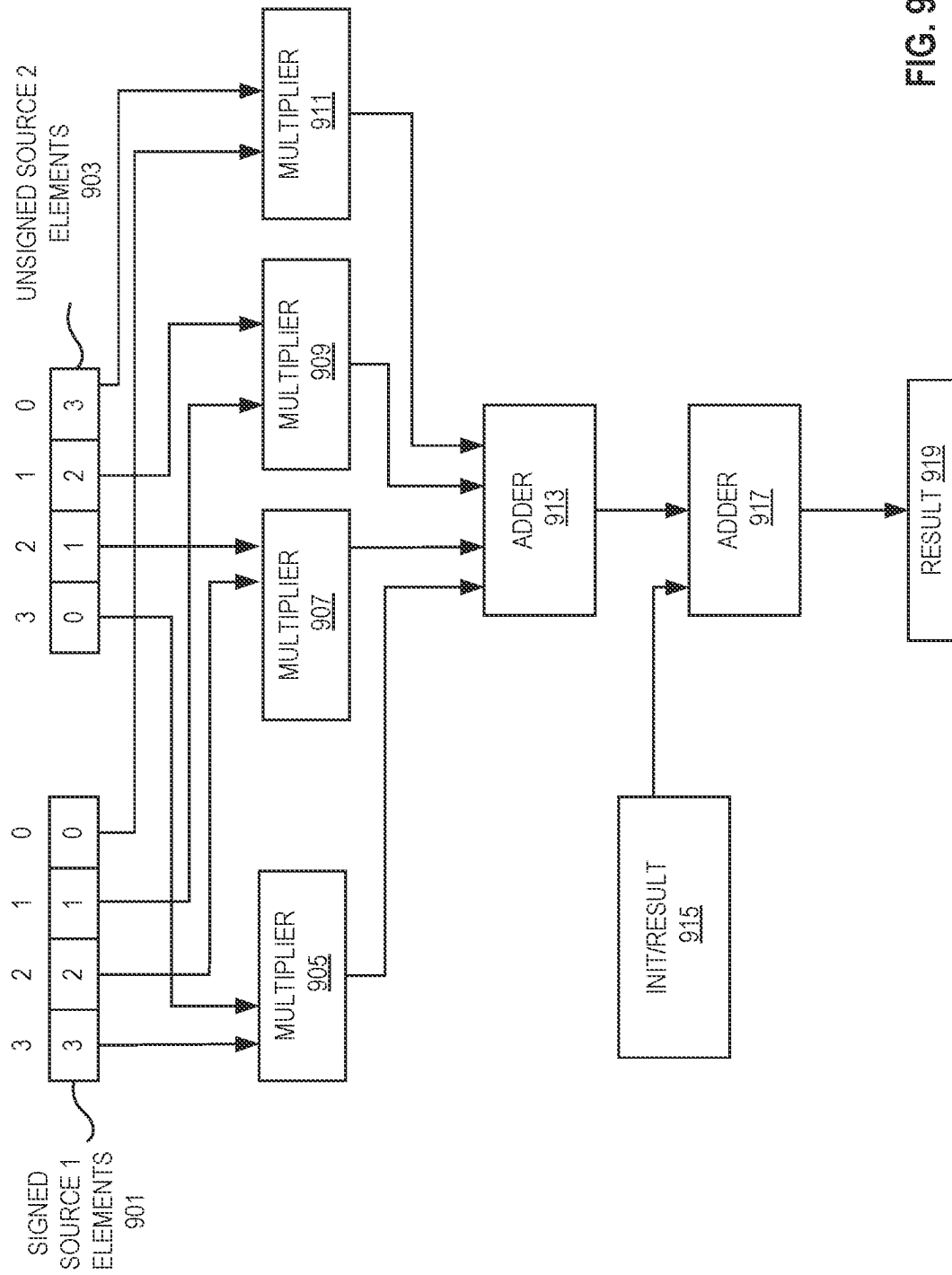
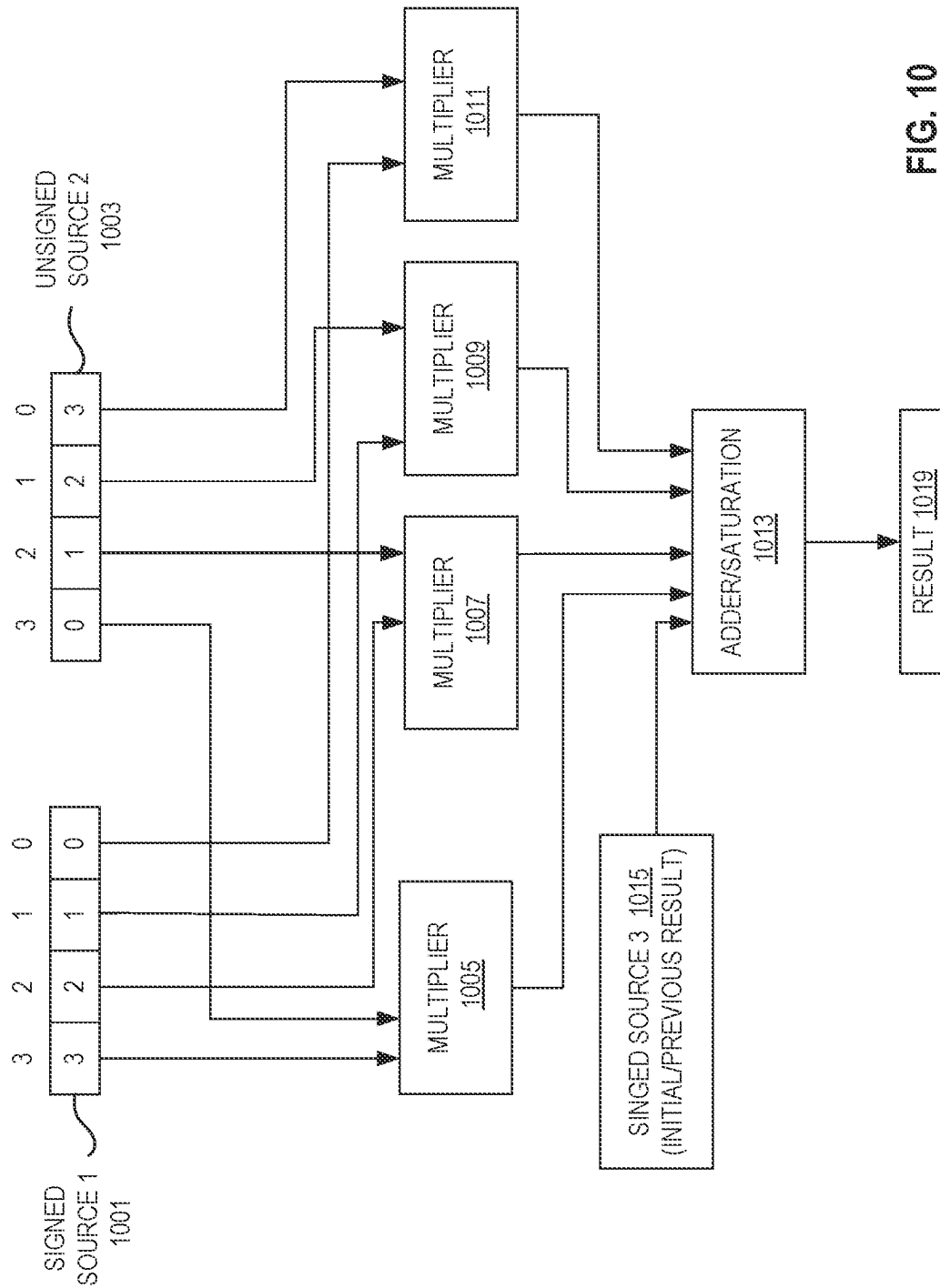


FIG. 9



ACCUMULATOR 2X INPUT SIZES 1101

SOURCES	BITS	ACCUMULATOR	BITS
BYTE	8	WORD/HPFP	16
WORD	16	INT32/SPFP	32
SPFP/INT32	32	INT64/DPFP	64

ACCUMULATOR 4X INPUT SIZES 1103

SOURCES	BITS	ACCUMULATOR	BITS
BYTE	8	INT32/SPFP	32
WORD	16	INT64/DPFP	64

ACCUMULATOR 8X INPUT SIZES 1105

SOURCES	BITS	ACCUMULATOR	BITS
BYTE	8	INT64/DPFP	64

FIG. 11

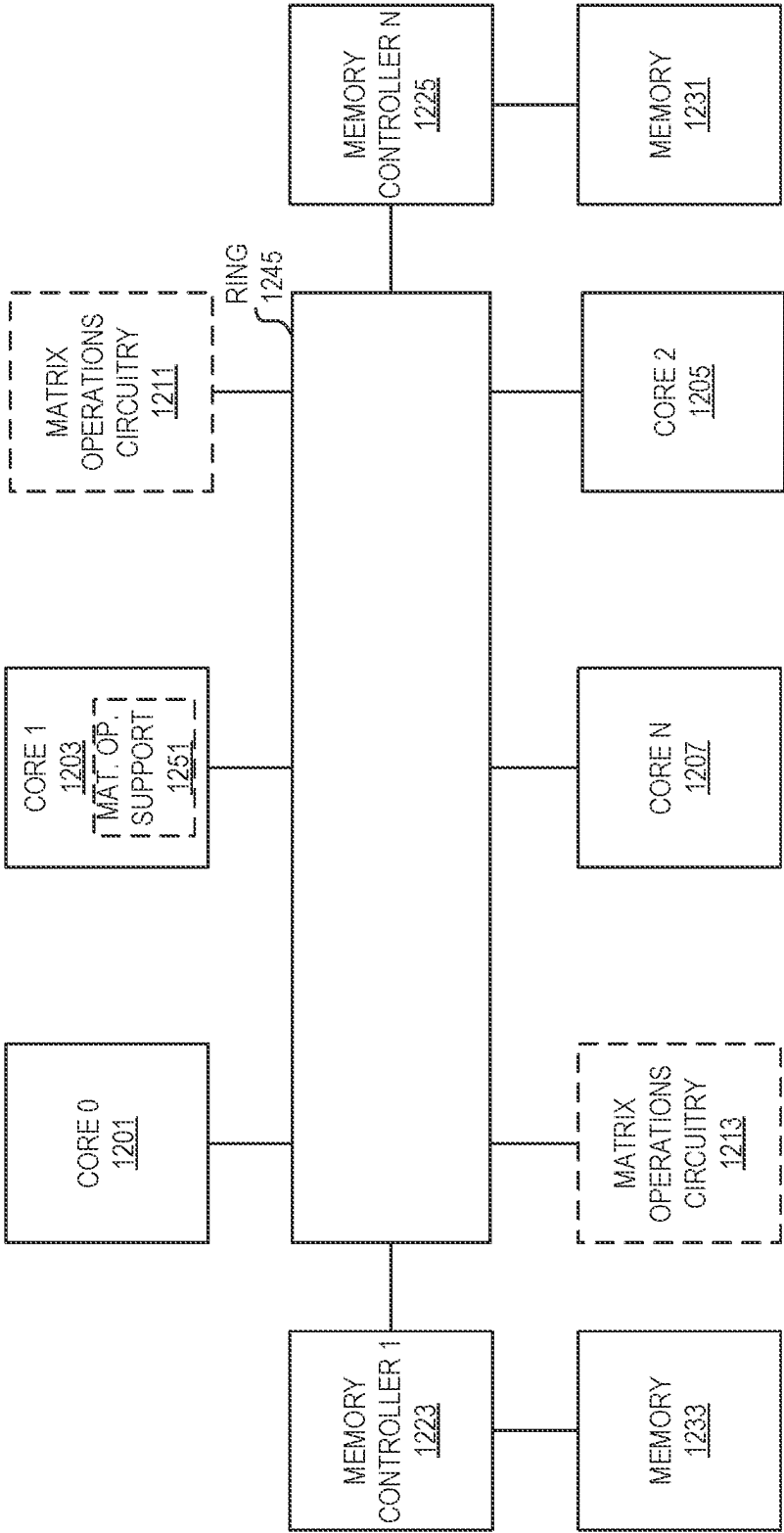


FIG. 12

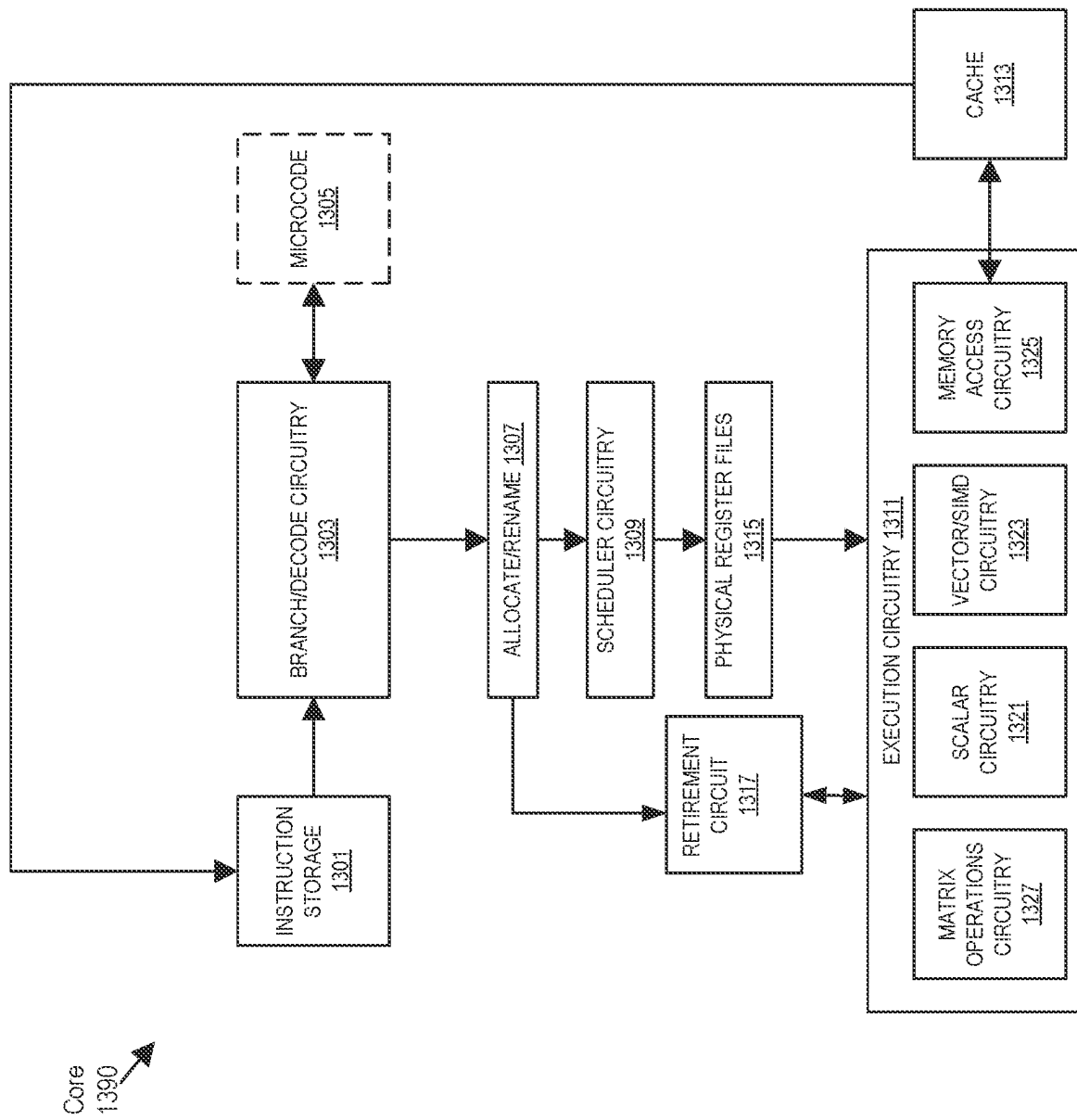


FIG. 13

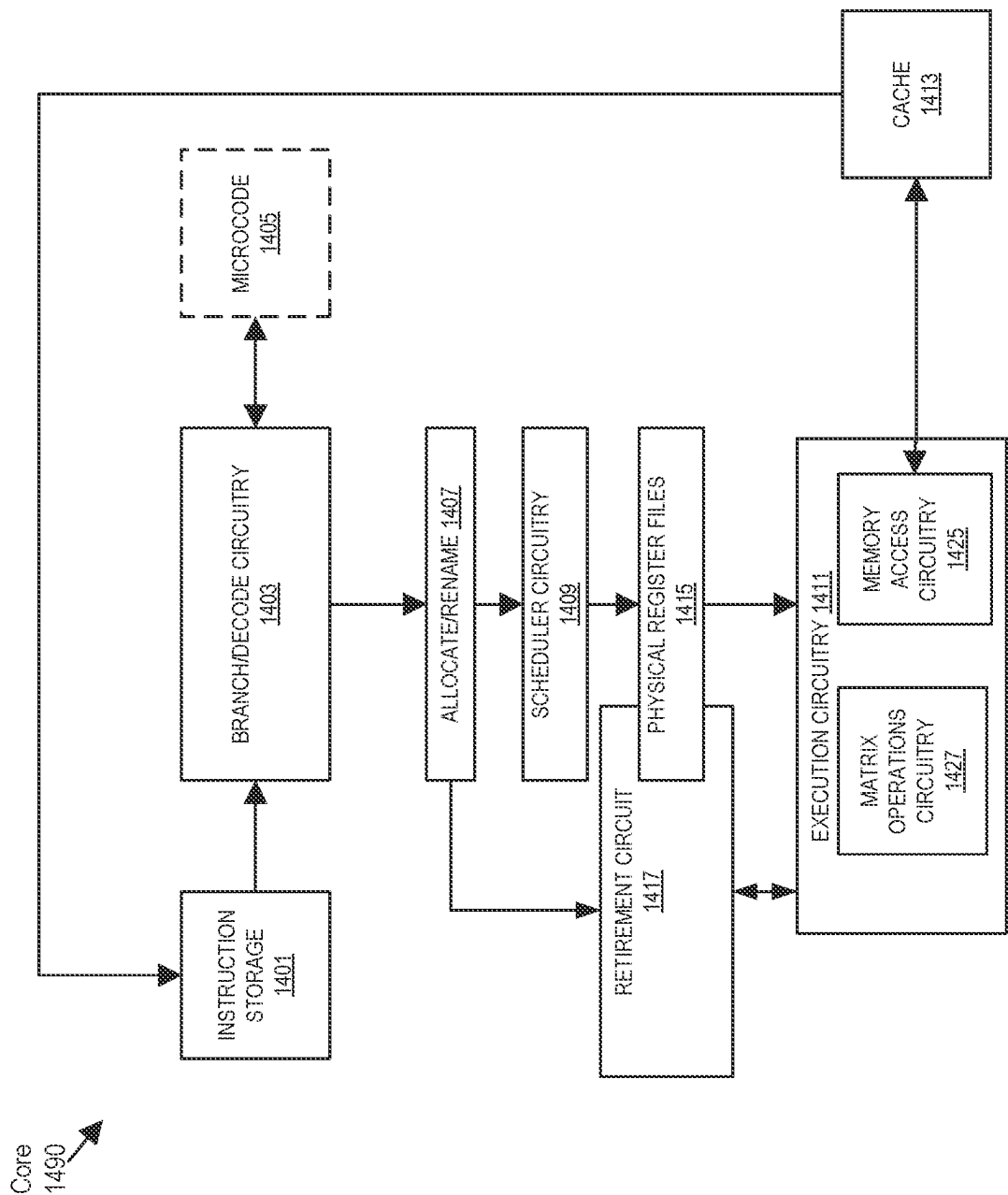


FIG. 14

$$A = \begin{bmatrix} A_{11} & A_{12} & A_{13} \\ A_{21} & A_{22} & A_{23} \end{bmatrix}$$

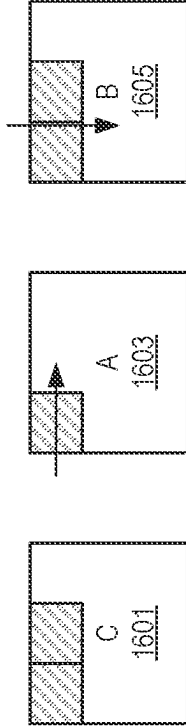
ADDR	VALUE
0	A_{11}
1	A_{12}
2	A_{13}
3	A_{21}
4	A_{22}
5	A_{23}

ROW MAJOR

ADDR	VALUE
0	A_{11}
1	A_{21}
2	A_{12}
3	A_{22}
4	A_{13}
5	A_{23}

COLUMN MAJOR

FIG. 15



```

TILECONFIG [RAX]
// ASSUME SOME OUTER LOOPS DRIVING THE CACHE TILING (NOT SHOWN)
{
    TILELOAD TMM0, RSI+RDI // SRCDST, RSI POINTS TO C, RDI HAS
    TILELOAD TMM1, RSI+RDI+N // SECOND TILE OF C, UNROLLING IN SIMD DIMENSION N
    MOV KK, 0
    LOOP:
        TILELOAD TMM2, R8+R9 // SRC2 IS STRIDED LOAD OF A, REUSED FOR 2 TMM A INSTR.
        TILELOAD TMM3, R10+R11 // SRC1 IS STRIDED LOAD OF B
        TMMAPS TMM0, TMM2, TMM3 // UPDATE LEFT TILE OF C
        TILELOAD TMM3, R10+R11+N // SRC1 LOADED WITH B FROM NEXT RIGHTMOST TILE
        TMMAPS TMM1, TMM2, TMM3 // UPDATE RIGHT TILE OF C
        ADD R8, K // UPDATE POINTERS BY CONSTANTS KNOWN OUTSIDE OF LOOP
        ADD R10, K*R11
        ADD KK, K
        CMP KK, LIMIT
        JNE LOOP
        TILESTORE RSI+RDI, TMM0 // UPDATE THE C MATRIX IN MEMORY
        TILESTORE RSI+RDI+M, TMM1
    } // END OF OUTER LOOP
    TILERelease // RETURN TILES TO INIT STATE

```

FIG. 16

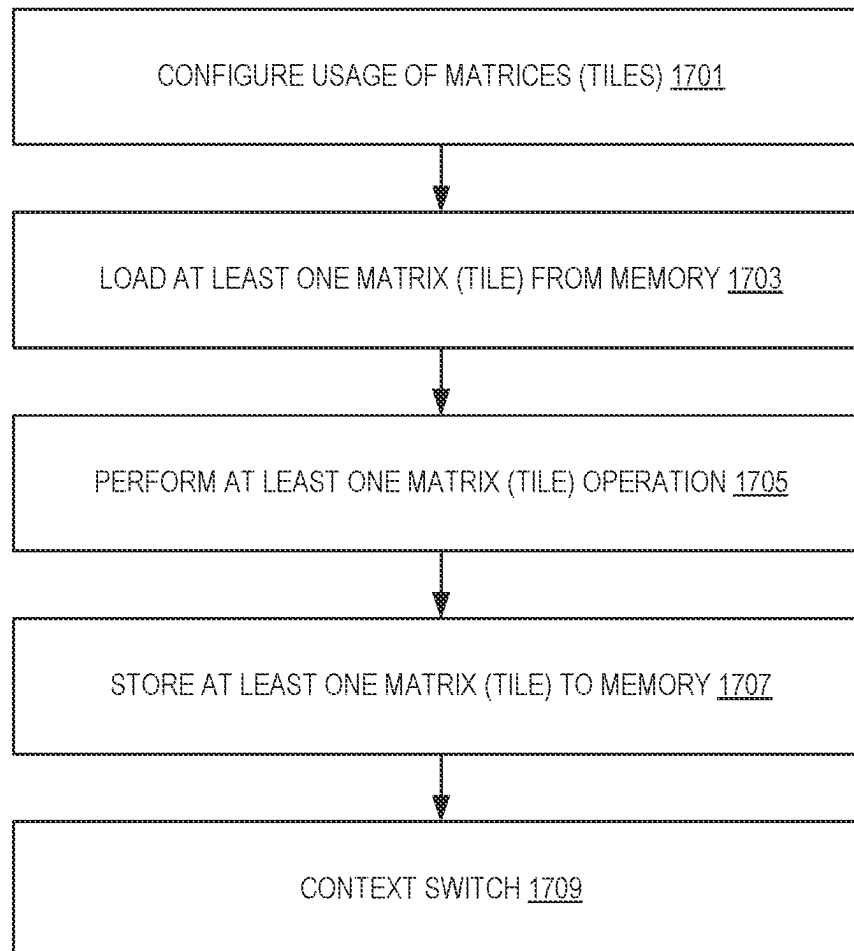


FIG. 17

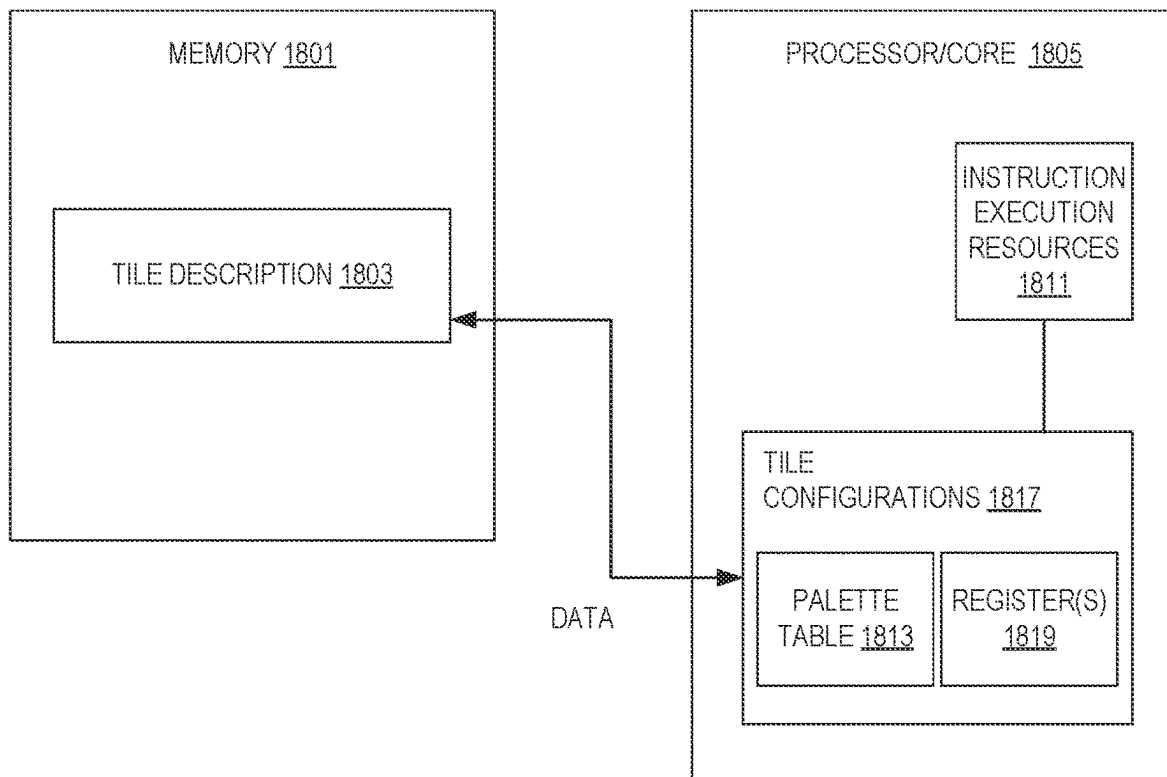


FIG. 18

PALETTE ID <u>1901</u>	STARTM <u>1903</u>
STARTP <u>1905</u>	PAIR INDICATORS <u>1907</u>
0	0
0	0

...

0	0
TMM0 ROWS <u>1913</u>	TMM0 COLUMNS <u>1915</u>
TMM1 ROWS	TMM1 COLUMNS
x x x	
TMM15 ROWS	TMM15 COLUMNS
0	

FIG. 19



FIG. 20(A)

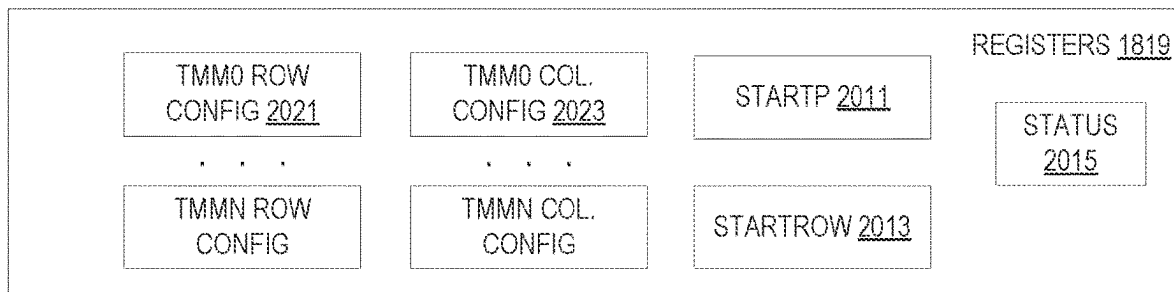


FIG. 20(B)



FIG. 20(C)



FIG. 20(D)

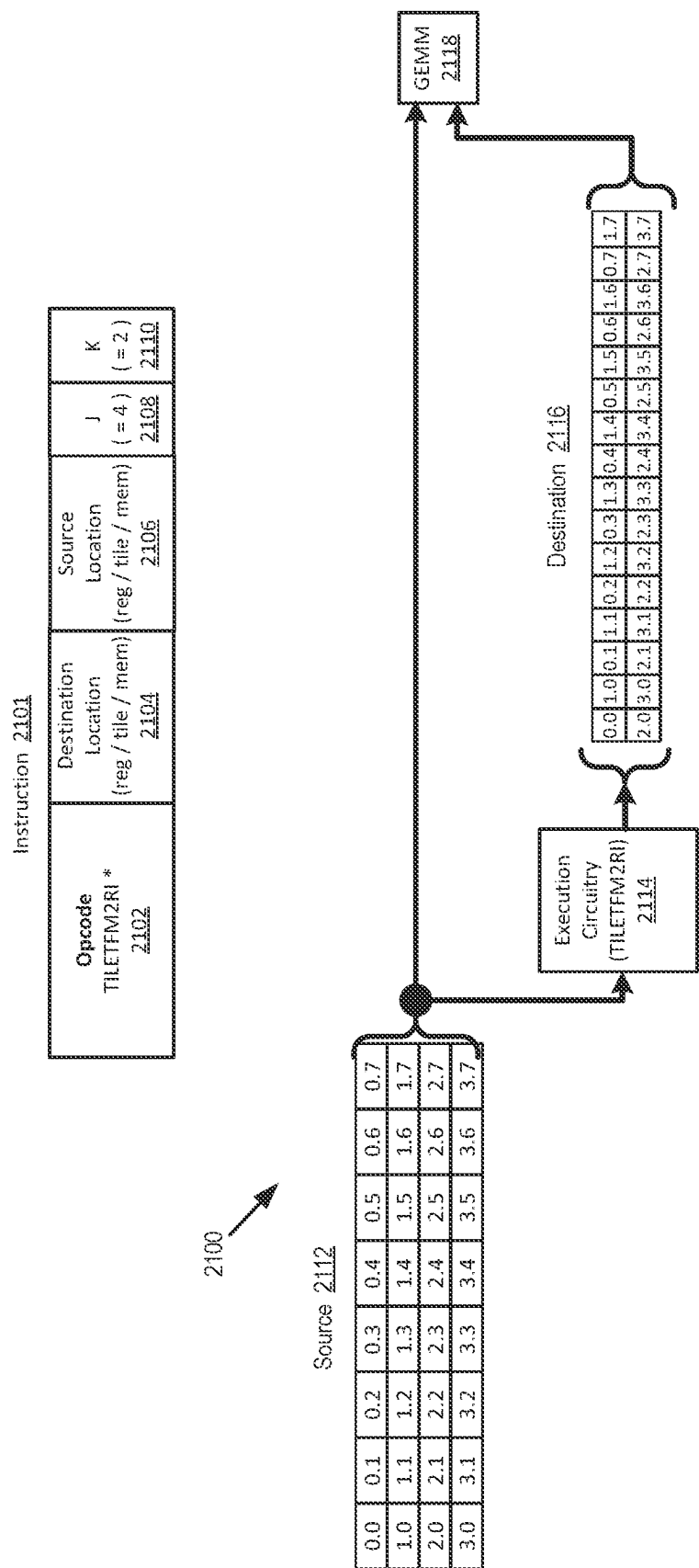


FIG. 21A

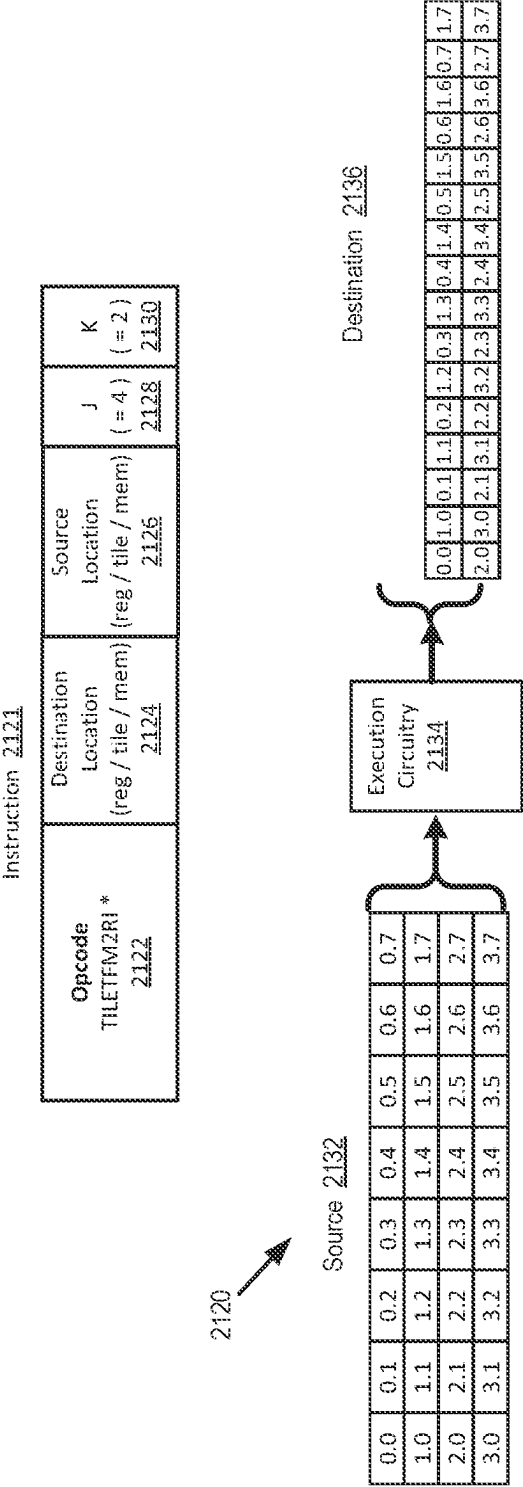
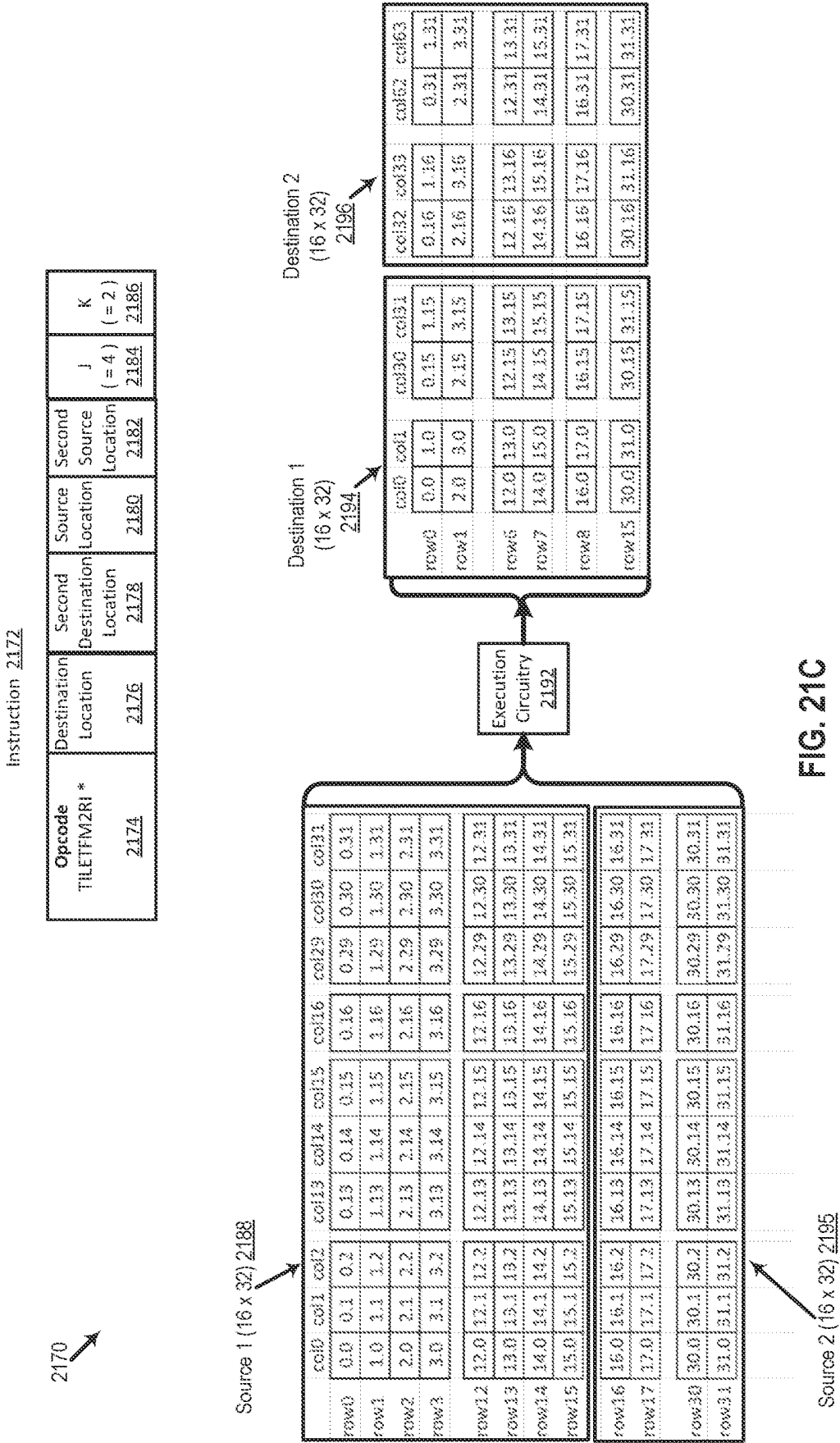


FIG. 21B



A^T – 4x8 BF16 matrix, flat format

0	10	20	30	40	50	60	70
1	11	21	31	41	51	61	71
2	12	22	32	42	52	62	72
3	13	23	33	43	53	63	73

A – 8x4 BF16 matrix, VNNI format

0	10	1	11	2	12	3	13
20	30	21	31	22	32	23	33
40	50	41	51	42	52	43	53
60	70	61	71	62	72	63	73

2220

2210

FIG. 22A

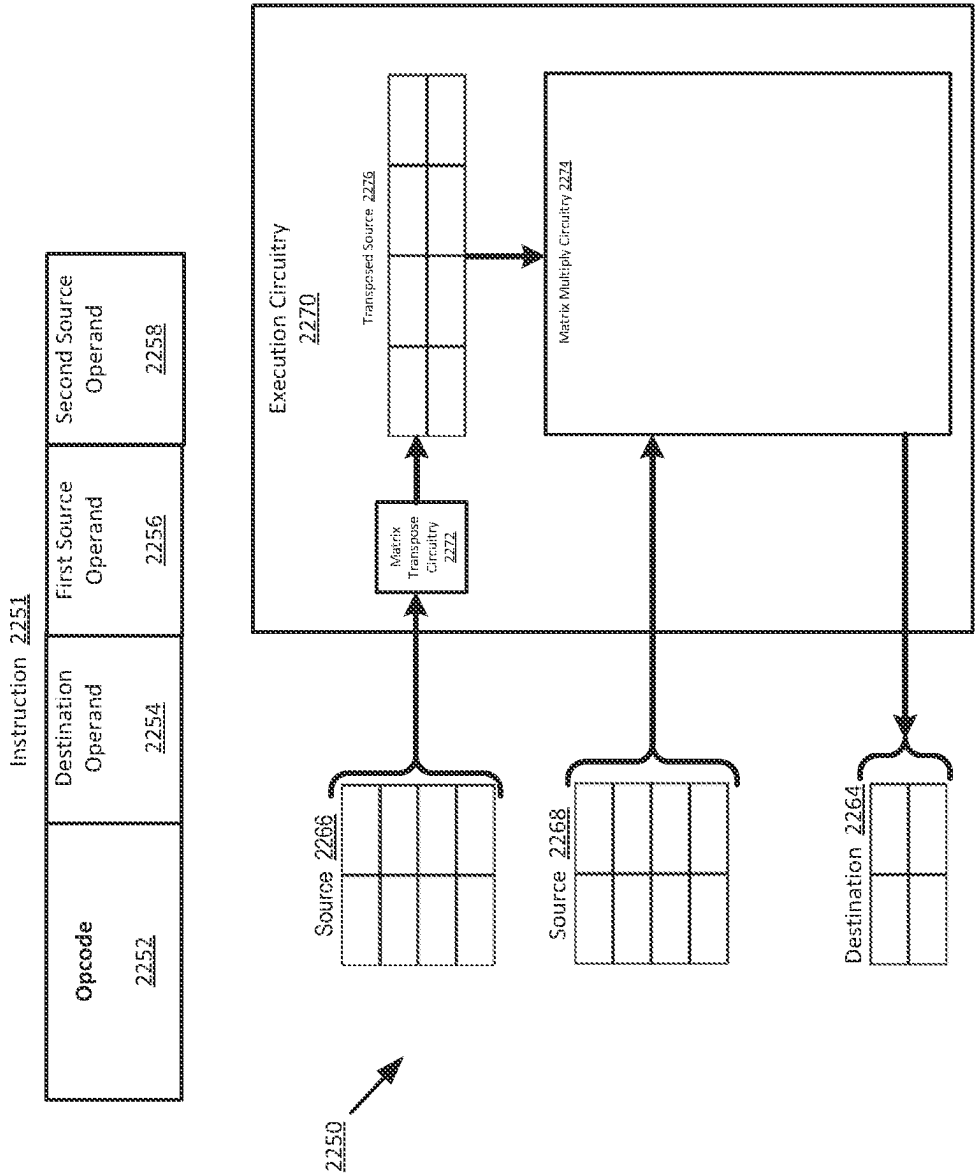


FIG. 22B

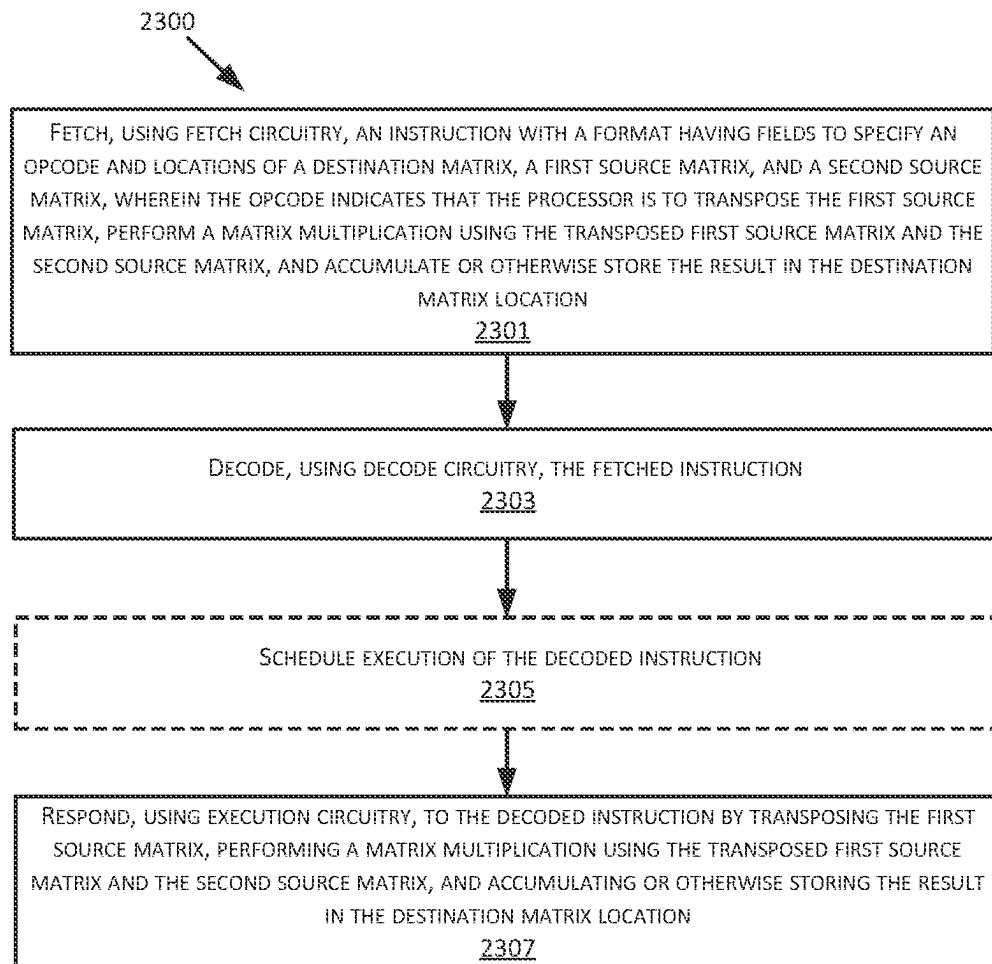


FIG. 23

Matrix Transpose+Multiply Instruction 2400

Opcode	Destination Operand	First Source Operand	Second Source Operand	Other Modifiers
<u>2402</u>	<u>2404</u>	<u>2406</u>	<u>2408</u>	<u>2420</u>

FIG. 24

25A
G²
L

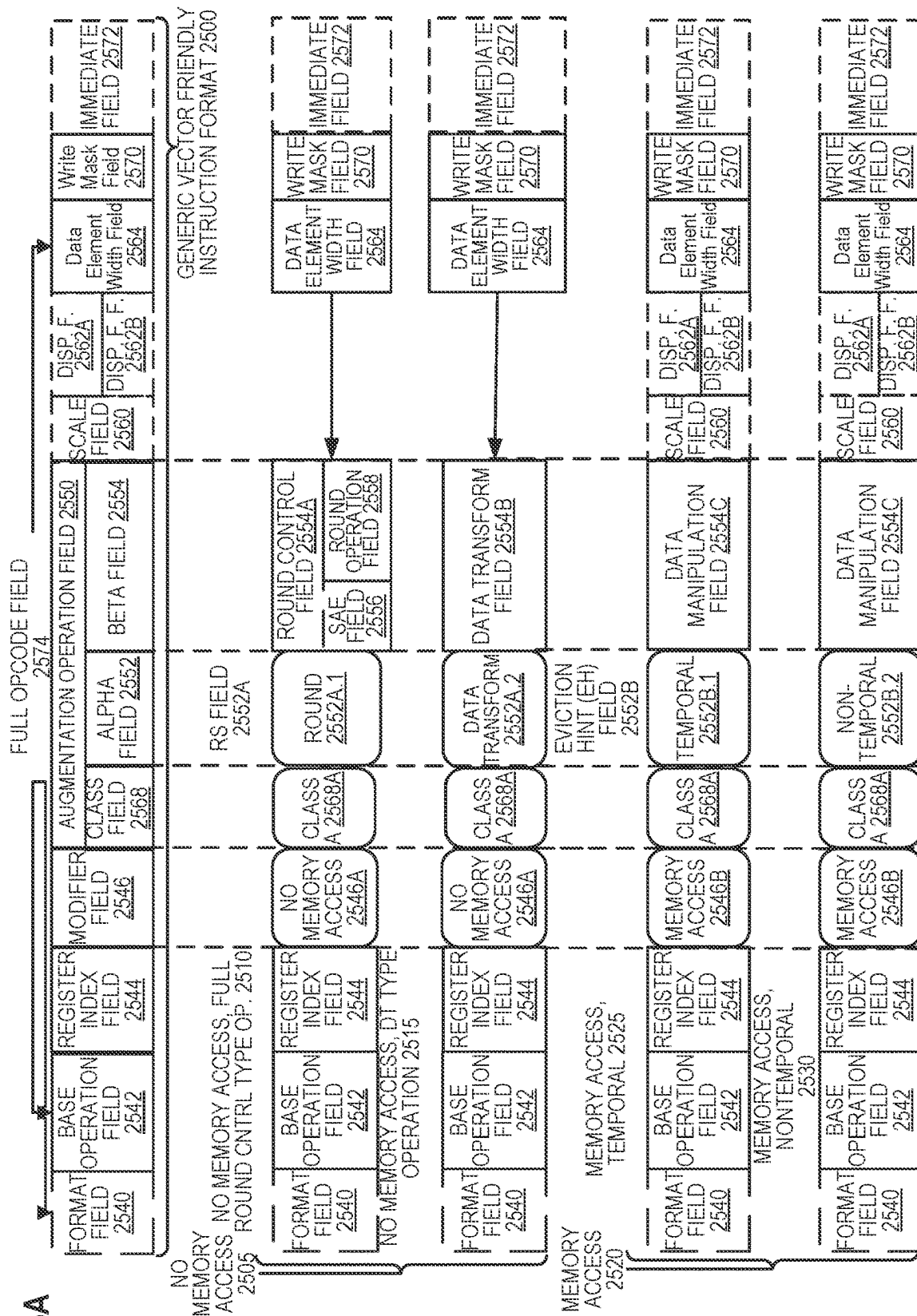
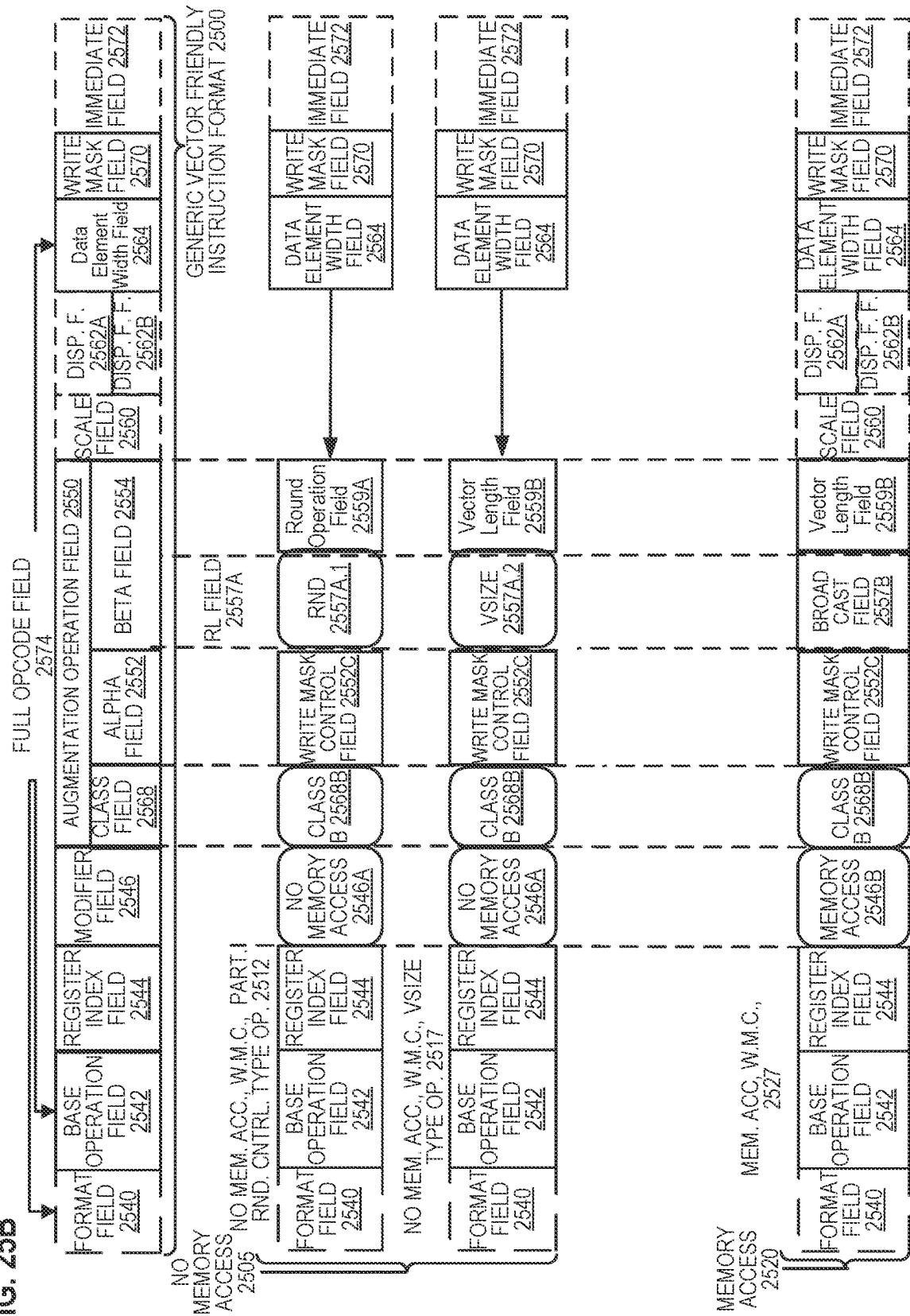


FIG. 25B



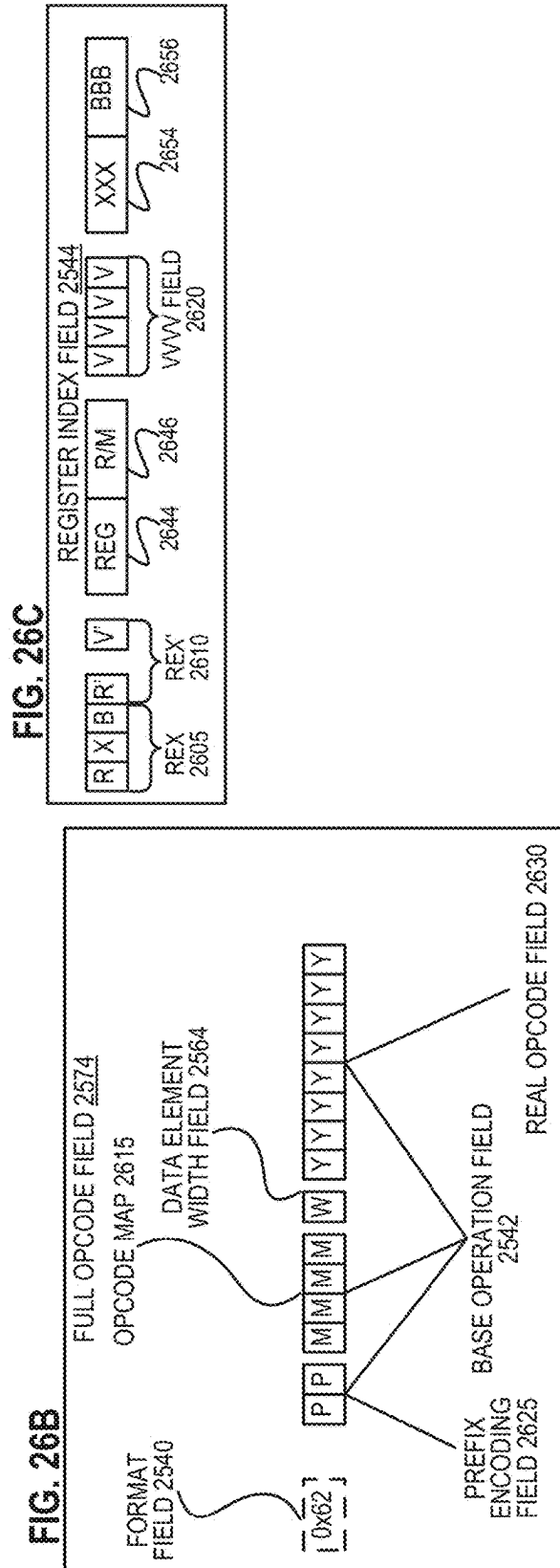
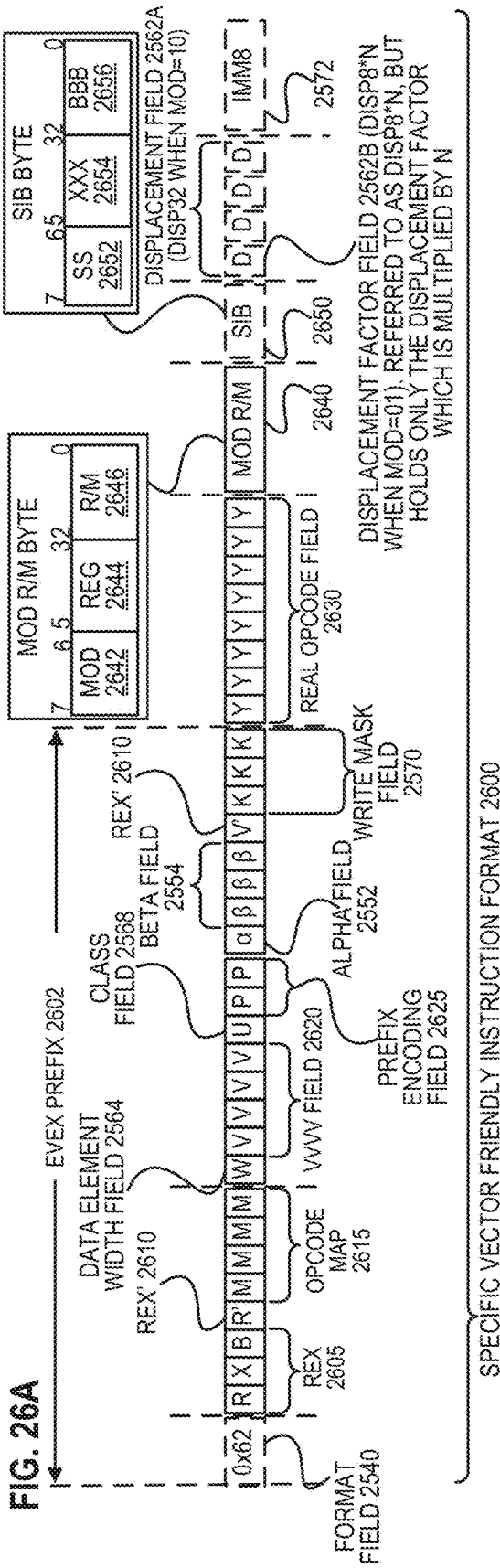
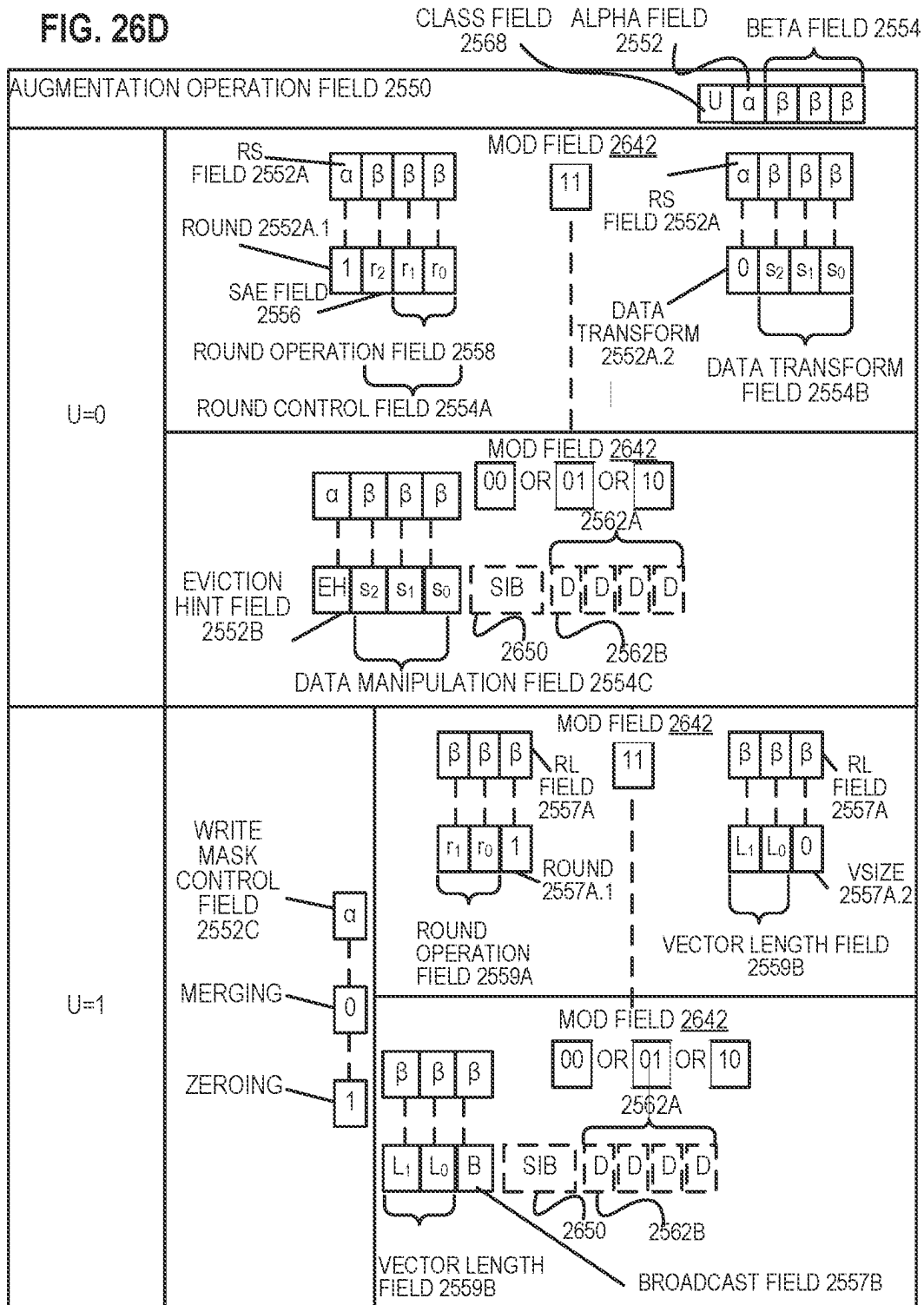
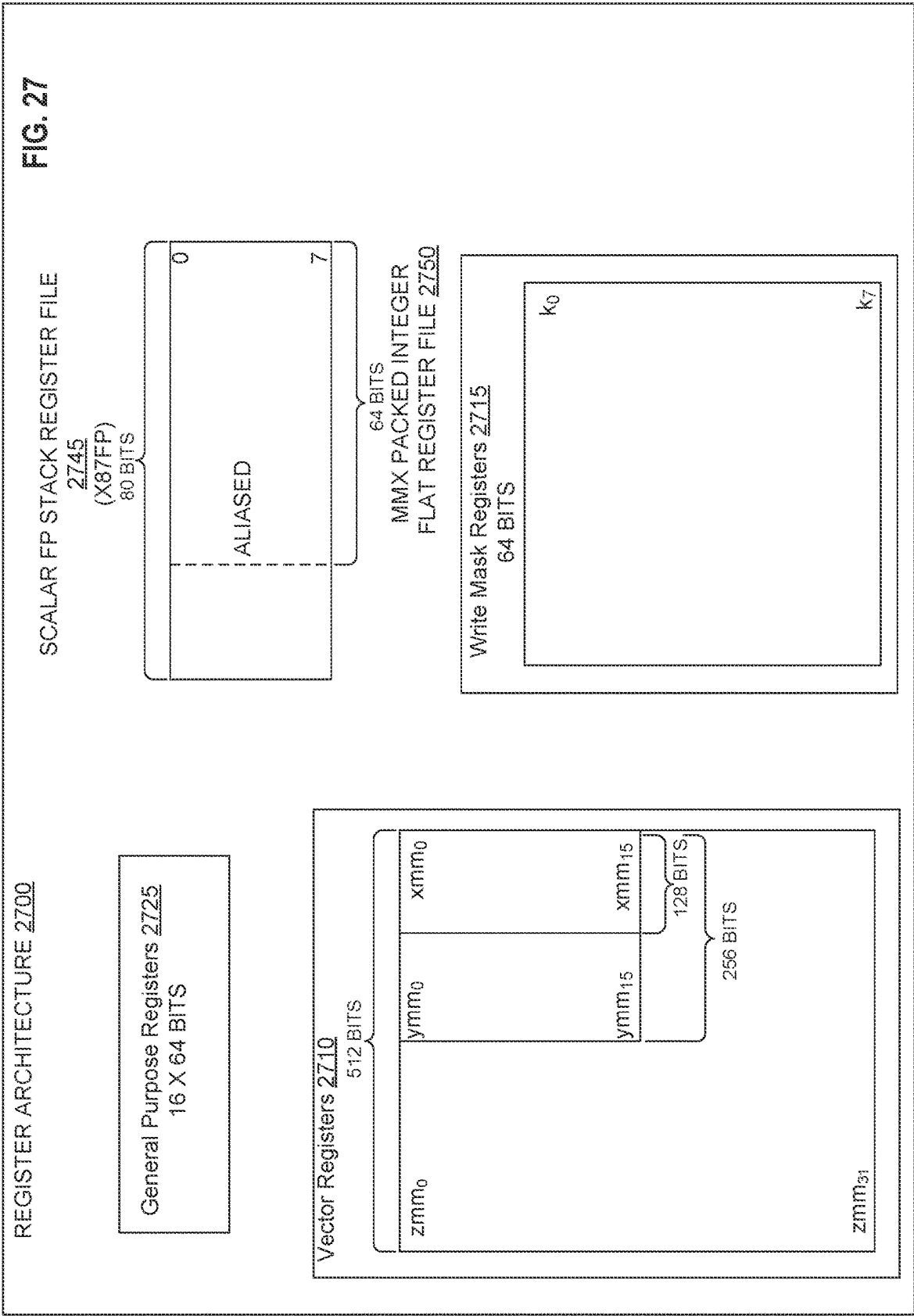


FIG. 26D





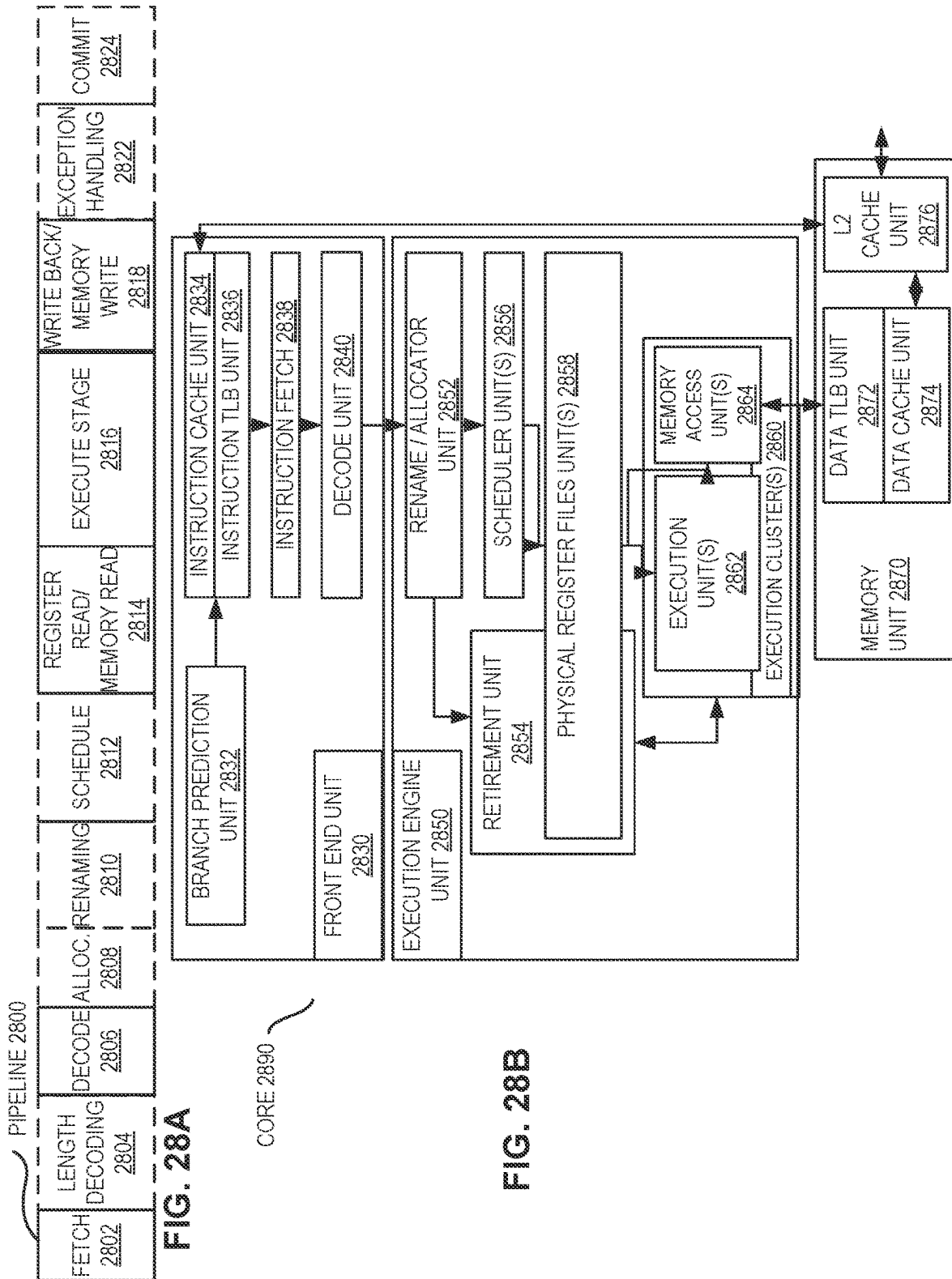


FIG. 29A

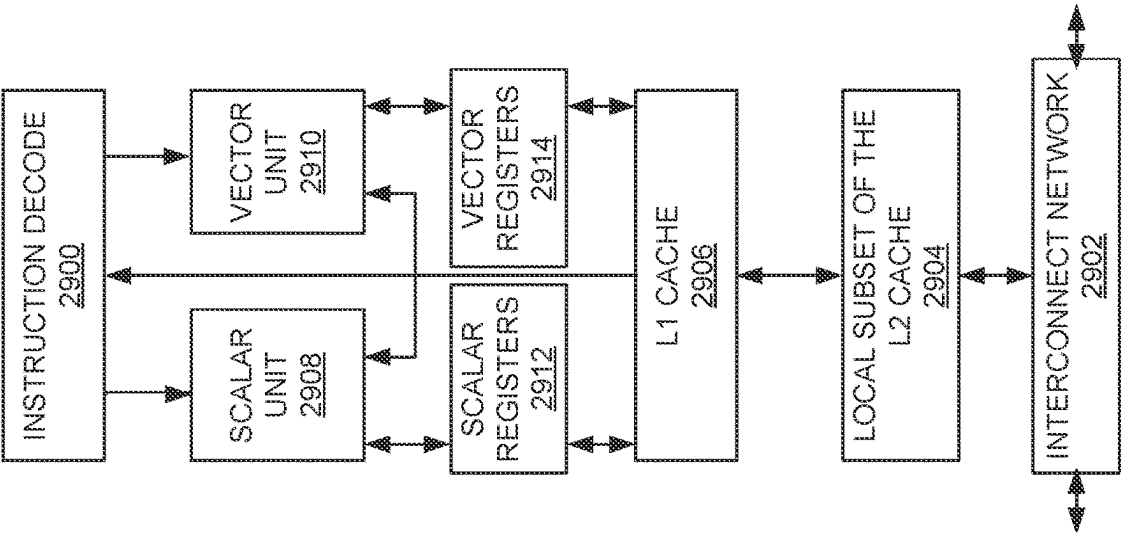
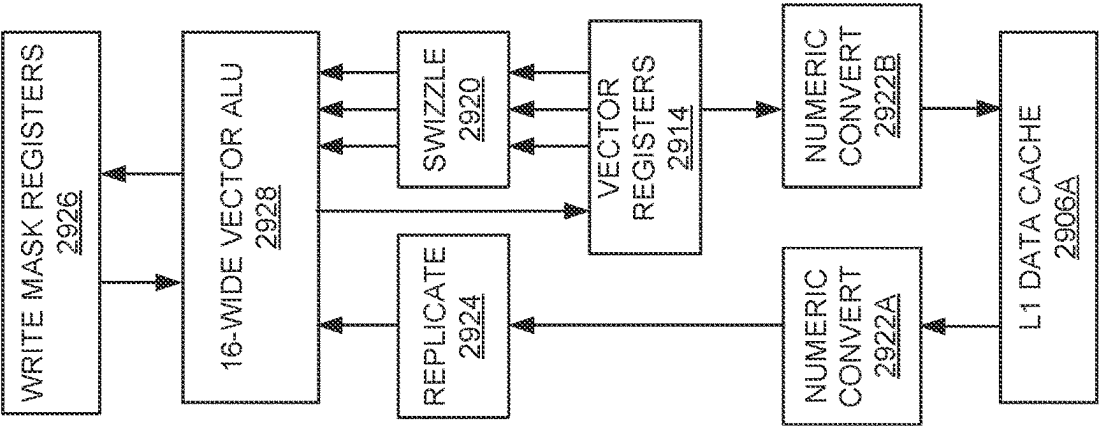
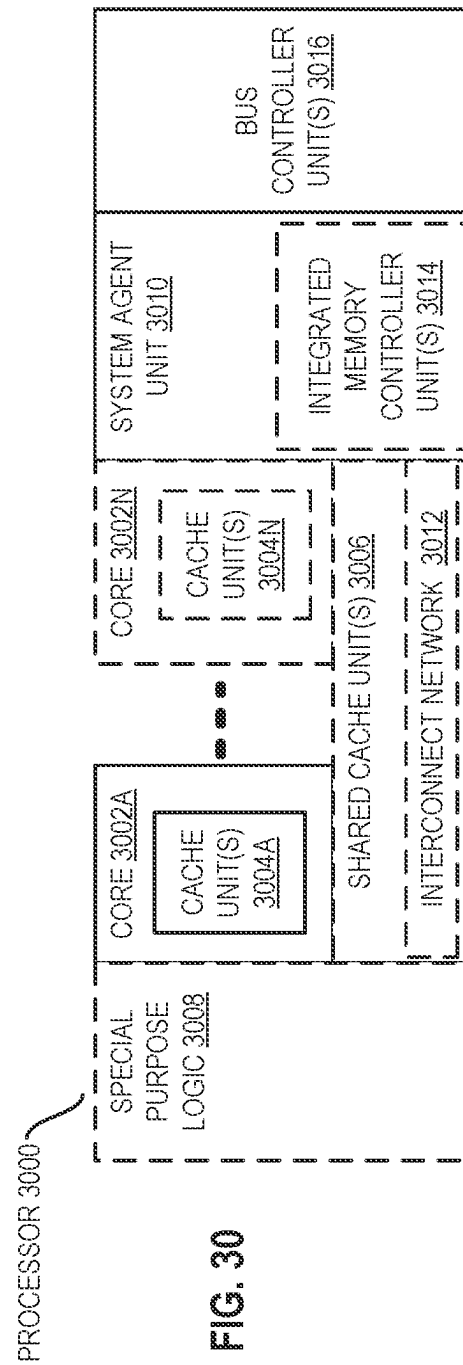


FIG. 29B





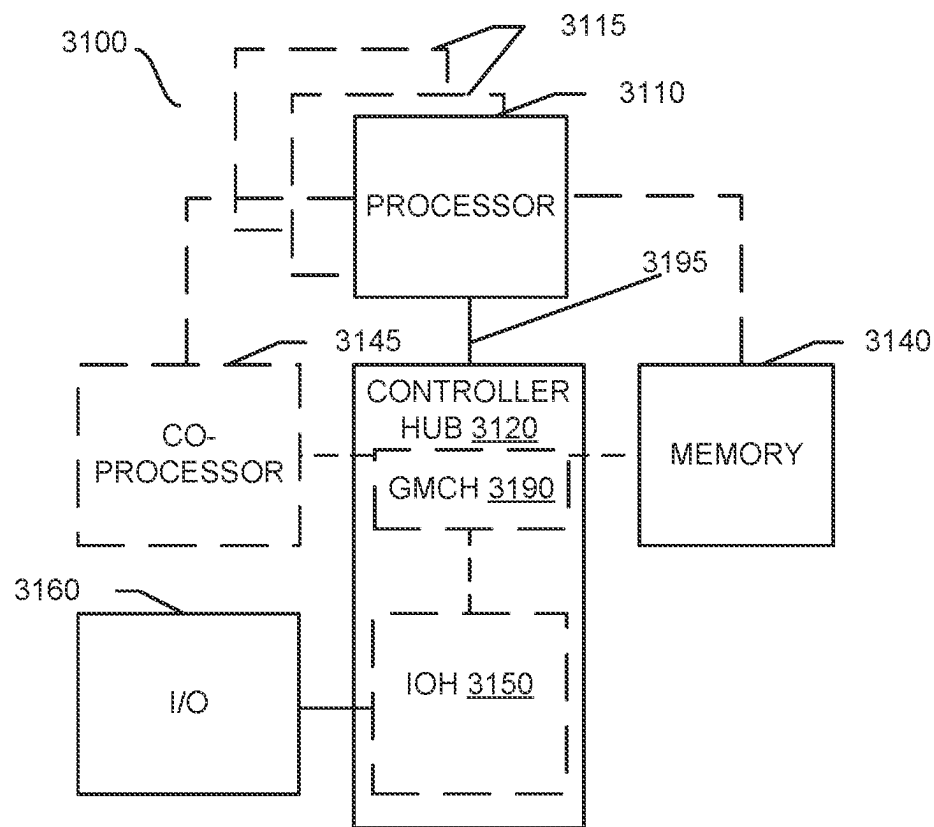


FIG. 31

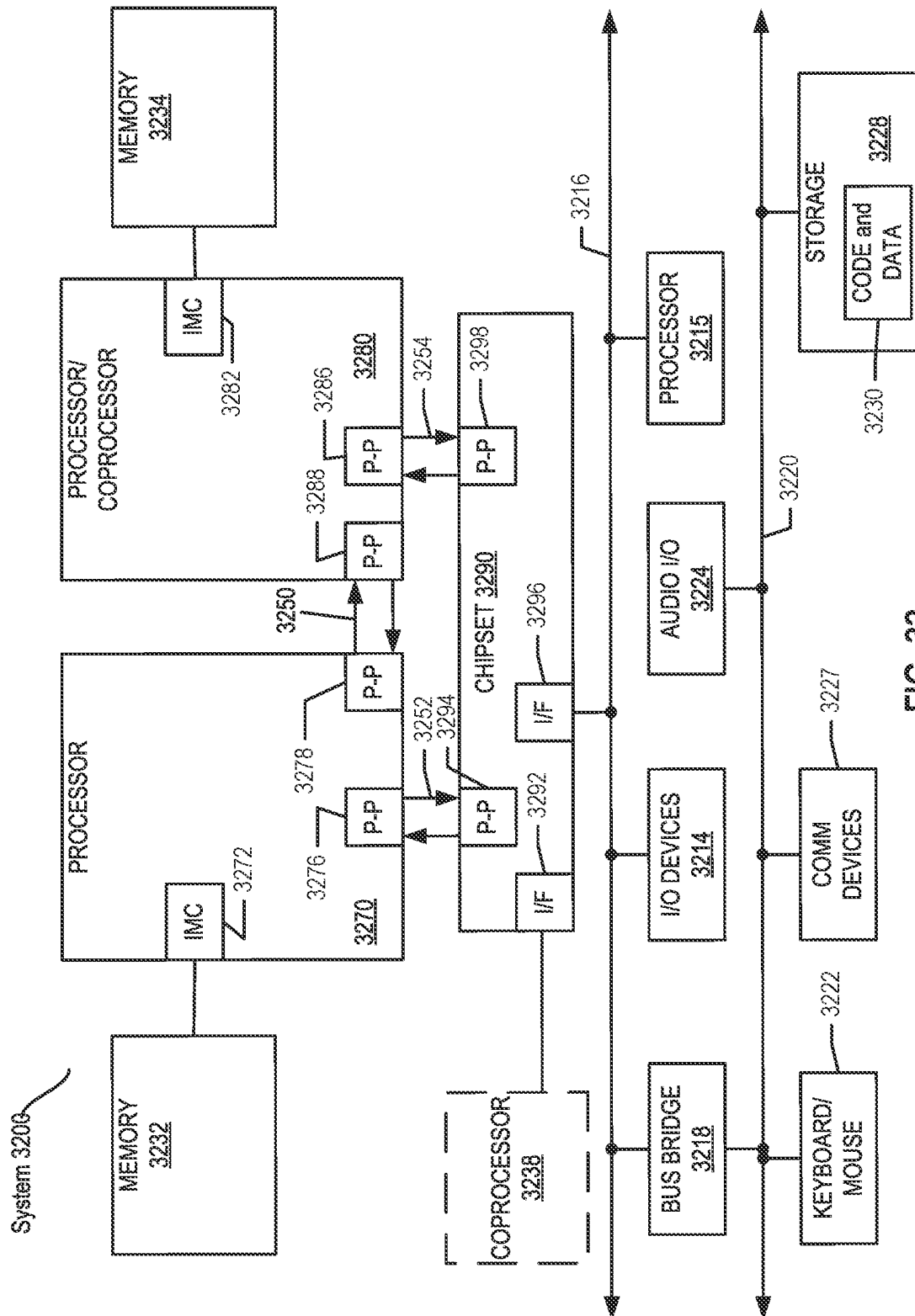


FIG. 32

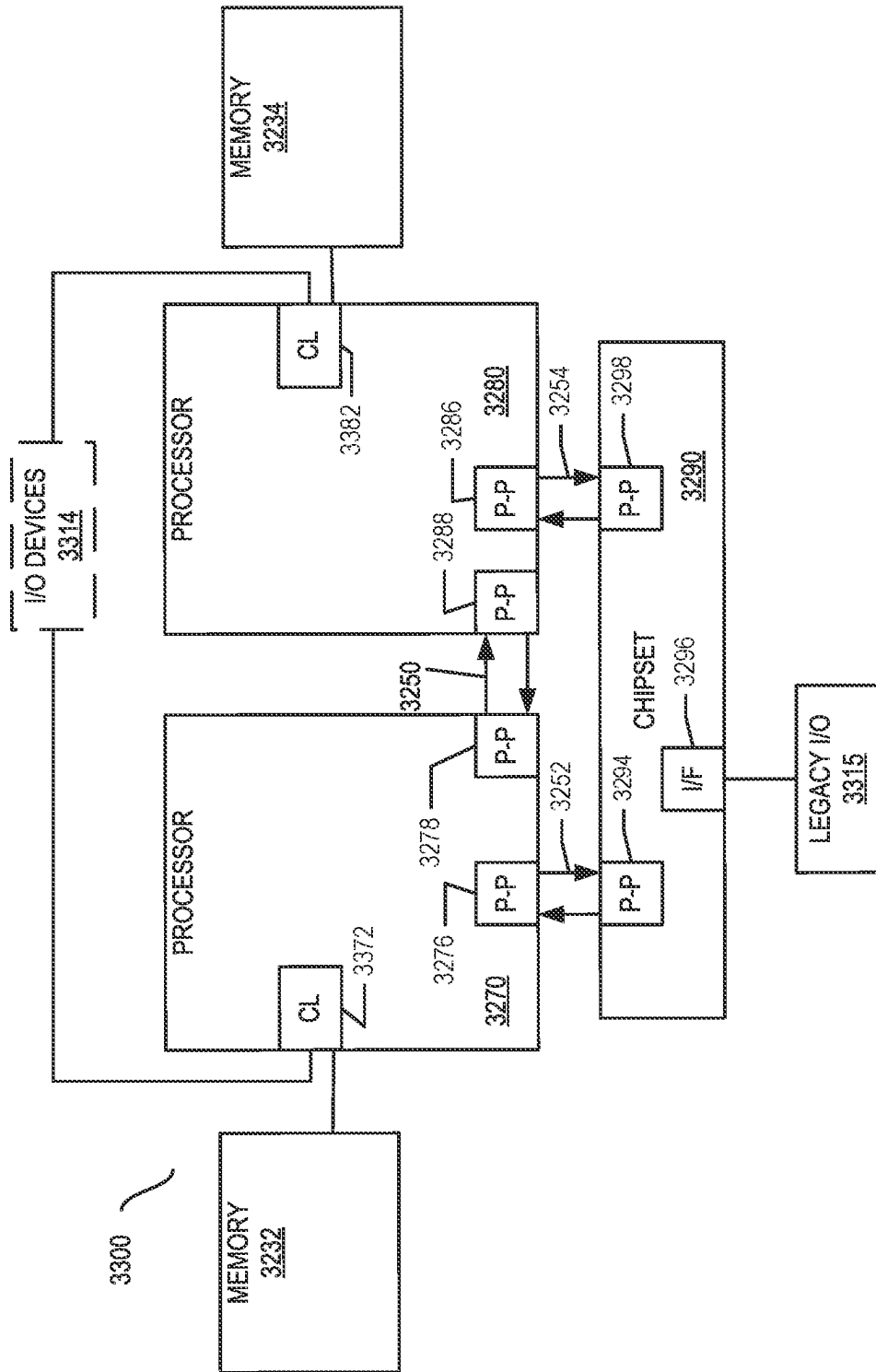


FIG. 33

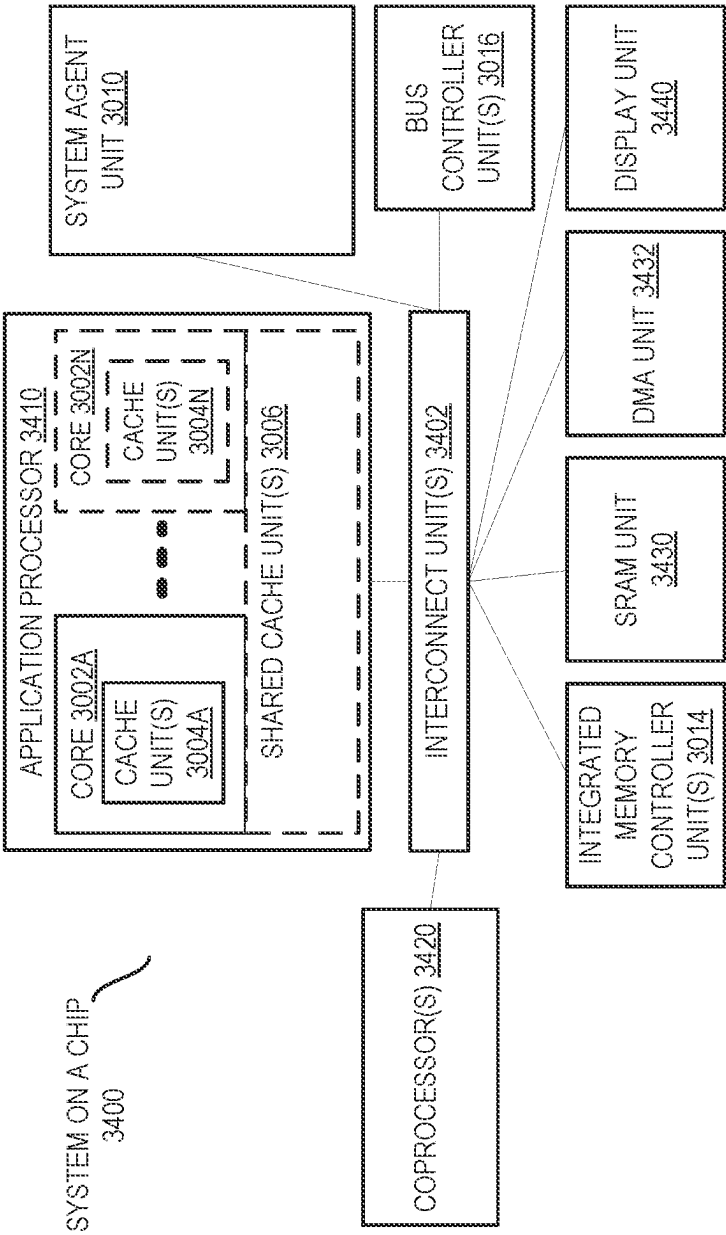
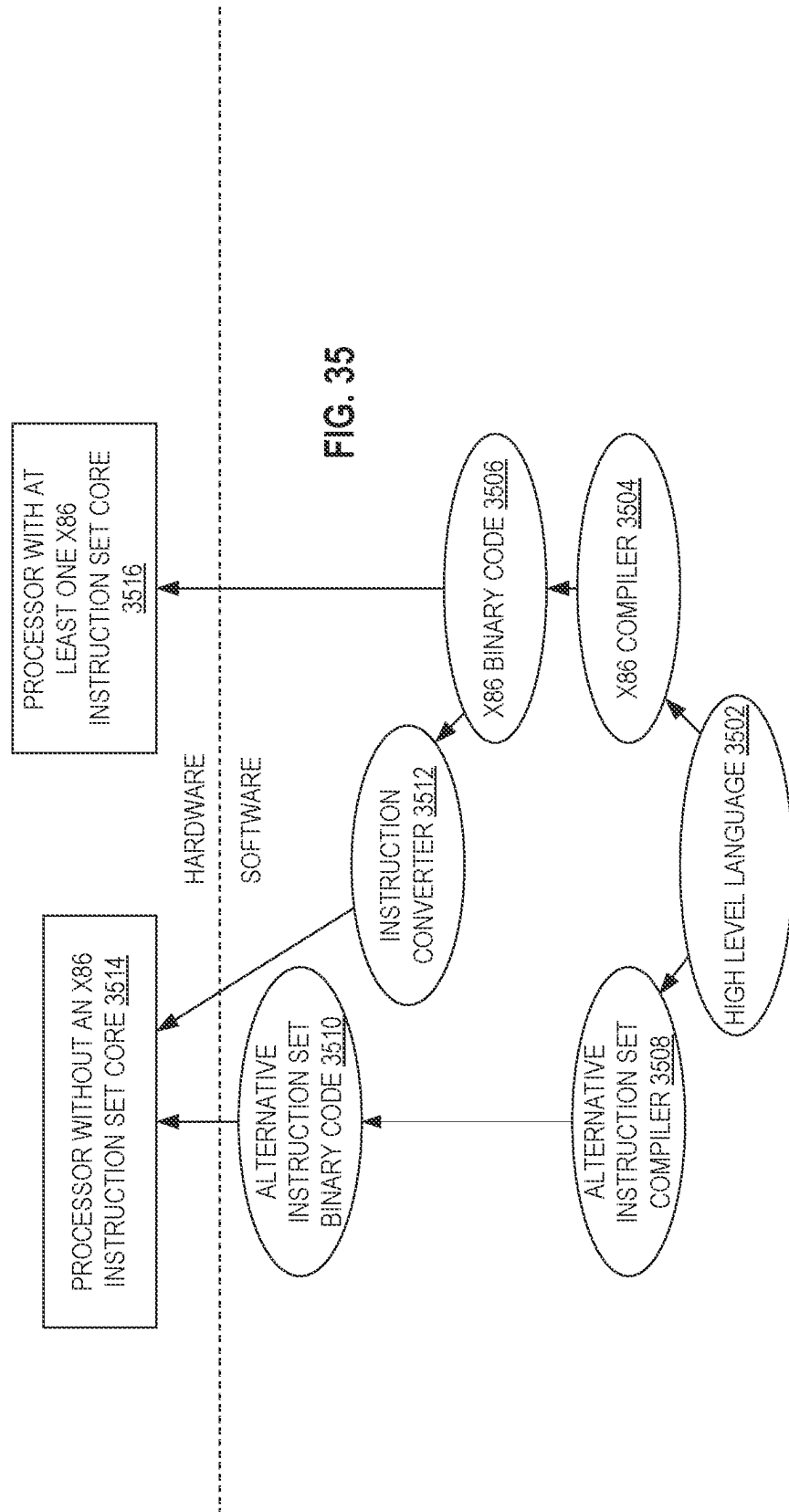


FIG. 34



MATRIX TRANSPOSE AND MULTIPLY**FIELD OF INVENTION**

The field of invention relates generally to computer processor architecture, and, more specifically, to processing matrix data.

BACKGROUND

Matrices are increasingly important in many computing tasks such as machine learning and other bulk data processing. Deep Learning is a class of machine learning algorithms. Deep learning architectures, such as deep neural networks, have been applied to fields including computer vision, speech recognition, natural language processing, audio recognition, social network filtering, machine translation, bioinformatics, and drug design.

Inference and training, two tools used for deep learning, are tending towards low precision arithmetic. Maximizing throughput of deep learning algorithms and computations may assist in meeting the needs of deep learning processors, for example, those performing deep learning in a data center.

Matrix-matrix multiplication (a.k.a., GEMM or General Matrix Multiplication) is a common compute-heavy operation on today's processors. Special hardware for matrix multiplication (e.g., GEMM) is a good option for improving the peak compute (and energy efficiency) of certain applications, such as deep learning. Some of these applications, including deep learning, may operate on input data elements with relatively few bits without losing accuracy, as long as the output elements have enough bits (i.e., more than the inputs).

BRIEF DESCRIPTION OF THE DRAWINGS

The present invention is illustrated by way of example and not limitation in the figures of the accompanying drawings, in which like references indicate similar elements and in which:

- FIG. 1A illustrates an embodiment of configured tiles;
- FIG. 1B illustrates an embodiment of configured tiles;
- FIG. 2 illustrates several examples of matrix storage;
- FIG. 3 illustrates an embodiment of a system utilizing a matrix (tile) operations accelerator;
- FIGS. 4 and 5 show different embodiments of how memory is shared using a matrix operations accelerator;
- FIG. 6 illustrates an embodiment of matrix multiply accumulate operation using tiles ("TMMA");
- FIG. 7 illustrates an embodiment of a subset of the execution of an iteration of a chained fused multiply accumulate instruction;
- FIG. 8 illustrates an embodiment of a subset of the execution of an iteration of a chained fused multiply accumulate instruction;
- FIG. 9 illustrates an embodiment of a subset of the execution of an iteration of a chained fused multiply accumulate instruction;
- FIG. 10 illustrates an embodiment of a subset of the execution of an iteration of chained fused multiply accumulate instruction;
- FIG. 11 illustrates power-of-two sized SIMD implementations wherein the accumulators use input sizes that are larger than the inputs to the multipliers according to an embodiment;
- FIG. 12 illustrates an embodiment of a system utilizing matrix operations circuitry;

FIG. 13 illustrates an embodiment of a processor core pipeline supporting matrix operations using tiles;

FIG. 14 illustrates an embodiment of a processor core pipeline supporting matrix operations using tiles;

FIG. 15 illustrates an example of a matrix expressed in row major format and column major format;

FIG. 16 illustrates an example of usage of matrices (tiles);

FIG. 17 illustrates an embodiment a method of usage of matrices (tiles);

FIG. 18 illustrates support for configuration of the usage of tiles according to an embodiment;

FIG. 19 illustrates an embodiment of a description of the matrices (tiles) to be supported;

FIGS. 20(A)-(D) illustrate examples of register(s);

FIGS. 21A, 21B, and 21C are block diagrams illustrating execution of a TILETFM2RI instruction according to some embodiments;

FIG. 22A illustrates the relationship between granularity and datatype in a source matrix transposition according to an embodiment;

FIG. 22B is a block diagram illustrating execution of a matrix transpose+multiply instruction according to some embodiments;

FIG. 23 illustrates an embodiment of a processor executing a flow to process a matrix transpose+multiply instruction;

FIG. 24 is a block diagram illustrating a format of a matrix transpose+multiply instruction according to some embodiments;

FIGS. 25A-25B are block diagrams illustrating a generic vector friendly instruction format and instruction templates thereof according to embodiments;

FIG. 25A is a block diagram illustrating a generic vector friendly instruction format and class A instruction templates thereof according to embodiments;

FIG. 25B is a block diagram illustrating the generic vector friendly instruction format and class B instruction templates thereof according to embodiments;

FIG. 26A is a block diagram illustrating an exemplary specific vector friendly instruction format according to embodiments;

FIG. 26B is a block diagram illustrating the fields of the specific vector friendly instruction format that make up the full opcode field according to one embodiment;

FIG. 26C is a block diagram illustrating the fields of the specific vector friendly instruction format that make up the register index field according to one embodiment;

FIG. 26D is a block diagram illustrating the fields of the specific vector friendly instruction format that make up the augmentation operation field according to one embodiment;

FIG. 27 is a block diagram of a register architecture according to one embodiment;

FIG. 28A is a block diagram illustrating both an exemplary in-order pipeline and an exemplary register renaming, out-of-order issue/execution pipeline according to embodiments;

FIG. 28B is a block diagram illustrating both an exemplary embodiment of an in-order architecture core and an exemplary register renaming, out-of-order issue/execution architecture core to be included in a processor according to embodiments;

FIGS. 29A-B illustrate a block diagram of a more specific exemplary in-order core architecture, which core would be one of several logic blocks (including other cores of the same type and/or different types) in a chip;

3

FIG. 29A is a block diagram of a single processor core, along with its connection to the on-die interconnect network and with its local subset of the Level 2 (L2) cache, according to embodiments;

FIG. 29B is an expanded view of part of the processor core in FIG. 29A according to embodiments;

FIG. 30 is a block diagram of a processor that may have more than one core, may have an integrated memory controller, and may have integrated graphics according to embodiments;

FIGS. 31-34 are block diagrams of exemplary computer architectures;

FIG. 31 shown a block diagram of a system in accordance with one embodiment of the present invention;

FIG. 32 is a block diagram of a first more specific exemplary system in accordance with an embodiment of the present invention;

FIG. 33 is a block diagram of a second more specific exemplary system in accordance with an embodiment of the present invention;

FIG. 34 is a block diagram of a System-on-a-Chip (SoC) in accordance with an embodiment of the present invention; and

FIG. 35 is a block diagram contrasting the use of a software instruction converter to convert binary instructions in a source instruction set to binary instructions in a target instruction set according to embodiments.

DETAILED DESCRIPTION

In the following description, numerous specific details are set forth. However, it is understood that embodiments may be practiced without these specific details. In other instances, well-known circuits, structures, and techniques have not been shown in detail in order not to obscure the understanding of this description.

References in the specification to “one embodiment,” “an embodiment,” “an example embodiment,” etc., indicate that the embodiment described may include a particular feature, structure, or characteristic, but every embodiment may not necessarily include the particular feature, structure, or characteristic. Moreover, such phrases are not necessarily referring to the same embodiment. Further, when a particular feature, structure, or characteristic is described in connection with an embodiment, it is submitted that it is within the knowledge of one skilled in the art to affect such feature, structure, or characteristic in connection with other embodiments whether or not explicitly described.

As used in this specification and the claims and unless otherwise specified, the use of the ordinal adjectives “first,” “second,” “third,” etc. to describe an element merely indicates that a particular instance of an element or different instances of like elements are being referred to, and is not intended to imply that the elements so described must be in a particular sequence, either temporally, spatially, in ranking, or in any other manner. Also, as used in descriptions of embodiments of the invention, a “/” character between terms may mean that what is described may include or be implemented using, with, and/or according to the first term and/or the second term (and/or any other additional terms).

In many mainstream processors, handling matrices is a difficult and/or instruction intensive task. For example, rows of a matrix could be put into a plurality of packed data (e.g., SIMD or vector) registers and then operated on individually. For example, an add two 8x2 matrices may require a load or gather into four packed data registers depending upon data sizes. Then a first add of packed data registers corresponding

4

to a first row from each matrix is performed and a second add of packed data registers corresponding to a second row from each matrix is performed. Then the resulting packed data registers are scattered back to memory. While for small matrices this scenario may be acceptable, it is often not acceptable with larger matrices.

Discussion

Described herein are mechanisms to support matrix operations in computer hardware such as central processing units (CPUs), graphic processing units (GPUs), and accelerators. The matrix operations utilize 2-dimensional (2-D) data structures representing one or more packed regions of memory such as registers. Throughout this description, these 2-D data structures are referred to as tiles. Note that a matrix may be smaller than a tile (use less than all of a tile) or utilize a plurality of tiles (the matrix is larger than the size of any one tile). Throughout the description, matrix (tile) language is used to indicate operations performed using tiles that impact a matrix; whether or not that matrix is larger than any one tile is not typically relevant. Architectures, operations, instructions, data, etc. to support and/or otherwise related to the processing of matrix data may be referred to, among other possibilities, as matrix, tile, and tile matrix multiply accumulate (TMMA) architectures, operations, instructions, data, etc. In some contexts, matrix and tile may be used interchangeably.

Each tile may be acted upon by different operations such as those that are detailed herein and include, but are not limited to: matrix (tile) multiplication, tile add, tile subtract, tile diagonal, tile zero, tile transform, tile dot product, tile broadcast, tile row broadcast, tile column broadcast, tile multiplication, tile multiplication and accumulation, tile move, etc. Additionally, support for operators such as the use of a scale and/or bias may be used with these operations or in support of non-numeric applications in the future, for instance, OpenCL “local memory,” data compression/decompression, etc. Also described herein are instructions for loading and storing matrix (tile) data with datatype conversion.

Portions of storage (such as memory (non-volatile and volatile), registers, cache, etc.) are arranged into tiles of different horizontal and vertical dimensions. For example, a tile may have horizontal dimension of 4 (e.g., four rows of a matrix) and a vertical dimension of 8 (e.g., 8 columns of the matrix). Typically, the horizontal dimension is related to element sizes (e.g., 2-, 4-, 8-, 16-, 32-, 64-, 128-bit, etc.). Multiple datatypes (single precision floating point, double precision floating point, integer, etc.) may be supported.

Exemplary Usage of Configured Tiles

In some embodiments, tile parameters may be configured. For example, a given tile may be configured to provide tile options. Exemplary tile options include but are not limited to: a number of rows of the tile, a number of columns of the tile, whether the tile is VALID, and whether the tile consists of a PAIR of equal-sized tiles.

FIG. 1A illustrates an embodiment of configured tiles. As shown, 4 KB of application memory 102 have stored thereon 4 1 kB tiles, tile t0 104, tile t1 106, tile t2 108, and tile t3 110. In this example, the 4 tiles do not consist of pairs, and each have elements arranged in rows and columns. Tile t0 104 and tile t1 106 have K rows and N columns of 4-byte elements (e.g., single precision data), where K equals 8 and N=32. Tile t2 108 and tile t3 110 have K rows and N/2 columns of 8-byte elements (e.g., double precision data). As the double precision operands are twice the width of single

precision, this configuration is consistent with a palette, used to provide tile options, supplying at least 4 names with total storage of at least 4 kB. In operation, the tiles may be loaded from and stored to memory using load and store operations. Depending upon the instruction encoding scheme used, the amount of available application memory, as well as the size, number, and configuration of available tiles varies.

FIG. 1B illustrates an embodiment of configured tiles. As shown, 4 kB of application memory 122 have stored thereon 2 pairs of 1 kB-tiles, the first pair being tile t4L 124 and tile t4R 126, and the second pair being tile t5L 128 and tile t5R 130. As shown the pairs of tiles are divided into a left tile and a right tile. In other embodiments, the pair of tiles are divided into an even tile and an odd tile. In this example, the 4 tiles each have elements arranged in rows and columns. Tile t4L 124 and tile t4R 126 have K rows and N columns of 4-byte elements (e.g., single precision floating point data), where K equals 8 and N equals 32. Tile t5L 128 and tile t5R 130 have K rows and N/2 columns of 8-byte elements (e.g., double precision floating point data). As the double precision operands are twice the width of single precision, this configuration is consistent with a palette, used to provide tile options, supplying at least 2 names with total storage of at least 4 kB. The four tiles of FIG. 1A use 4 names, each naming a 1 kB tile, whereas the 2 pairs of tiles in FIG. 1B may use 2 names to specify the paired tiles. In some embodiments, tile instructions accept a name of a paired tile as an operand. In operation, the tiles may be loaded from and stored to memory using load and store operations. Depending upon the instruction encoding scheme used, the amount of available application memory, as well as the size, number, and configuration of available tiles varies.

In some embodiments, tile parameters are definable. For example, a “palette” is used to provide tile options. Exemplary options include, but are not limited to: the number of tile names, the number of bytes in a row of storage, the number of rows and columns in a tile, etc. For example, a maximum “height” (number of rows) of a tile may be defined as: $\text{Tile Max Rows} = \text{Architected Storage} / (\text{The Number of Palette Names} * \text{The Number of Bytes per row})$.

As such, an application may be written such that a fixed usage of names will be able to take advantage of different storage sizes across implementations.

Configuration of tiles may be done using a tile configuration (“TILECONFIG”) instruction, where a particular tile usage is defined in a selected palette. This declaration includes the number of tile names to be used, the requested number of rows and columns per name (tile), and, in some embodiments, the requested datatype of each tile. In some embodiments, consistency checks are performed during the execution of a TILECONFIG instruction to determine that it matches the restrictions of the palette entry. Exemplary Tile Storage Types

FIG. 2 illustrates several examples of matrix storage. In (A), a tile is stored in memory. As shown, each “row” consists of four packed data elements. To get to the next “row,” a stride value is used. Note that rows may be consecutively stored in memory. Strided memory accesses allow for access of one row to the next when the tile storage does not map the underlying memory array row width.

Tile loads from memory and stores to memory are typically strided accesses from the application memory to packed rows of data. Exemplary TILELOAD and TILESTORE instructions, or other instruction references to application memory as a TILE operand in load-op instructions, are, in some embodiments, restartable to handle (up to)

2*rows of page faults, unmasked floating point exceptions, and/or interrupts per instruction.

In (B), a matrix is stored in a tile comprised of a plurality of registers such as packed data registers (single instruction, multiple data (SIMD) or vector registers). In this example, the tile is overlaid on three physical registers. Typically, consecutive registers are used, however, this need not be the case.

In (C), a matrix is stored in a tile in non-register storage accessible to a fused multiple accumulate (FMA) circuit used in tile operations. This storage may be inside of an FMA, or adjacent to it. Additionally, in some embodiments, discussed below, the storage may be for a data element and not an entire row or tile.

The supported parameters for the TMMA architecture are reported via CPUID. In some embodiments, the list of information includes a maximum height and a maximum SIMD dimension. Configuring the TMMA architecture requires specifying the dimensions for each tile, the element size for each tile and the palette identifier. This configuration is done by executing the TILECONFIG instruction.

Successful execution of a TILECONFIG instruction enables subsequent TILE operators. A TILERELLEASEALL instruction clears the tile configuration and disables the TILE operations (until the next TILECONFIG instructions executes). In some embodiments, XSAVE, XSTORE, etc. are used in context switching using tiles. In some embodiments, 2 XCR0 bits are used in XSAVE, one for TILECONFIG metadata and one bit corresponding to actual tile payload data.

TILECONFIG not only configures the tile usage, but also sets a state variable indicating that the program is in a region of code with tiles configured. An implementation may enumerate restrictions on other instructions that may be used with a tile region such as no usage of an existing register set, etc.

Exiting a tile region is typically done with the TILERELLEASEALL instruction. It takes no parameters and swiftly invalidates all tiles (indicating that the data no longer needs any saving or restoring) and clears the internal state corresponding to being in a tile region.

In some embodiments, tile operations may zero any rows and any columns beyond the dimensions specified by the tile configuration. For example, tile operations may zero the data beyond the configured number of columns (factoring in the size of the elements) as each row is written. For example, with 64-byte rows and a tile configured with 10 rows and 12 columns, an operation writing FP32 elements would write each of the first 10 rows with 12*4 bytes with output/result data and zero the remaining 4*4 bytes in each row. Tile operations also fully zero any rows after the first 10 configured rows. When using 1K tile with 64-byte rows, there would be 16 rows, so in this example, the last 6 rows would also be zeroed.

In some embodiments, a context restore instruction (e.g., XRSTOR), when loading data, enforces that the data beyond the configured rows for a tile may be maintained as zero. If there is no valid configuration, all rows are zeroed. XRSTOR of tile data may load garbage in the columns beyond those configured. It should not be possible for XRSTOR to clear beyond the number of columns configured because there is not an element width associated with the tile configuration.

Context save (e.g., XSAVE) exposes the entire TILE storage area when writing it to memory. If XRSTOR loaded garbage data into the rightmost part of a tile, that data may

be saved by XSAVE. XSAVE may write zeros for rows beyond the number specified for each tile.

In some embodiments, tile instructions are restartable. The operations that access memory allow restart after page faults. The computational instructions that deal with floating point operations also allow for unmasked floating-point exceptions, with the masking of the exceptions controlled by a control and/or status register.

To support restarting instructions after these events, the instructions store information in the start registers detailed below.

Matrix (Tile) Operation Systems Exemplary Hardware Support

FIG. 3 illustrates an embodiment of a system utilizing a matrix (tile) operations accelerator. In this illustration, a host processor/processing system 301 communicates commands 311 (e.g., matrix manipulation operations such as arithmetic or matrix manipulation operations, or load and store operations) to a matrix operations accelerator 307. However, this is shown this way for discussion purposes only. As detailed later, this accelerator 307 may be a part of a processing core. Typically, commands 311 that are tile manipulation operator instructions may refer to tiles as register-register (“reg-reg”) or register-memory (“reg-mem”) format. Other commands such as TILESTORE, TILELOAD, TILECONFIG, etc., do not perform data operations on a tile. Commands may be decoded instructions (e.g., micro-ops) or macro-instructions for the accelerator 307 to handle.

In this example, a coherent memory interface 303 is coupled to the host processor/processing system 301 and matrix operations accelerator 307 such that they may share memory. FIGS. 4 and 5 show different embodiments of how memory is shared using a matrix operations accelerator. As shown in FIG. 4, the host processor 401 and matrix operations accelerator circuitry 405 share the same memory 403. FIG. 5 illustrates an embodiment where the host processor 501 and matrix operations accelerator 505 do not share memory but may access each other’s memory. For example, processor 501 may access tile memory 507 and utilize its host memory 503 as normal. Similarly, the matrix operations accelerator 505 may access host memory 503, but more typically uses its own memory 507. Note these memories may be of different types.

In some embodiments, tiles are supported using an overlay over physical registers. For example, a tile may utilize 16 1,024-bit registers, 32 512-bit registers, etc. depending on the implementation. In some embodiments, the matrix operations utilize 2-dimensional (2-D) data structures representing one or more packed regions of memory such as registers. Throughout this description, these 2-D data structures are referred to as tiles or tile registers.

In some embodiments, the matrix operations accelerator 307 includes a plurality of FMAs 309 coupled to data buffers 305 (in some implementations, one or more of these buffers 305 are stored in the FMAs of the grid as shown). The data buffers 305 buffer tiles loaded from memory and/or tiles to be stored to memory (e.g., using a tileload or tilestore instruction). Data buffers may be, for example, a plurality of registers. Typically, these FMAs are arranged as a grid of chained FMAs 309 which are able to read and write tiles. In this example, the matrix operations accelerator 307 is to perform a matrix multiply operation using tiles T0, T1, and T2. At least one of the tiles may be housed in the FMA grid 309. In some embodiments, all tiles in an operation are stored in the FMA grid 309. In other embodiments, only a subset is stored in the FMA grid 309. As shown, T1 is housed

and T0 and T2 are not. Note that A, B, and C refer to the matrices of these tiles which may or may not take up the entire space of the tile.

FIG. 6 illustrates an embodiment of matrix multiply accumulate operation using tiles (“TMMA”).

The number of rows in the matrix (TILE A 601) matches the number of serial (chained) FMAs comprising the computation’s latency. An implementation is free to recirculate on a grid of smaller height, but the computation remains the same.

The source/destination vector comes from a tile of N rows (TILE C 605) and the grid of FMAs 611 performs N vector-matrix operations resulting in a complete instruction performing a matrix multiplication of tiles. Tile B 603 is the other vector source and supplies “broadcast” terms to the FMAs in each stage.

In operation, in some embodiments, the elements of matrix B (stored in a tile B 603) are spread across the rectangular grid of FMAs. Matrix B (stored in tile A 601) has its elements of a row transformed to match up with the columnar dimension of the rectangular grid of FMAs. At each FMA in the grid, an element of A and B are multiplied and added to the incoming summand (from above in the Figure) and the outgoing sum is passed to the next row of FMAs (or the final output).

The latency of a single step is proportional to K (row height of matrix B) and dependent TMMA’s typically have enough source-destination rows (either in a single tile or across tile) to hide that latency. An implementation may also split the SIMD (packed data element) dimension M (row height of matrix A) across time steps, but this simply changes the constant that K is multiplied by. When a program specifies a smaller K than the maximum enumerated by the TMACC, an implementation is free to implement this with “masking” or “early outs.”

The latency of an entire TMMA is proportional to $N \times K$. The repeat rate is proportional to N. The number of MACs per TMMA instruction is $N \times K \times M$.

FIG. 7 illustrates an embodiment of a subset of the execution of an iteration of a chained fused multiply accumulate instruction. In particular, this illustrates execution circuitry of an iteration of one packed data element position of the destination. In this embodiment, the chained fused multiply accumulate is operating on signed sources wherein the accumulator is $2 \times$ the input data size.

A first signed source (source 1 701) and a second signed source (source 2 703) each have four packed data elements. Each of these packed data elements stores signed data such as floating-point data. A third signed source (source 3 709) has two packed data elements, each of which stores signed data. The sizes of the first and second signed sources 701 and 703 are half that of the third signed source (initial value or previous result) 709. For example, the first and second signed sources 701 and 703 could have 32-bit packed data elements (e.g., single precision floating point) while the third signed source 709 could have 64-bit packed data elements (e.g., double precision floating point).

In this illustration, only the two most significant packed data element positions of the first and second signed sources 701 and 703 and the most significant packed data element position of the third signed source 709 are shown. Of course, the other packed data element positions would also be processed.

As illustrated, packed data elements are processed in pairs. For example, the data of the most significant packed data element positions of the first and second signed sources 701 and 703 are multiplied using a multiplier circuit 705,

and the data from second most significant packed data element positions of the first and second signed sources **701** and **703** are multiplied using a multiplier circuit **707**. In some embodiments, these multiplier circuits **705** and **707** are reused for other packed data elements positions. In other 5 embodiments, additional multiplier circuits are used so that the packed data elements are processed in parallel. In some contexts, parallel execution is done using lanes that are the size of the signed third source **709**. The results of each of the multiplications are added using addition circuitry **711**.

The result of the addition of the results of the multiplications is added to the data from most significant packed data element position of the signed source **3 709** (using a different adder **713** or the same adder **711**).

Finally, the result of the second addition is either stored into the signed destination **715** in a packed data element position that corresponds to the packed data element position used from the signed third source **709** or passed on to the next iteration if there is one. In some embodiments, a writemask is applied to this storage such that if a corresponding writemask (bit) is set, the storage happens, and, if not set, the storage does not happen.

FIG. **8** illustrates an embodiment of a subset of the execution of an iteration of a chained fused multiply accumulate instruction. In particular, this illustrates execution 25 circuitry of an iteration of one packed data element position of the destination. In this embodiment, the chained fused multiply accumulate is operating on signed sources wherein the accumulator is 2× the input data size.

A first signed source (source **1 801**) and a second signed source (source **2 803**) each have four packed data elements. Each of these packed data elements stores signed data such as integer data. A third signed source (source **3 809**) has two packed data elements, each of which stores signed data. The sizes of the first and second signed sources **801** and **803** are half that of the third signed source **809**. For example, the first and second signed sources **801** and **803** could have 32-bit packed data elements (e.g., single precision floating point) the third signed source **809** could have 64-bit packed data elements (e.g., double precision floating point).

In this illustration, only the two most significant packed data element positions of the first and second signed sources **801** and **803** and the most significant packed data element position of the third signed source **809** are shown. Of course, the other packed data element positions would also be processed.

As illustrated, packed data elements are processed in pairs. For example, the data of the most significant packed data element positions of the first and second signed sources **801** and **803** are multiplied using a multiplier circuit **805**, 50 and the data from second most significant packed data element positions of the first and second signed sources **801** and **803** are multiplied using a multiplier circuit **807**. In some embodiments, these multiplier circuits **805** and **807** are reused for other packed data elements positions. In other 55 embodiments, additional multiplier circuits are used so that the packed data elements are processed in parallel. In some contexts, parallel execution is done using lanes that are the size of the signed third source (initial value or previous iteration result) **809**. The results of each of the multiplications are added to the signed third source **809** using addition/saturation circuitry **813**.

Addition/saturation (accumulator) circuitry **813** preserves a sign of an operand when the addition results in a value that is too big. In particular, saturation evaluation occurs on the 65 infinite precision result between the multi-way-add and the write to the destination or next iteration. When the accumu-

lator **813** is floating point and the input terms are integer, the sum of products and the floating-point accumulator input value are turned into infinite precision values (fixed point numbers of hundreds of bits), the addition of the multiplication results and the third input is performed, and a single rounding to the actual accumulator type is performed.

Unsigned saturation means the output values are limited to a maximum unsigned number for that element width (all 1s). Signed saturation means a value is limited to the be in the range between a minimum negative number and a max positive number for that element width (for bytes for example, the range is from -128 ($=-2^7$) to 127 ($=2^7-1$)).

The result of the addition and saturation check is stored into the signed result **815** in a packed data element position that corresponds to the packed data element position used from the signed third source **809** or passed on to the next iteration if there is one. In some embodiments, a writemask is applied to this storage such that if a corresponding writemask (bit) is set, the storage happens, and, if not set, the storage does not happen.

FIG. **9** illustrates an embodiment of a subset of the execution of an iteration of a chained fused multiply accumulate instruction. In particular, this illustrates execution 25 circuitry of an iteration of one packed data element position of the destination. In this embodiment, the chained fused multiply accumulate is operating on a signed source and an unsigned source wherein the accumulator is 4× the input data size.

A first signed source (source **1 901**) and a second unsigned source (source **2 903**) each have four packed data elements. Each of these packed data elements has data such as floating point or integer data. A third signed source (initial value or result **915**) has a packed data element of which stores signed data. The sizes of the first and second sources **901** and **903** are a quarter of the third signed source **915**. For example, the first and second sources **901** and **903** could have 16-bit packed data elements (e.g., word) and the third signed source **915** could have 64-bit packed data elements (e.g., double precision floating point or 64-bit integer).

In this illustration, the four most significant packed data element positions of the first and second sources **901** and **903** and the most significant packed data element position of the third signed source **915** are shown. Of course, other packed data element positions would also be processed if there are any.

As illustrated, packed data elements are processed in quadruplets. For example, the data of the most significant packed data element positions of the first and second sources **901** and **903** are multiplied using a multiplier circuit **905**, 50 data from second most significant packed data element positions of the first and second sources **901** and **903** are multiplied using a multiplier circuit **907**, data from third most significant packed data element positions of the first and second sources **901** and **903** are multiplied using a multiplier circuit **909**, and data from the least significant packed data element positions of the first and second sources **901** and **903** are multiplied using a multiplier circuit **911**. In some embodiments, the signed packed data elements of the first source **901** are sign extended and the unsigned packed data elements of the second source **903** are zero extended prior to the multiplications.

In some embodiments, these multiplier circuits **905-911** are reused for other packed data elements positions. In other 65 embodiments, additional multiplier circuits are used so that the packed data elements are processed in parallel. In some contexts, parallel execution is done using lanes that are the

11

size of the signed third source **915**. The results of each of the multiplications are added using addition circuitry **913**.

The result of the addition of the results of the multiplications is added to the data from most significant packed data element position of the signed source **3 915** (using a different adder **917** or the same adder **913**).

Finally, the result **919** of the second addition is either stored into the signed destination in a packed data element position that corresponds to the packed data element position used from the signed third source **915** or passed to the next iteration. In some embodiments, a writemask is applied to this storage such that if a corresponding writemask (bit) is set, the storage happens, and, if not set, the storage does not happen.

FIG. **10** illustrates an embodiment of a subset of the execution of an iteration of chained fused multiply accumulate instruction. In particular, this illustrates execution circuitry of an iteration of one packed data element position of the destination. In this embodiment, the chained fused multiply accumulate is operating on a signed source and an unsigned source wherein the accumulator is 4× the input data size.

A first signed source **1001** and a second unsigned source **1003** each have four packed data elements. Each of these packed data elements stores data such as floating point or integer data. A third signed source **1015** (initial or previous result) has a packed data element of which stores signed data. The sizes of the first and second sources are a quarter of the third signed source **1015** (initial or previous result). For example, the first and second sources could have 16-bit packed data elements (e.g., word) and the third signed source **1015** (initial or previous result) could have 64-bit packed data elements (e.g., double precision floating point or 64-bit integer).

In this illustration, the four most significant packed data element positions of the first signed source **1001** and the second unsigned source **1003** and the most significant packed data element position of the third signed source **1015** are shown. Of course, other packed data element positions would also be processed if there are any.

As illustrated, packed data elements are processed in quadruplets. For example, the data of the most significant packed data element positions of the first signed source **1001** and the second unsigned source **1003** are multiplied using a multiplier circuit **1005**, data from second most significant packed data element positions of the first signed source **1001** and the second unsigned source **1003** are multiplied using a multiplier circuit **1007**, data from third most significant packed data element positions of the first signed source **1001** and the second unsigned source **1003** are multiplied using a multiplier circuit **1009**, and data from the least significant packed data element positions of the first signed source **1001** and the second unsigned source **1003** are multiplied using a multiplier circuit **1011**. In some embodiments, the signed packed data elements of the first signed source **1001** are sign extended and the unsigned packed data elements of the second unsigned source **1003** are zero extended prior to the multiplications.

In some embodiments, these multiplier circuits **1005-1011** are reused for other packed data elements positions. In other embodiments, additional multiplier circuits are used so that the packed data elements are processed in parallel. In some contexts, parallel execution is done using lanes that are the size of third signed source **1015** (initial or previous result). The result of the addition of the results of the multiplications is added to the data from most significant packed data

12

element position of third signed source **1015** (initial or previous result) using adder/saturation **1013** circuitry.

Addition/saturation (accumulator) circuitry **1013** preserves a sign of an operand when the addition results in a value that is too big or too small for signed saturation. In particular, saturation evaluation occurs on the infinite precision result between the multi-way-add and the write to the destination. When the accumulator **1013** is floating point and the input terms are integer, the sum of products and the floating-point accumulator input value are turned into infinite precision values (fixed point numbers of hundreds of bits), the addition of the multiplication results and the third input is performed, and a single rounding to the actual accumulator type is performed.

The result **1019** of the addition and saturation check is stored into the signed destination in a packed data element position that corresponds to the packed data element position used from third signed source **1015** (initial or previous result) or passed to the next iteration. In some embodiments, a writemask is applied to this storage such that if a corresponding writemask (bit) is set, the storage happens, and, if not set, the storage does not happen.

FIG. **11** illustrates power-of-two sized SIMD implementations wherein the accumulators use input sizes that are larger than the inputs to the multipliers according to an embodiment. Note the source (to the multipliers) and accumulator values may be signed or unsigned values. For an accumulator having 2× input sizes (in other words, the accumulator input value is twice the size of the packed data element sizes of the sources), table **1101** illustrates different configurations. For byte sized sources, the accumulator uses word or half-precision floating-point (HPFP) values that are 16-bit in size. For word sized sources, the accumulator uses 32-bit integer or single-precision floating-point (SPFP) values that are 32-bit in size. For SPFP or 32-bit integer sized sources, the accumulator uses 64-bit integer or double-precision floating-point (DPFP) values that are 64-bit in size.

For an accumulator having 4× input sizes (in other words, the accumulator input value is four times the size of the packed data element sizes of the sources), table **1103** illustrates different configurations. For byte sized sources, the accumulator uses 32-bit integer or single-precision floating-point (SPFP) values that are 32-bit in size. For word sized sources, the accumulator uses 64-bit integer or double-precision floating-point (DPFP) values that are 64-bit in size in some embodiments.

For an accumulator having 8× input sizes (in other words, the accumulator input value is eight times the size of the packed data element sizes of the sources), table **1105** illustrates a configuration. For byte sized sources, the accumulator uses 64-bit integer.

As hinted at earlier, matrix operations circuitry may be included in a core, or as an external accelerator. FIG. **12** illustrates an embodiment of a system utilizing matrix operations circuitry. In this illustration, multiple entities are coupled with a ring interconnect **1245**.

A plurality of cores, core 0 **1201**, core 1 **1203**, core 2 **1205**, and core N **1207** provide non-tile-based instruction support. In some embodiments, matrix operations circuitry **1251** is provided in a core **1203**, and in other embodiments matrix operations circuitry **1211** and **1213** are accessible on the ring interconnect **1245**.

Additionally, one or more memory controllers **1223-1225** are provided to communicate with memory **1233** and **1231** on behalf of the cores and/or matrix operations circuitry.

FIG. **13** illustrates an embodiment of a processor core pipeline supporting matrix operations using tiles. Branch

prediction and decode circuitry **1303** performs branch predicting of instructions, decoding of instructions, and/or both from instructions stored in instruction storage **1301**. For example, instructions detailed herein may be stored in instruction storage. In some implementations, separate circuitry is used for branch prediction and in some embodiments, at least some instructions are decoded into one or more micro-operations, micro-code entry points, microinstructions, other instructions, or other control signals using microcode **1305**. The branch prediction and decode circuitry **1303** may be implemented using various different mechanisms. Examples of suitable mechanisms include, but are not limited to, look-up tables, hardware implementations, programmable logic arrays (PLAs), microcode read only memories (ROMs), etc.

The branch prediction and decode circuitry **1303** is coupled to allocate/rename **1307** circuitry which is coupled, in some embodiments, to scheduler circuitry **1309**. In some embodiments, these circuits provide register renaming, register allocation, and/or scheduling functionality by performing one or more of: 1) renaming logical operand values to physical operand values (e.g., a register alias table in some embodiments), 2) allocating status bits and flags to the decoded instruction, and 3) scheduling the decoded instruction for execution on execution circuitry out of an instruction pool (e.g., using a reservation station in some embodiments).

The scheduler circuitry **1309** represents any number of different schedulers, including reservations stations, central instruction window, etc. The scheduler circuitry **1309** is coupled to, or includes, physical register file(s) **1315**. Each of the physical register file(s) **1315** represents one or more physical register files, different ones of which store one or more different data types, such as scalar integer, scalar floating point, packed integer, packed floating point, vector integer, vector floating point, status (e.g., an instruction pointer that is the address of the next instruction to be executed), tiles, etc. In one embodiment, the physical register file(s) **1315** comprises vector registers circuitry, write mask registers circuitry, and scalar registers circuitry. These register circuits may provide architectural vector registers, vector mask registers, and general-purpose registers. The physical register file(s) **1315** is overlapped by a retirement circuit **1317** to illustrate various ways in which register renaming and out-of-order execution may be implemented (e.g., using a reorder buffer(s) and a retirement register file(s); using a future file(s), a history buffer(s), and a retirement register file(s); using a register maps and a pool of registers; etc.). The retirement circuit **1317** and the physical register file(s) **1315** are coupled to the execution circuitry **1311**.

While register renaming is described in the context of out-of-order execution, it should be understood that register renaming may be used in an in-order architecture. While the illustrated embodiment of the processor may also include separate instruction and data cache units and a shared L2 cache unit, alternative embodiments may have a single internal cache for both instructions and data, such as, for example, a Level 1 (L1) internal cache, or multiple levels of internal cache. In some embodiments, the system may include a combination of an internal cache and an external cache that is external to the core and/or the processor. Alternatively, all of the cache may be external to the core and/or the processor.

The execution circuitry **1311** is a set of one or more execution circuits, including scalar circuitry **1321**, vector/SIMD circuitry **1323**, and matrix operations circuitry **1327**,

as well as memory access circuitry **1325** to access cache **1313**. The execution circuits perform various operations (e.g., shifts, addition, subtraction, multiplication) and on various types of data (e.g., scalar floating point, packed integer, packed floating point, vector integer, vector floating point). While some embodiments may include a number of execution units dedicated to specific functions or sets of functions, other embodiments may include only one execution unit or multiple execution units that all perform all functions. The scalar circuitry **1321** performs scalar operations, the vector/SIMD circuitry **1323** performs vector/SIMD operations, and matrix operations circuitry **1327** performs matrix (tile) operations detailed herein.

By way of example, the exemplary register renaming, out-of-order issue/execution core architecture may implement a pipeline as follows: 1) an instruction fetch circuit performs fetch and length decoding stages; 2) the branch and decode circuitry **1303** performs a decode stage; 3) the allocate/rename **1307** circuitry performs an allocation stage and renaming stage; 4) the scheduler circuitry **1309** performs a schedule stage; 5) physical register file(s) (coupled to, or included in, the scheduler circuitry **1309** and allocate/rename **1307** circuitry and a memory unit perform a register read/memory read stage; the execution circuitry **1311** performs an execute stage; 6) a memory unit and the physical register file(s) unit(s) perform a write back/memory write stage; 7) various units may be involved in the exception handling stage; and 8) a retirement unit and the physical register file(s) unit(s) perform a commit stage.

The core may support one or more instructions sets (e.g., the x86 instruction set (with some extensions that have been added with newer versions); the MIPS instruction set of MIPS Technologies of Sunnyvale, CA; the ARM instruction set (with optional additional extensions such as NEON) of ARM Holdings of Sunnyvale, CA), including the instruction(s) described herein. In one embodiment, the core **1390** includes logic to support a packed data instruction set extension (e.g., AVX1, AVX2), thereby allowing the operations used by many multimedia applications to be performed using packed data.

It should be understood that the core may support multithreading (executing two or more parallel sets of operations or threads), and may do so in a variety of ways including time sliced multithreading, simultaneous multithreading (where a single physical core provides a logical core for each of the threads that physical core is simultaneously multithreading), or a combination thereof (e.g., time sliced fetching and decoding and simultaneous multithreading thereafter such as in the Intel® Hyperthreading technology).

FIG. **14** illustrates an embodiment of a processor core pipeline supporting matrix operations using tiles. Branch prediction and decode circuitry **1403** performs branch predicting of instructions, decoding of instructions, and/or both from instructions stored in instruction storage **1401**. For example, instructions detailed herein may be stored in instruction storage. In some implementations, separate circuitry is used for branch prediction and in some embodiments, at least some instructions are decoded into one or more micro-operations, micro-code entry points, microinstructions, other instructions, or other control signals using microcode **1405**. The branch prediction and decode circuitry **1403** may be implemented using various different mechanisms. Examples of suitable mechanisms include, but are not limited to, look-up tables, hardware implementations, programmable logic arrays (PLAs), microcode read only memories (ROMs), etc.

15

The branch prediction and decode circuitry **1403** is coupled to allocate/rename **1407** circuitry which is coupled, in some embodiments, to scheduler circuitry **1409**. In some embodiments, these circuits provide register renaming, register allocation, and/or scheduling functionality by performing one or more of: 1) renaming logical operand values to physical operand values (e.g., a register alias table in some embodiments), 2) allocating status bits and flags to the decoded instruction, and 3) scheduling the decoded instruction for execution on execution circuitry out of an instruction pool (e.g., using a reservation station in some embodiments).

The scheduler circuitry **1409** represents any number of different schedulers, including reservations stations, central instruction window, etc. The scheduler unit(s) scheduler circuitry **1409** is coupled to, or includes, physical register file(s) **1415**. Each of the physical register file(s) **1415** represents one or more physical register files, different ones of which store one or more different data types, such as scalar integer, scalar floating point, packed integer, packed floating point, vector integer, vector floating point, status (e.g., an instruction pointer that is the address of the next instruction to be executed), tiles, etc. In one embodiment, the physical register file(s) **1415** comprises vector registers circuitry, write mask registers circuitry, and scalar registers circuitry. These register circuits may provide architectural vector registers, vector mask registers, and general-purpose registers. The physical register file(s) **1415** is overlapped by a retirement circuit **1417** to illustrate various ways in which register renaming and out-of-order execution may be implemented (e.g., using a reorder buffer(s) and a retirement register file(s); using a future file(s), a history buffer(s), and a retirement register file(s); using a register maps and a pool of registers; etc.). The retirement circuit **1417** and the physical register file(s) **1415** are coupled to the execution circuitry **1411**.

While register renaming is described in the context of out-of-order execution, it should be understood that register renaming may be used in an in-order architecture. While the illustrated embodiment of the processor may also include separate instruction and data cache units and a shared L2 cache unit, alternative embodiments may have a single internal cache for both instructions and data, such as, for example, a Level 1 (L1) internal cache, or multiple levels of internal cache. In some embodiments, the system may include a combination of an internal cache and an external cache that is external to the core and/or the processor. Alternatively, all of the cache may be external to the core and/or the processor.

The execution circuitry **1411** a set of one or more execution circuits **1427** and a set of one or more memory access circuits **1425** to access cache **1413**. The execution circuits **1427** perform matrix (tile) operations detailed herein.

By way of example, the exemplary register renaming, out-of-order issue/execution core architecture may implement a pipeline as follows: 1) an instruction fetch circuit performs fetch and length decoding stages; 2) the branch and decode circuitry **1403** performs a decode stage; 3) the allocate/rename **1407** circuitry performs an allocation stage and renaming stage; 4) the scheduler circuitry **1409** performs a schedule stage; 5) physical register file(s) (coupled to, or included in, the scheduler circuitry **1409** and allocate/rename **1407** circuitry and a memory unit perform a register read/memory read stage; the execution circuitry **1411** performs an execute stage; 6) a memory unit and the physical register file(s) unit(s) perform a write back/memory write stage; 7) various units may be involved in the exception

16

handling stage; and 8) a retirement unit and the physical register file(s) unit(s) perform a commit stage.

The core may support one or more instructions sets (e.g., the x86 instruction set (with some extensions that have been added with newer versions); the MIPS instruction set of MIPS Technologies of Sunnyvale, CA; the ARM instruction set (with optional additional extensions such as NEON) of ARM Holdings of Sunnyvale, CA), including the instruction(s) described herein. In one embodiment, the core **1490** includes logic to support a packed data instruction set extension (e.g., AVX1, AVX2), thereby allowing the operations used by many multimedia applications to be performed using packed data.

It should be understood that the core may support multithreading (executing two or more parallel sets of operations or threads), and may do so in a variety of ways including time sliced multithreading, simultaneous multithreading (where a single physical core provides a logical core for each of the threads that physical core is simultaneously multithreading), or a combination thereof (e.g., time sliced fetching and decoding and simultaneous multithreading thereafter such as in the Intel® Hyperthreading technology).

Layout

Throughout this description, data is expressed using row major data layout. Column major users should translate the terms according to their orientation. FIG. **15** illustrates an example of a matrix expressed in row major format and column major format. As shown, matrix A is a 2x3 matrix. When this matrix is stored in row major format, the data elements of a row are consecutive. When this matrix is stored in column major format, the data elements of a column are consecutive. It is a well-known property of matrices that $A^T * B^T = (BA)^T$, where superscript T means transform. Reading column major data as row major data results in the matrix looking like the transform matrix.

In some embodiments, row-major semantics are utilized in hardware, and column major data is to swap the operand order with the result being transforms of matrix, but for subsequent column-major reads from memory it is the correct, non-transformed matrix.

For example, if there are two column-major matrices to multiply:

$$\begin{array}{cc} \begin{array}{cc} a & b \\ c & d \\ e & f \end{array} & * & \begin{array}{cc} g & i & k \\ h & j & l \end{array} \\ (3 \times 2) & & (2 \times 3) \end{array} = \begin{array}{cc} ag + bh & ai + bj \\ cg + dh & ci + dj \\ eg + fh & ei + fj \end{array} \begin{array}{cc} ak + bl \\ ck + dl \\ ek + fl \end{array} \\ & & (3 \times 3)$$

The input matrices would be stored in linear memory (column-major) as:

a c e b d f
and
g h i j k l.

Reading those matrices as row-major with dimensions 2x3 and 3x2, they would appear as:

$$\begin{array}{cc} \begin{array}{cc} a & c & e \\ b & d & f \\ k & l \end{array} & \text{and} & \begin{array}{cc} g & h \\ i & j \end{array} \end{array}$$

Swapping the order and matrix multiplying:

$$\begin{array}{cc} \begin{array}{cc} g & h \\ i & j \\ k & l \end{array} & * & \begin{array}{cc} a & c & e \\ b & d & f \end{array} \\ & & (3 \times 2) \end{array} = \begin{array}{cc} ag + bh & cg + dh \\ ai + bj & ci + dj \\ ak + bl & ck + dl \end{array} \begin{array}{cc} eg + fh \\ ei + fj \\ ek + fl \end{array} \\ & & (2 \times 3)$$

17

The transform matrix is out and may then be stored in in row-major order:

ag+bh cg+dh eg+fh ai+bj ci+dj ei+fj ak+bl ck+dl ek+fl
and used in subsequent column major computations, it is the correct un-transformed matrix:

ag + bh	ai + bj	ak + bl
cg + dh	ci + dj	ck + dl
eg + fh	ei + fj	ek + fl

Exemplary Usage

FIG. 16 illustrates an example of usage of matrices (tiles). In this example, matrix C **1601** includes two tiles, matrix A **1603** includes one tile, and matrix B **1605** includes two tiles. This figure shows an example of the inner loop of an algorithm to compute a matrix multiplication. In this example, two result tiles, tmm0 and tmm1, from matrix C **1601** are used to accumulate the intermediate results. One tile from the matrix A **1603** (tmm2) is re-used twice as it multiplied by two tiles from matrix B **1605**. Pointers to load a new A matrix (tile) and two new B matrices (tiles) from the directions indicated by the arrows. An outer loop, not shown, adjusts the pointers for the C tiles.

The exemplary code as shown includes the usage of a tile configuration instruction and is executed to configure tile usage, load tiles, a loop to process the tiles, store tiles to memory, and release tile usage.

FIG. 17 illustrates an embodiment of usage of matrices (tiles). At **1701**, tile usage is configured. For example, a TILECONFIG instruction is executed to configure tile usage including setting a number of rows and columns per tile. Typically, at least one matrix (tile) is loaded from memory at **1703**. At least one matrix (tile) operation is performed at **1705** using the matrices (tiles). At **1707**, at least one matrix (tile) is stored out to memory and a context switch may occur at **1709**.

Exemplary Configuration

Tile Configuration Hardware Support

As discussed above, tile usage typically needs to be configured prior to use. For example, full usage of all rows and columns may not be needed. Not only does not configuring these rows and columns save power in some embodiments, but the configuration may be used to determine if an operation will generate an error. For example, a matrix multiplication of the form $(N \times M) \times (L \times N)$ typically not work if M and L are not the same.

Prior to using matrices using tiles, in some embodiments, tile support is to be configured. For example, how many rows and columns per tile, tiles that are to be used, etc. are configured. A TILECONFIG instruction is an improvement to a computer itself as it provides for support to configure the computer to use a matrix accelerator (either as a part of a processor core, or as an external device). In particular, an execution of the TILECONFIG instruction causes a configuration to be retrieved from memory and applied to matrix (tile) settings within a matrix accelerator.

Tile Usage Configuration

FIG. 18 illustrates support for configuration of the usage of tiles according to an embodiment. A memory **1801** contains the tile description **1803** of the matrices (tiles) to be supported.

Instruction execution resources **1811** of a processor/core **1805** stores aspects of a tile description **1803** into tile configurations **1817**. The tile configurations **1817** include palette table **1813** to detail what tiles for a palette are configured (the number of rows and columns in each tile)

18

and a marking that matrix support is in use. In particular, instruction execution resources **1811** are configured to use tiles as specified by the tile configurations **1817**. The instruction execution resources **1811** may also include a machine specific register or configuration register to indicate tile usage. Additional values such as in-use and start values are also set. The tile configurations **1817** utilize register(s) **1819** to store tile usage and configuration information.

FIG. 19 illustrates an embodiment of a description of the matrices (tiles) to be supported. This is the description that is to be stored upon an execution of a STTILECFG instruction. In this example, each field is a byte. In byte [0], a palette ID **1901** is stored. The palette ID is used to index a palette table **1813** which stores, per palette ID, a number of bytes in a tile, and bytes per row of the tiles that are associated with this ID as defined by the configuration.

Byte 1 stores a value to be stored in a "startRow" register **1903** and byte 2 stores a value to be stored in a register, startP **1905**. To support restarting instructions after these events, the instructions store information these registers. To support restarting instructions after break events such as those detailed above, the instructions store information in these registers. The startRow value indicates the row that should be used for restart. The startP value indicates the position within the row for store operations when pairs are used and, in some embodiments, indicates the lower half of the row (in the lower tile of a pair) or higher half of the row (in the higher tile of a pair). Generally, the position in the row (the column) is not needed.

With the exception of TILECONFIG and STTILECFG, successfully executing matrix (tile) instructions may set both startRow and startP to zero.

Any time an interrupted matrix (tile) instruction is not restarted, it is the responsibility of software to zero the startRow and startP values. For example, unmasked floating point exception handlers might decide to finish the operation in software and change the program counter value to another instruction, usually the next instruction. In this case the software exception handler must zero the startRow and startP values in the exception presented to it by the operating system before resuming the program. The operating system may subsequently reload those values using a restore instruction.

Byte 3 stores an indication of pairs (1 b per tile) of tiles **1907**.

Bytes 16-17 store the number of rows **1913** and columns **1915** for tile 0, bytes 18-19 store the number of rows and columns for tile 1, etc. In other words, each 2-byte group specifies a number of rows and columns for a tile. If a group of 2 bytes is not used to specify tile parameters, they should have the value zero. Specifying tile parameters for more tiles than the implementation limit or the palette limit results in a fault. Unconfigured tiles are set to an initial state with 0 rows, 0 columns.

Finally, the configuration in memory typically ends with an ending delineation such as all zeros for several consecutive bytes.

Exemplary Tile and Tile Configuration Storage

FIGS. 20(A)-(D) illustrate examples of register(s) **1819**. FIG. 20(A) illustrates a plurality of registers **1819**. As shown each tile (TMM0 **2001** . . . TMMN **2003**) has a separate register with each register storing a row and column size for that particular tile. StartP **2011** and StartRow **2013** are stored in separate registers. One or more status registers **2015** are set (e.g., TILES_CONFIGURED=1) to indicate tiles are configured for use.

FIG. 20(B) illustrates a plurality of registers **1819**. As shown each tile has separate registers for its rows and columns. For example, TMM0 rows configuration **2021**, TMM0 columns configuration **2023**, StartP **2011** and StartRow **2013** are stored in separate registers. One or more status registers **2015** are set (e.g., TILES_CONFIGURED=1) to indicate tiles are configured for use.

FIG. 20(C) illustrates a single register **1819**. As shown, this register stores tile configurations (rows and columns per tile) **2031**, StartP **2011**, and StartRow **2013** are stored in single register as packed data registers. One or more status registers **2015** are set (e.g., TILES_CONFIGURED=1) to indicate tiles are configured for use.

FIG. 20(D) illustrates a plurality of registers **1819**. As shown, a single register stores tile configuration (rows and columns per tile) **2031**. StartP and StartRow are stored in separate registers **2011** and **2013**. One or more status registers **2015** are set (e.g., TILES_CONFIGURED=1) to indicate tiles are configured for use.

Other combinations are contemplated such as combining the start registers into a single register where they are shown separately, etc.

TILETFM2RI

As mentioned above, special hardware for General Matrix Multiplication (a.k.a., GEMM), is a good option for improving the peak compute (and energy efficiency) of certain applications, such as deep learning. Some of these applications, including deep learning, can operate on input data elements with relatively few bits without losing accuracy, as long as the output elements have enough bits (i.e., more than the inputs).

Accordingly, disclosed methods and systems perform a transform instruction, TILETFM2RI, that transforms a matrix (tile)—i.e., a two-dimensional (2D) block of data—and writes its results into a row-interleaved (RowInt) formatted matrix/tile/2D register. In particular, a RowInt-formatted matrix can be used when performing fused multiply-add (FMA) operations (or other arithmetic) to perform the FMA and then combine (add together) the results of adjacent operations. For example, all of the element-wise multiplies specified by the vector or tile FMA can be performed, and then added together pairs of adjacent elements into accumulators being twice as large as the input values, which yields the same total number of bits in the output as in the input. TILETFM2RI instructions enable such an optimization in a GEMM context.

Deep learning training (and possibly other applications) uses some tiles as both RowInt-formatted and regular-formatted. Thus, the application will be sped up if a tile is transformed from a conventional 2D block format (i.e., with normal rows and columns) into RowInt format. The RowInt format takes a group of consecutive rows of a matrix in “normal” format, and merges/interleaves those rows. In the context of GEMM-matrix multiplication operations, what is ultimately needed is to multiply each row of matrix A with each column of matrix B. If B is in this RowInt format, then in effect each column of matrix B is “shorter.”

FIG. 21A is a block diagram illustrating use of a TILETFM2RI instruction to accelerate matrix multiplication, according to some embodiments. As shown, instruction **2101** includes an opcode **2102** (e.g. TILETFM2RI), which indicates that the processor is to transform the specified source matrix into the specified destination matrix having a row-interleaved (RowInt) format. In particular, in response to the opcode, the processor is to interleave J elements of each J-element sub-column of the specified source matrix in row-major order into a K-wide submatrix of the specified

destination matrix, the K-wide submatrix having K columns and enough rows to hold the J elements.

J **2108** and K **2110**, which here equal four and two, are to be specified in one or more of several ways: as operands to the TILETFM2RI instruction (as here), as suffixes or prefixes to the specified opcode, as part of an immediate provided with the instruction (e.g., J to be specified by the lower 8 bits, and K to be specified by the upper 8 bits of a 16-bit immediate), as part of control registers programmed by software before issuing the instruction (e.g., XTILE-CONFIG), or even as architectural default values. J and K may each choose from an unlimited range of integer values.

Instruction **2101** further specifies destination matrix (tile) location **2104** and source matrix (tile) location **2106**. Each specified matrix locations can be in any of a memory location, a collection of vector registers, and a collection of tile registers. Here, specified source matrix **2112** and destination matrix **2116** each includes thirty-two (32) word-sized elements. The specified source matrix **2112** includes four rows and eight columns, while the specified destination matrix **2116** includes 2 rows and 16 columns. As shown, specified destination matrix **2116** is a row-interleaved (Row-Int) format transformation of the specified source matrix **2112**.

Also shown is system **2100** for executing the TILETFM2RI instruction. The system includes specified source matrix (tile) **2112**, execution circuitry **2114**, and specified destination matrix (tile) **2116**. Both the specified destination matrix (tile) **2116** and the specified source matrix in “normal” form **2112** are routed to matrix multiplication circuitry (GEMM) **2118**, improving GEMM performance and efficiency.

Alternate, inferior approaches to transforming matrix data may exist, but do not achieve the power and performance gains of the disclosed embodiments performing the TILETFM2RI instruction. In some other approaches, software can load data into vector/SIMD registers, perform the transform using vector instructions, write the reformatted data to memory, and then load the reformatted data into a 2D/vector/tile register. But doing the format transitions in vector instructions is slow, requires complex software tuning, and may require more space in the cache.

Disclosed embodiments improve upon alternative approaches by allowing software to perform the TILETFM2RI instruction to transform two-dimensional (2D) matrices (tiles) of data and place them into RowInt-formatted 2D matrix (tile) registers. Advantageously, it is expected to be simple for software to use this instruction, compared to approaches limited to using vector registers.

In one embodiment, as further illustrated and described below with respect to FIG. 21C, a processor is to perform a TILETFM2RI instruction to transform words from a pair of source matrices (tiles) and store the result into a pair of destination matrices (tiles). TILETFM2RI transforms 2D tiles of data into so-called RowInt format and places them into one or more 2D (i.e., tile) registers. In one embodiment, the lower numbered tile of the source register pair represents the first 16 rows of the tile pair, while the upper numbered tile of the source register pair represents the next 16 rows of the tile pair. The first destination tile will contain data from the first 16 columns of the source register pair, while the second destination tile will contain data from the second 16 columns of the source register pair. In this embodiment, the source tile (register-block) pair appears as upper/lower (which means the first tile holds rows 1-16 and the second tile holds rest of the rows 17-32). The destination tile

21

(register block) pair appears as left/right (which means the first tile holds columns 1-16 and the second tile holds rest of the columns 17-32).

Advantageously, disclosed embodiments can handle data elements of any size (e.g., 1 byte, 2 bytes, 4 bytes). This includes sub-byte-sized elements, such as 2-bit or 4-bit data elements.

As a further advantage, disclosed embodiments can also handle different numbers of input and output tiles, including a single tile, a pair, or more than a pair.

The input data for this instruction may come from one or more tile registers, from a set of vector registers, or from memory. Instructions may specify multiple tiles or vector registers as sources and destinations and may explicitly encode each one. In some embodiments, on the other hand, the instruction specifies the first register (for instance, X), and the remaining registers are a fixed function of that one (e.g., X+1, X+2, etc., or X+2, X+4, etc.). In some embodiments, a TILETFM2RI instruction specifies memory locations as locations of one or more source or destination matrices (tiles).

When transforming rectangular tiles (with more rows than columns, or vice versa), a tile architecture may have an asymmetric limit on the number of rows and columns. Thus, some embodiments require more or fewer registers to hold the input than the output.

In some embodiments, software is to add padding to an output tile (e.g., to make the tile square), or remove it from an input tile. Disclosed embodiments allow for this. In some embodiments, the TILETFM2RI instruction uses a parameter (e.g., in a register, as an immediate, or as part of the opcode) that indicates the number of elements to add to each output row (or column) as padding. The value of a padded element may be zero, or some other pre-selected value, or this may be an input to the instruction (as a register, an immediate, or part of the opcode). Alternatively, or additionally, the instruction may take a parameter indicating elements to remove from each input row or column and whether these are at the beginning, end, or some of each, of each row/column.

FIG. 21B illustrates an exemplary execution of a TILETFM2RI instruction according to some embodiments. As shown, instruction 2121 includes an opcode 2122 (e.g. TILETFM2RI), which indicates that the processor is to transform the specified source matrix into the specified destination matrix having a row-interleaved (RowInt) format. In particular, in response to the opcode, the processor is to interleave J elements of each J-element sub-column of the specified source matrix in row-major order into a K-wide submatrix of the specified destination matrix, the K-wide submatrix having K columns and enough rows to hold the J elements.

J 2128 and K 2130, which here equal four and two, are to be specified in one or more of several ways: as operands to the TILETFM2RI instruction (as here), as suffixes or prefixes to the specified opcode, as part of an immediate provided with the instruction (e.g., J to be specified by the lower 8 bits, and K to be specified by the upper 8 bits of a 16-bit immediate), as part of control registers programmed by software before issuing the instruction (e.g., XTILECONFIG), or even as architectural default values. J and K may each choose from an unlimited range of integer values.

Instruction 2121 further specifies destination matrix (tile) location 2124 and source matrix (tile) location 2126. Each specified matrix locations can be in any of a memory location, a collection of vector registers, and a collection of tile registers. Here, specified source matrix 2132 and desti-

22

nation matrix 2136 each includes thirty-two (32) word-sized elements. The specified source matrix 2132 includes four rows and eight columns, while the specified destination matrix 2136 includes 2 rows and 16 columns. As shown, specified destination matrix 2136 is a row-interleaved (RowInt) format transformation of the specified source matrix 2132.

Also shown is system 2120 for executing the TILETFM2RI instruction. The system includes specified source matrix (tile) 2132, execution circuitry 2134, and specified destination matrix (tile) 2136.

In operation, fetch and decode circuitry (not shown) are to fetch and decode the TILETFM2RI instruction 2121. Execution circuitry 2136 is to respond to the decoded transform instruction by transforming the specified source matrix into the specified RowInt-formatted destination matrix by interleaving J elements of each J-element sub-column of the specified source matrix in row-major order into a K-wide submatrix of the specified destination matrix, the K-wide submatrix having K columns and enough rows to hold the J elements.

FIG. 21C illustrates an exemplary execution of a TILETFM2RI instruction according to some embodiments. As shown, the format of instruction 2172 includes fields to specify an opcode 2174 (e.g. TILETFM2RI) and locations of destination matrix (tile) 2176, second destination matrix (tile) 2178, source matrix (tile) 2180, and second source matrix (tile) 2182.

Instruction 2172 further specifies J 2184 and K 2186, which here are equal to four and two, respectively. It should be noted, however, that J and K may be specified in one or more of several other ways: as operands to the TILETFM2RI instruction, as suffixes or prefixes to the specified opcode, as part of an immediate provided with the instruction (e.g., J to be specified by the lower 8 bits, and K to be specified by the upper 8 bits of a 16-bit immediate), as part of control registers programmed by software before issuing the instruction (e.g., XTILECONFIG), or even as architectural default values. J and K may each be set to an unlimited range of integer values.

Also shown is system 2170 for executing the TILETFM2RI instruction 2172. The system includes first specified source matrix (tile) 2188 and second specified source matrix (tile) 2195. System 2170 further includes execution circuitry 2192 and specified first and second destination matrices (tiles) 2194 and 2196.

In operation, a processor is to perform a TILETFM2RI instruction 2172 to transform words from the pair of source matrices (tiles) and store the result into the pair of destination matrices (tiles). TILETFM2RI transforms specified source 1 2184 and specified source 2 2186 matrices (tiles) into row-interleaved (RowInt) format and places them into specified destination 1 and destination 2 2195 and 2192. In one embodiment, as shown, the lower numbered tile of the source tile pair represents the first 16 rows of the tile pair, while the upper numbered tile of the source tile pair represents the next 16 rows of the tile pair. The first destination tile is to contain data from the first 16 columns of the source tile pair, while the second destination tile is to contain data from the second 16 columns of the source tile pair. In this embodiment, the source tile (register-block) pair appears as upper/lower (which means the first tile holds rows 1-16 and the second tile holds rest of the rows 17-32). The destination tile (register block) pair appears as left/right (which means the first tile holds columns 1-16 and the second tile holds rest of the columns 17-32). In other embodiments, the destination tile (register block) pair appears as upper/lower.

In operation, fetch and decode circuitry, not shown, are to fetch and decode the TILETFM2RI instruction 2172. Execution circuitry 2192 is to respond to the decoded matrix transform instruction by transforming the specified source matrix into the specified RowInt-formatted destination matrix by interleaving J elements of each J-element sub-column of the specified source matrix in row-major order into a K-wide submatrix of the specified destination matrix, the K-wide submatrix having K columns and enough rows to hold the J elements. In other embodiments, the J elements are to be interleaved into the K-wide submatrix in column-major order.

Matrix Transpose and Multiply

As described above, two matrices (e.g., tile A and tile B) may be used as inputs to matrix multiplication operations and/or hardware. Also as described above, it may be possible to improve performance by transforming one of these two matrices (e.g., tile B) to a row-interleaved (RowInt) format (which may also be called VNNI format). For deep learning training (and possibly other applications), a particular tile may sometimes be used as a tile A and sometimes as a tile B. Additionally, for deep learning (and possibly other applications), it may be desired to transpose a tile, which may be in RowInt format, for use as an input (e.g., tile A) to a matrix multiplication. Therefore, embodiments may include hardware, within a matrix multiplication unit, that transposes an input matrix, to support a single instruction that transposes an input matrix and performs a matrix multiplication with performance similar to using a regular matrix multiply instruction without the additional overhead of using vector and/or memory instructions (or another approach) to transpose an input matrix. Furthermore, since embodiments transpose a matrix “on the fly” to use as an input (e.g., tile A) to a particular matrix multiplication and leave the matrix stored as is (i.e., not transposed), that matrix remains available as is to use as an input (e.g., as tile B) to a different matrix multiply operation.

For example, a single instruction, which may be referred to as a matrix transpose+multiply instruction, may be used according to embodiments to transpose a first source matrix and perform a matrix multiplication using the transposed first source matrix and a second source matrix. In embodiments, the first source matrix may be in a RowInt format. Embodiments may include any one or more of a variety of types of matrix transpose+multiply instructions, for example, a first instruction to operate on matrices storing a first datatype, a second instruction to operate on matrices storing a second datatype, etc., as described by way of example embodiments below.

In addition or instead of one or more regular and/or known matrix multiply instructions, any one, set, or subset of type(s) of matrix transpose+multiply instructions may be implemented in various embodiments as entirely separate instructions (e.g., entirely separate opcodes to distinguish the operation, datatype of the source and/or destination matrices, and or width/size of the source and/or destination values and/or elements), related instructions (e.g., sharing one or more bits of an opcode with one or more prefix, suffix, immediate, and/or other bit(s) to distinguish the operation, datatype(s), and/or width/size(s)), and/or leaves in a tree or hierarchy of instructions (e.g., the operation, datatype(s), and/or width/size(s) may be distinguished by content of one or more registers and/or other storage when the instruction used), each as may be further described below. For example, a first datatype of the source matrices and a second datatype of the destination matrix for a matrix transpose+multiply instruction may be specified by the opcode of the instruc-

tion; explicitly in a one or more prefixes or suffixes to the opcode; explicitly in one or more operands or immediates of the instruction; explicitly in a one or more configuration or control registers (e.g., programmed by software prior to execution of the instruction); implicitly by one or more datatypes and/or sizes associated with one or more registers, memory ranges, or other storage; implicitly as one or more architectural default values; etc. Regardless of the implementation of such sets or subsets, embodiments provide for an instruction set architecture (ISA) of an information processing system that allows a programmer or software developer to use a single instruction, rather than multiple instructions, to transpose an input matrix and perform a matrix multiplication.

For example, in embodiments, a processor or processor core may include an instruction unit and an execution unit, as shown in and described in connection with the figures depicting processors and processor cores according to embodiments of the invention. The instruction unit may include a decoder or other instruction hardware to decode or otherwise receive instructions of an instruction set architecture that includes a one or more types of matrix transpose+multiply instructions, and in response to the decoder decoding and/or otherwise receiving an instance of a matrix transpose+multiply instruction, hardware in and/or associated with the execution unit may transpose a first source matrix (tile) and perform a matrix multiplication using the transposed first source matrix (tile) and a second source matrix (tile).

In embodiments, the transposed first source matrix (tile) may be used as a tile A input to the matrix multiplication and the second source matrix (tile) may be used as a tile B input to the matrix multiplication (each of the tile A input, tile B input, and matrix multiplication as described above or as known). In embodiments, the result of the matrix multiplication may be accumulated or otherwise stored in a destination matrix (tile). In embodiments, the transposition of the first source matrix (tile) for use as an input to the matrix multiplication does not modify (i.e., leaves as is) the data in the first source matrix (tile) for use in other operations. In embodiments, the execution unit may include matrix transposition hardware and matrix multiply hardware, such that both the matrix transposition hardware and the matrix multiply hardware may be used to execute a matrix transpose+multiply instruction, but the matrix transposition hardware may be bypassed and the matrix multiply hardware used to execute a regular (i.e., without source matrix transposition) matrix multiply instruction. Similarly, the same matrix transposition hardware, the same execution unit (e.g., bypassing the matrix multiplication hardware), different matrix transposition hardware, and/or a different execution unit may be used in embodiments to execute a matrix transposition instruction (e.g., according to embodiments of matrix transposition described in this specification) that does not include a matrix multiplication.

In embodiments, a decoder and execution hardware of a processor or processor core may be implemented to decode and execute one or more types of matrix transpose+multiply instructions; for example, a first instruction to use for source matrices in which the elements are pairs of a first source datatype (e.g., 16-bit brain floating-point, bfloat16, or BF16) to be accumulated to results of a first destination datatype (e.g., 32-bit floating-point or FP32), a second instruction to use for source matrices in which the elements are pairs of a second source datatype (e.g., 16-bit floating-point or FP16) to be accumulated to results of the first destination datatype (e.g., 32-bit floating-point or FP32), a third instruction to use

25

for source matrices in which the elements are quadruplets (quads) of a third source datatype (e.g., 8-bit integer or INT8) to be accumulated to results of a first destination datatype (e.g., 32-bit integer or INT32), etc.

Embodiments may provide for transpositions of various granularities. For example, a matrix having a row width of 64 bytes may be transposed such that each row becomes a column having a width of one byte, two bytes, four bytes, etc. In embodiments, the granularity of the transposition may be determined according to the width (and/or datatype of and number of values per element) of the matrix (tile) elements (as may be indicated by the instruction, a parameter of the instruction, etc., as described above). For example, if the matrix multiplication is composed of dot-product operations between/among 32-bit-wide tile elements, where each element consists of a 32-bit-wide group of values (e.g., two FP16 or four INT8 values), the transposition will be performed on 32-bit tile elements. In other words, each column of FP16 pairs or INT8 quads will be transposed into a row (i.e., each row will be transposed into a 32-bit wide column).

26

including an even column and an odd column of BF16 values) in tile A represents a column in the 8x4 BF16 matrix. On the right side, Tile A^T 2220 shows the tile contents after transposition at tile-element granularity (here, 32 bits for two BF16 values). As shown, the tile-element wide columns become rows and the transposed tile holds the 4x8 transposed BF16 matrix, which may then be used as an input to matrix multiplication.

In an embodiment, a matrix transpose+multiply instruction may be executed to transpose a first source matrix (tile) storing bfloat16 pairs as elements and perform a matrix multiplication using the transposed first source matrix (tile) and a second source matrix (tile), also storing bfloat16 pairs as elements, accumulating the results as FP32 elements. An embodiment may include hardware to perform the matrix transposition and the matrix multiply as set forth in the following pseudocode and as described below:

```
// C = M x N (tsrdest), A = K x M (tsrc1), B = K x N (tsrc2)
// src1 and src2 elements are pairs of bfloat16
elements_dest:= tsrdest.colsb/4
elements_temp:= tsrdest.colsb/2 //count is in bfloat16 prior to horizontal
for m in 0 ... tsrdest.rows-1
    temp1[0 ... elements_temp-1] := 0
    for k in 0 ... tsrc1.rows-1:
        for n in 0 ... elements_dest-1:
            // FP32 FMA with DAZ=FTZ=1, RNE rounding. MXCSR is neither
            // consulted nor updated. No exceptions raised or denoted.
            temp1.fp32[2*n+0] += make_fp32(tsrc1.row[k].bfloat16[2*m+0]) *
                                make_fp32(tsrc2.row[k].bfloat16[2*n+0])
            temp1.fp32[2*n+1] += make_fp32(tsrc1.row[k].bfloat16[2*m+1]) *
                                make_fp32(tsrc2.row[k].bfloat16[2*n+1])
        for n in 0 ... Elements_dest-1:
            // FP32 FMA with DAZ=FTZ=1, RNE rounding. MXCSR is neither
            // consulted nor updated. No exceptions raised or denoted.
            tmpf32 := temp1.fp32[2*n+0] + temp1.fp32[2*n+1]
            tsrdest.row[m].fp32[n] := srcdest.row[m].fp32[n] + tmpf32
        write_row_and_zero(tsrdest, m, tmp, tsrdest.colsb)
    zero_upper_rows(tsrdest, tsrdest.rows)
    zero_tileconfig_start( )
```

FIG. 22A illustrates the relationship between granularity and datatype in a source matrix transposition according to an embodiment. In FIG. 22A, each square represents a value having a 16-bit datatype (e.g., BF16) and tile elements are pairs of such values. On the left side, tile A 2210 holds thirty-two BF16 values in RowInt format (e.g., tile A may have been transformed, as described above with reference to FIG. 21B, from an 8 row, 4 column (8x4) BF16, with J=4 and K=2. In other words, each column of BF16 pairs (i.e.,

In an embodiment, a matrix transpose+multiply instruction may be executed to transpose a first source matrix (tile) storing FP16 pairs as elements and perform a matrix multiplication using the transposed first source matrix (tile) and a second source matrix (tile), also storing FP16 pairs as elements, accumulating the results as FP32 elements. An embodiment may include hardware to perform the matrix transposition and the matrix multiply as set forth in the following pseudocode and as described below:

```
// C = M x N (tsrdest), A = K x M (tsrc1), B = K x N (tsrc2)
# src1 and src2 elements are pairs of fp16
elements_dest:= tsrdest.colsb/4
elements_temp:= tsrdest.colsb/2 //count is in fp16 prior to horizontal
for m in 0 ... tsrdest.rows-1:
    temp1[0 ... elements_temp-1] := 0
    for k in 0 ... tsrc1.rows-1:
        for n in 0 ... elements_dest-1:
            // FP32 FMA with DAZ=FTZ=1, RNE rounding. MXCSR is neither
            // consulted nor updated. No exceptions raised or denoted.
            temp1.fp32[2*n+0] += cvt_fp16_to_fp32(tsrc1.row[k].fp16[2*m+0]) *
                                cvt_fp16_to_fp32(tsrc2.row[k].fp16[2*n+0])
            temp1.fp32[2*n+1] += cvt_fp16_to_fp32(tsrc1.row[k].fp16[2*m+1]) *
                                cvt_fp16_to_fp32(tsrc2.row[k].fp16[2*n+1])
        for n in 0 ... Elements_dest-1:
            // FP32 FMA with DAZ=FTZ=1, RNE rounding. MXCSR is neither
            // consulted nor updated. No exceptions raised or denoted.
```

```

tmpf32 := tmp1.fp32[2*n+0] + tmp1.fp32[2*n+1]
tsrctest.row[m].fp32[n] := srcdest.row[m].fp32[n] + tmpf32
write_row_and_zero(tsrctest, m, tmp, tsrctest.colsb)
zero_upper_rows(tsrctest, tsrctest.rows)
zero_tileconfig_start( )

```

In an embodiment, a matrix transpose+multiply instruction may be executed to transpose a first source matrix (tile) storing INT8 quads as elements and perform a matrix multiplication using the transposed first source matrix (tile) and a second source matrix (tile), also storing INT8 quads as elements, accumulating the results as INT32 elements. An embodiment may include hardware to perform the matrix transposition and the matrix multiply as set forth in the following pseudocode and as described below:

```

define DPBD(c,x,y): // arguments are dwords
  if *x operand is signed*:
    extend_src1 := SIGN_EXTEND
  else:
    extend_src1 := ZERO_EXTEND
  if *y operand is signed*:
    extend_src2 := SIGN_EXTEND
  else:
    extend_src2 := ZERO_EXTEND
  p0dword := extend_src1(x.byte[0]) * extend_src2(y.byte[0])
  p1dword := extend_src1(x.byte[1]) * extend_src2(y.byte[1])
  p2dword := extend_src1(x.byte[2]) * extend_src2(y.byte[2])
  p3dword := extend_src1(x.byte[3]) * extend_src2(y.byte[3])
  c := c + p0dword + p1dword + p2dword + p3dword

```

```

// C = M x N (tsrctest), A = K x M (tsrc1), B = K x N (tsrc2)
elements_dest:= tsrctest.colsb/4
for m in 0 ... tsrctest.rows-1
  tmp:= tsrctest.row[m]
  for n in 0 ... tsrc1.rows-1:
    for k in 0 ... elements_dest-1:
      DPBD(tmp.dword[n],
            tsrc1.row[k].dword[m],
            tsrc2.row[k].dword[n])
    write_row_and_zero(tsrctest, m, tmp, tsrctest.colsb)
zero_upper_rows(tsrctest, tsrctest.rows)
zero_tileconfig_start( )

```

As may be seen from the pseudocode, a matrix transpose-multiply instruction may have a first operand to specify a destination location (e.g., tsrctest, which may represent an operand to specify a tile register or other location for a destination matrix having M rows and N element-wide columns), a second operand to specify a first source location (e.g., tsrc1, which may represent an operand to specify a tile register or other location for a first source matrix having K rows and M element-wide columns), and a third operand to specify a second source location (e.g., tsrc2, which may represent an operand to specify a tile register or other location for a second source matrix having K rows and N element-wide columns), and execution of the instruction may include transposing the first source matrix, multiplying the transposed first source matrix and the second source matrix, and accumulating the result in the destination matrix.

As may also be seen from the pseudocode, for each instruction, the transposition granularity is based on the element size, while the matrix multiplication includes a number of dot-product operations with each dot-product operation performed based on the value size and number of values per element. For example, the pseudocode shows that

the granularity is 32-bits (“elements_dest:=tsrctest.colsb/4” and “for n in 0 . . . elements_dest-1” are based on a granularity of 4 bytes because each bfloat16 pair, each FP16 pair, and each INT8 quad is 4 bytes) and that each dot-product operation is performed using each value in a group as inputs to the dot-product operation (e.g., a bfloat16 or FP16 pair (see bodies of “for n in 0 . . . elements_dest-1” and “for n in 0 . . . Elements_dest-1” statements) or an INT8 quad (see “DPBD” definition)).

As may also be seen from the pseudocode, values may be converted (e.g., to a different datatype for use as inputs to the dot-product operations (see the use of the “make” and “cvt” helper functions in the pseudocode for the first and second instructions, respectively, and the if-then statements in the pseudocode for the third instruction).

As may also be seen from the pseudocode, storing the result in the destination matrix location may include accumulating terms from a number of dot-product operations (see “tsrctest.row [m].fp32 [n]:=tsrctest.row [m].fp32 [n]+tmpf32” in pseudocode for first two instructions and “c:=c+p0dword+p1dword+p2dword+p3dword” in pseudocode for third instruction).

FIG. 22B is a block diagram illustrating use of a matrix transpose+multiply instruction according to some embodiments. As shown, instruction **2251** includes an opcode **2252** which indicates that the processor is to transpose a first source matrix, multiply the transposed first source matrix and a second source matrix, and accumulate the result in a destination matrix. Instruction **2251** has a first operand **2254** to specify the destination matrix (tile) location, a second operand **2256** to specify the first source matrix (tile) location, and a third operand **2258** to specify the second source matrix (tile) location.

Also shown is system **2250** for executing the matrix transpose+multiply instruction. The system includes first source tile **2266**, second source tile **2268**, execution unit **2270**, and destination tile **2264**. Here, specified first source tile **2266** has four rows and two columns, specified second source tile **2268** has four rows and two columns, and specified destination tile **2264** has two rows and two columns. Execution unit **2270** may include matrix transposition circuitry **2272** and matrix multiplication circuitry **2274**. A representation of transposed first source tile **2276**, having two rows and four columns, is shown within execution unit **2270**, but might not be stored in execution unit **2270** or elsewhere, since the transposition may occur “on the fly” or incrementally during the matrix multiply operation. Exemplary Method(s) of Execution

FIG. 23 illustrates an embodiment of a processor executing a flow **2300** to process a matrix transpose+multiply instruction. As shown, at **2301**, the processor is to fetch, using fetch circuitry, an instruction with a format having fields to specify an opcode and locations of a destination matrix, a first source matrix, and a second source matrix, wherein the opcode indicates that the processor is to transpose the first source matrix, perform a matrix multiplication using the transposed first source matrix and the second

source matrix, and accumulate or otherwise store the result in the destination matrix location.

At **2303**, the processor is to decode, using decode circuitry, the fetched instruction. For example, the fetched instruction is decoded by decode circuitry such as that detailed herein. In the context of the illustrated system, decode circuitry is similar to that illustrated and described at least with respect to FIGS. **13**, **14**, and **28A-B**.

Execution of the decoded instruction is scheduled (as needed) at **2305**, which is optional (as indicated by its dashed border) insofar as it may occur at a different time, or not at all. At **2307**, the processor is to respond, using execution circuitry, to the decoded instruction by transposing the first source matrix, performing a matrix multiplication using the transposed first source matrix and the second source matrix, and accumulating or otherwise storing the result in the destination matrix location. Note that although **2307** is described as three actions/operations, these three actions/operations need not occur as three separate or sequential actions/operations, since embodiments may include hardware to perform the transposition “on the fly” or incrementally during the matrix multiply operation, and/or to perform the accumulation/storage incrementally during the matrix multiply operation.

Execution circuitry is further illustrated and described with respect to FIGS. **3-14**. In some embodiments, execution circuitry includes a matrix operations accelerator, such as that illustrated and described as accelerator **307** (FIG. **3**). In some embodiments, execution circuitry includes a matrix operations circuit, such as matrix operations circuitry **405** (FIG. **4**), **505** (FIG. **5**), or **1213** (FIG. **12**), and **1327** (FIG. **13**).

Exemplary Instruction Format(s)

FIG. **24** is a block diagram illustrating a format of a matrix transpose+multiply instruction, according to some embodiments. As shown, matrix transpose+multiply instruction **2400** includes fields for specifying an opcode **2402**, which indicates that the processor is to transpose a first source matrix, perform a matrix multiplication using the transposed first source matrix and a second source matrix, and accumulate or otherwise store the result in a destination matrix location.

Instruction **2400** further includes a destination operand **2404** to specify a location of the destination matrix, a first source operand **2406** to specify a location of the first source matrix, and a second source operand **2408** to specify a location of the second source operand.

In some embodiments, one or more of optional instructions modifiers **2420** (e.g., any parameters discussed above) are encoded in another operand and/or immediate field optionally included with the instruction **2400**. In some embodiments, one or more of optional instructions modifiers may be specified via a configuration/status register (e.g., XTILECONFIG). In other words, when any one or more of optional modifiers **2420** are not specified by the instruction, they sometimes use implicit parameters that are inherited from other parts of the tile architecture.

Detailed Exemplary Systems, Processors, and Emulation

Detailed herein are examples of hardware, software, etc. to execute the above described instructions. For example, what is described below details aspects of instruction execution including various pipeline stages such as fetch, decode, schedule, execute, retire, etc.

Instruction Sets

An instruction set may include one or more instruction formats. A given instruction format may define various fields (e.g., number of bits, location of bits) to specify, among

other things, the operation to be performed (e.g., opcode) and the operand(s) on which that operation is to be performed and/or other data field(s) (e.g., mask). Some instruction formats are further broken down through the definition of instruction templates (or subformats). For example, the instruction templates of a given instruction format may be defined to have different subsets of the instruction format's fields (the included fields are typically in the same order, but at least some have different bit positions because there are less fields included) and/or defined to have a given field interpreted differently. Thus, each instruction of an ISA is expressed using a given instruction format (and, if defined, in a given one of the instruction templates of that instruction format) and includes fields for specifying the operation and the operands. For example, an exemplary ADD instruction has a specific opcode and an instruction format that includes an opcode field to specify that opcode and operand fields to select operands (source1/destination and source2); and an occurrence of this ADD instruction in an instruction stream may have specific contents in the operand fields that select specific operands. A set of SIMD extensions referred to as the Advanced Vector Extensions (AVX) (AVX1 and AVX2) and using the Vector Extensions (VEX) coding scheme has been released and/or published (e.g., see Intel® 64 and IA-32 Architectures Software Developer's Manual, September 2014; and see Intel® Advanced Vector Extensions Programming Reference, October 2014).

Exemplary Instruction Formats

Embodiments of the instruction(s) described herein may be embodied in different formats. Additionally, exemplary systems, architectures, and pipelines are detailed below. Embodiments of the instruction(s) may be executed on such systems, architectures, and pipelines, but are not limited to those detailed.

Generic Vector Friendly Instruction Format

A vector friendly instruction format is an instruction format that is suited for vector instructions (e.g., there are certain fields specific to vector operations). While embodiments are described in which both vector and scalar operations are supported through the vector friendly instruction format, alternative embodiments use only vector operations the vector friendly instruction format.

FIGS. **25A-25B** are block diagrams illustrating a generic vector friendly instruction format and instruction templates thereof according to embodiments. FIG. **25A** is a block diagram illustrating a generic vector friendly instruction format and class A instruction templates thereof according to embodiments; while FIG. **25B** is a block diagram illustrating the generic vector friendly instruction format and class B instruction templates thereof according to embodiments. Specifically, a generic vector friendly instruction format **2500** for which are defined class A and class B instruction templates, both of which include no memory access **2505** instruction templates and memory access **2520** instruction templates. The term generic in the context of the vector friendly instruction format refers to the instruction format not being tied to any specific instruction set.

While embodiments will be described in which the vector friendly instruction format supports the following: a 64 byte vector operand length (or size) with 32 bit (4 byte) or 64 bit (8 byte) data element widths (or sizes) (and thus, a 64 byte vector consists of either 16 doubleword-size elements or alternatively, 8 quadword-size elements); a 64 byte vector operand length (or size) with 16 bit (2 byte) or 8 bit (1 byte) data element widths (or sizes); a 32 byte vector operand length (or size) with 32 bit (4 byte), 64 bit (8 byte), 16 bit (2 byte), or 8 bit (1 byte) data element widths (or sizes); and

31

a 16 byte vector operand length (or size) with 32 bit (4 byte), 64 bit (8 byte), 16 bit (2 byte), or 8 bit (1 byte) data element widths (or sizes); alternative embodiments may support more, less and/or different vector operand sizes (e.g., 256 byte vector operands) with more, less, or different data element widths (e.g., 128 bit (16 byte) data element widths).

The class A instruction templates in FIG. 25A include: 1) within the no memory access **2505** instruction templates there is shown a no memory access, full round control type operation **2510** instruction template and a no memory access, data transform type operation **2515** instruction template; and 2) within the memory access **2520** instruction templates there is shown a memory access, temporal **2525** instruction template and a memory access, non-temporal **2530** instruction template. The class B instruction templates in FIG. 25B include: 1) within the no memory access **2505** instruction templates there is shown a no memory access, write mask control, partial round control type operation **2512** instruction template and a no memory access, write mask control, vsize type operation **2517** instruction template; and 2) within the memory access **2520** instruction templates there is shown a memory access, write mask control **2527** instruction template.

The generic vector friendly instruction format **2500** includes the following fields listed below in the order illustrated in FIGS. 25A-25B.

Format field **2540**—a specific value (an instruction format identifier value) in this field uniquely identifies the vector friendly instruction format, and thus occurrences of instructions in the vector friendly instruction format in instruction streams. As such, this field is optional in the sense that it is not needed for an instruction set that has only the generic vector friendly instruction format.

Base operation field **2542**—its content distinguishes different base operations.

Register index field **2544**—its content, directly or through address generation, specifies the locations of the source and destination operands, be they in registers or in memory. These include a sufficient number of bits to select N registers from a P×Q (e.g. 32×512, 16×128, 32×1024, 64×1024) register file. While in one embodiment N may be up to three sources and one destination register, alternative embodiments may support more or less sources and destination registers (e.g., may support up to two sources where one of these sources also acts as the destination, may support up to three sources where one of these sources also acts as the destination, may support up to two sources and one destination).

Modifier field **2546**—its content distinguishes occurrences of instructions in the generic vector instruction format that specify memory access from those that do not; that is, between no memory access **2505** instruction templates and memory access **2520** instruction templates. Memory access operations read and/or write to the memory hierarchy (in some cases specifying the source and/or destination addresses using values in registers), while non-memory access operations do not (e.g., the source and destinations are registers). While in one embodiment this field also selects between three different ways to perform memory address calculations, alternative embodiments may support more, less, or different ways to perform memory address calculations.

Augmentation operation field **2550**—its content distinguishes which one of a variety of different operations to be performed in addition to the base operation. This field is context specific. In one embodiment, this field is divided into a class field **2568**, an alpha field **2552**, and a beta field

32

2554. The augmentation operation field **2550** allows common groups of operations to be performed in a single instruction rather than 2, 3, or 4 instructions.

Scale field **2560**—its content allows for the scaling of the index field's content for memory address generation (e.g., for address generation that uses $2^{\text{scale}} \times \text{index} + \text{base}$).

Displacement Field **2562A**—its content is used as part of memory address generation (e.g., for address generation that uses $2^{\text{scale}} \times \text{index} + \text{base} + \text{displacement}$).

Displacement Factor Field **2562B** (note that the juxtaposition of displacement field **2562A** directly over displacement factor field **2562B** indicates one or the other is used)—its content is used as part of address generation; it specifies a displacement factor that is to be scaled by the size of a memory access (N)—where N is the number of bytes in the memory access (e.g., for address generation that uses $2^{\text{scale}} \times \text{index} + \text{base} + \text{scaled displacement}$). Redundant low-order bits are ignored and hence, the displacement factor field's content is multiplied by the memory operands total size (N) in order to generate the final displacement to be used in calculating an effective address. The value of N is determined by the processor hardware at runtime based on the full opcode field **2574** (described later herein) and the data manipulation field **2554C**. The displacement field **2562A** and the displacement factor field **2562B** are optional in the sense that they are not used for the no memory access **2505** instruction templates and/or different embodiments may implement only one or none of the two.

Data element width field **2564**—its content distinguishes which one of a number of data element widths is to be used (in some embodiments for all instructions; in other embodiments for only some of the instructions). This field is optional in the sense that it is not needed if only one data element width is supported and/or data element widths are supported using some aspect of the opcodes.

Write mask field **2570**—its content controls, on a per data element position basis, whether that data element position in the destination vector operand reflects the result of the base operation and augmentation operation. Class A instruction templates support merging-writemasking, while class B instruction templates support both merging—and zeroing-writemasking. When merging, vector masks allow any set of elements in the destination to be protected from updates during the execution of any operation (specified by the base operation and the augmentation operation); in other one embodiment, preserving the old value of each element of the destination where the corresponding mask bit has a 0. In contrast, when zeroing vector masks allow any set of elements in the destination to be zeroed during the execution of any operation (specified by the base operation and the augmentation operation); in one embodiment, an element of the destination is set to 0 when the corresponding mask bit has a 0 value. A subset of this functionality is the ability to control the vector length of the operation being performed (that is, the span of elements being modified, from the first to the last one); however, it is not necessary that the elements that are modified be consecutive. Thus, the write mask field **2570** allows for partial vector operations, including loads, stores, arithmetic, logical, etc. While embodiments are described in which the write mask field's **2570** content selects one of a number of write mask registers that contains the write mask to be used (and thus the write mask field's **2570** content indirectly identifies that masking to be performed), alternative embodiments instead or additional allow the mask write field's **2570** content to directly specify the masking to be performed.

Immediate field **2572**—its content allows for the specification of an immediate. This field is optional in the sense that it is not present in an implementation of the generic vector friendly format that does not support immediate and it is not present in instructions that do not use an immediate.

Class field **2568**—its content distinguishes between different classes of instructions. With reference to FIGS. **25A-B**, the contents of this field select between class A and class B instructions. In FIGS. **25A-B**, rounded corner squares are used to indicate a specific value is present in a field (e.g., class A **2568A** and class B **2568B** for the class field **2568** respectively in FIGS. **25A-B**).

Instruction Templates of Class A

In the case of the non-memory access **2505** instruction templates of class A, the alpha field **2552** is interpreted as an RS field **2552A**, whose content distinguishes which one of the different augmentation operation types are to be performed (e.g., round **2552A.1** and data transform **2552A.2** are respectively specified for the no memory access, round type operation **2510** and the no memory access, data transform type operation **2515** instruction templates), while the beta field **2554** distinguishes which of the operations of the specified type is to be performed. In the no memory access **2505** instruction templates, the scale field **2560**, the displacement field **2562A**, and the displacement scale field **2562B** are not present.

No—Memory Access Instruction Templates—Full Round Control Type Operation

In the no memory access full round control type operation **2510** instruction template, the beta field **2554** is interpreted as a round control field **2554A**, whose content(s) provide static rounding. While in the described embodiments the round control field **2554A** includes a suppress all floating-point exceptions (SAE) field **2556** and a round operation control field **2558**, alternative embodiments may support may encode both these concepts into the same field or only have one or the other of these concepts/fields (e.g., may have only the round operation control field **2558**).

SAE field **2556**—its content distinguishes whether or not to disable the exception event reporting; when the SAE field's **2556** content indicates suppression is enabled, a given instruction does not report any kind of floating-point exception flag and does not raise any floating-point exception handler.

Round operation control field **2558**—its content distinguishes which one of a group of rounding operations to perform (e.g., Round-up, Round-down, Round-towards-zero and Round-to-nearest). Thus, the round operation control field **2558** allows for the changing of the rounding mode on a per instruction basis. In one embodiment where a processor includes a control register for specifying rounding modes, the round operation control field's **2550** content overrides that register value.

No Memory Access Instruction Templates—Data Transform Type Operation

In the no memory access data transform type operation **2515** instruction template, the beta field **2554** is interpreted as a data transform field **2554B**, whose content distinguishes which one of a number of data transforms is to be performed (e.g., no data transform, swizzle, broadcast).

In the case of a memory access **2520** instruction template of class A, the alpha field **2552** is interpreted as an eviction hint field **2552B**, whose content distinguishes which one of the eviction hints is to be used (in FIG. **25A**, temporal **2552B.1** and non-temporal **2552B.2** are respectively specified for the memory access, temporal **2525** instruction template and the memory access, non-temporal **2530**

instruction template), while the beta field **2554** is interpreted as a data manipulation field **2554C**, whose content distinguishes which one of a number of data manipulation operations (also known as primitives) is to be performed (e.g., no manipulation; broadcast; up conversion of a source; and down conversion of a destination). The memory access **2520** instruction templates include the scale field **2560**, and optionally the displacement field **2562A** or the displacement scale field **2562B**.

Vector memory instructions perform vector loads from and vector stores to memory, with conversion support. As with regular vector instructions, vector memory instructions transfer data from/to memory in a data element-wise fashion, with the elements that are actually transferred is dictated by the contents of the vector mask that is selected as the write mask.

Memory Access Instruction Templates—Temporal

Temporal data is data likely to be reused soon enough to benefit from caching. This is, however, a hint, and different processors may implement it in different ways, including ignoring the hint entirely.

Memory Access Instruction Templates—Non-Temporal

Non-temporal data is data unlikely to be reused soon enough to benefit from caching in the 1st-level cache and should be given priority for eviction. This is, however, a hint, and different processors may implement it in different ways, including ignoring the hint entirely.

Instruction Templates of Class B

In the case of the instruction templates of class B, the alpha field **2552** is interpreted as a write mask control (Z) field **2552C**, whose content distinguishes whether the write masking controlled by the write mask field **2570** should be a merging or a zeroing.

In the case of the non-memory access **2505** instruction templates of class B, part of the beta field **2554** is interpreted as an RL field **2557A**, whose content distinguishes which one of the different augmentation operation types are to be performed (e.g., round **2557A.1** and vector length (VSIZE) **2557A.2** are respectively specified for the no memory access, write mask control, partial round control type operation **2512** instruction template and the no memory access, write mask control, VSIZE type operation **2517** instruction template), while the rest of the beta field **2554** distinguishes which of the operations of the specified type is to be performed. In the no memory access **2505** instruction templates, the scale field **2560**, the displacement field **2562A**, and the displacement scale field **2562B** are not present.

In the no memory access, write mask control, partial round control type operation **2510** instruction template, the rest of the beta field **2554** is interpreted as a round operation field **2559A** and exception event reporting is disabled (a given instruction does not report any kind of floating-point exception flag and does not raise any floating-point exception handler).

Round operation control field **2559A**—just as round operation control field **2558**, its content distinguishes which one of a group of rounding operations to perform (e.g., Round-up, Round-down, Round-towards-zero and Round-to-nearest). Thus, the round operation control field **2559A** allows for the changing of the rounding mode on a per instruction basis. In one embodiment where a processor includes a control register for specifying rounding modes, the round operation control field's **2550** content overrides that register value.

In the no memory access, write mask control, VSIZE type operation **2517** instruction template, the rest of the beta field **2554** is interpreted as a vector length field **2559B**, whose

content distinguishes which one of a number of data vector lengths is to be performed on (e.g., 128, 256, or 512 byte).

In the case of a memory access **2520** instruction template of class B, part of the beta field **2554** is interpreted as a broadcast field **2557B**, whose content distinguishes whether or not the broadcast type data manipulation operation is to be performed, while the rest of the beta field **2554** is interpreted the vector length field **2559B**. The memory access **2520** instruction templates include the scale field **2560**, and optionally the displacement field **2562A** or the displacement scale field **2562B**.

With regard to the generic vector friendly instruction format **2500**, a full opcode field **2574** is shown including the format field **2540**, the base operation field **2542**, and the data element width field **2564**. While one embodiment is shown where the full opcode field **2574** includes all of these fields, the full opcode field **2574** includes less than all of these fields in embodiments that do not support all of them. The full opcode field **2574** provides the operation code (opcode).

The augmentation operation field **2550**, the data element width field **2564**, and the write mask field **2570** allow these features to be specified on a per instruction basis in the generic vector friendly instruction format.

The combination of write mask field and data element width field create typed instructions in that they allow the mask to be applied based on different data element widths.

The various instruction templates found within class A and class B are beneficial in different situations. In some embodiments, different processors or different cores within a processor may support only class A, only class B, or both classes. For instance, a high performance general purpose out-of-order core intended for general-purpose computing may support only class B, a core intended primarily for graphics and/or scientific (throughput) computing may support only class A, and a core intended for both may support both (of course, a core that has some mix of templates and instructions from both classes but not all templates and instructions from both classes is within the purview of the invention). Also, a single processor may include multiple cores, all of which support the same class or in which different cores support different class. For instance, in a processor with separate graphics and general-purpose cores, one of the graphics cores intended primarily for graphics and/or scientific computing may support only class A, while one or more of the general-purpose cores may be high performance general purpose cores with out of order execution and register renaming intended for general-purpose computing that support only class B. Another processor that does not have a separate graphics core, may include one more general purpose in-order or out-of-order cores that support both class A and class B. Of course, features from one class may also be implement in the other class in different embodiments. Programs written in a high level language would be put (e.g., just in time compiled or statically compiled) into an variety of different executable forms, including: 1) a form having only instructions of the class(es) supported by the target processor for execution; or 2) a form having alternative routines written using different combinations of the instructions of all classes and having control flow code that selects the routines to execute based on the instructions supported by the processor which is currently executing the code.

Exemplary Specific Vector Friendly Instruction Format

FIG. 26A is a block diagram illustrating an exemplary specific vector friendly instruction format according to embodiments. FIG. 26A shows a specific vector friendly instruction format **2600** that is specific in the sense that it

specifies the location, size, interpretation, and order of the fields, as well as values for some of those fields. The specific vector friendly instruction format **2600** may be used to extend the x86 instruction set, and thus some of the fields are similar or the same as those used in the existing x86 instruction set and extension thereof (e.g., AVX). This format remains consistent with the prefix encoding field, real opcode byte field, MOD R/M field, SIB field, displacement field, and immediate fields of the existing x86 instruction set with extensions. The fields from FIG. 25 into which the fields from FIG. 26A map are illustrated.

It should be understood that, although embodiments are described with reference to the specific vector friendly instruction format **2600** in the context of the generic vector friendly instruction format **2500** for illustrative purposes, the invention is not limited to the specific vector friendly instruction format **2600** except where claimed. For example, the generic vector friendly instruction format **2500** contemplates a variety of possible sizes for the various fields, while the specific vector friendly instruction format **2600** is shown as having fields of specific sizes. By way of specific example, while the data element width field **2564** is illustrated as a one-bit field in the specific vector friendly instruction format **2600**, the invention is not so limited (that is, the generic vector friendly instruction format **2500** contemplates other sizes of the data element width field **2564**).

The generic vector friendly instruction format **2500** includes the following fields listed below in the order illustrated in FIG. 26A.

EVEX Prefix **2602** (Bytes 0-3)—is encoded in a four-byte form.

Format Field **2540** (EVEX Byte 0, bits [7:0])—the first byte (EVEX Byte 0) is the format field **2540** and it contains 0x62 (the unique value used for distinguishing the vector friendly instruction format in one embodiment).

The second-fourth bytes (EVEX Bytes 1-3) include a number of bit fields providing specific capability.

REX field **2605** (EVEX Byte 1, bits [7-5])—consists of a EVEX.R bit field (EVEX Byte 1, bit [7]—R), EVEX.X bit field (EVEX byte 1, bit [6]—X), and **2557BEX** byte 1, bit [5]—B). The EVEX.R, EVEX.X, and EVEX.B bit fields provide the same functionality as the corresponding VEX bit fields, and are encoded using 1s complement form, i.e. ZMM0 is encoded as 1111B, ZMM15 is encoded as 0000B. Other fields of the instructions encode the lower three bits of the register indexes as is known in the art (rrr, xxx, and bbb), so that Rrrr, Xxxx, and Bbbb may be formed by adding EVEX.R, EVEX.X, and EVEX.B.

REX' field **2510**—this is the first part of the REX' field **2510** and is the EVEX.R' bit field (EVEX Byte 1, bit [4]—R') that is used to encode either the upper 16 or lower 16 of the extended 32 register set. In one embodiment, this bit, along with others as indicated below, is stored in bit inverted format to distinguish (in the well-known x86 32-bit mode) from the BOUND instruction, whose real opcode byte is 62, but does not accept in the MOD R/M field (described below) the value of 11 in the MOD field; alternative embodiments do not store this and the other indicated bits below in the inverted format. A value of 1 is used to encode the lower 16 registers. In other words, R'Rrrr is formed by combining EVEX.R', EVEX.R, and the other RRR from other fields.

Opcode map field **2615** (EVEX byte 1, bits [3:0]—mmmm)—its content encodes an implied leading opcode byte (0F, 0F 38, or 0F 3).

Data element width field **2564** (EVEX byte 2, bit [7]—W)—is represented by the notation EVEX.W. EVEX.W is

used to define the granularity (size) of the datatype (either 32-bit data elements or 64-bit data elements).

EVEX.vvvv **2620** (EVEX Byte 2, bits [6:3]-vvvv)—the role of EVEX.vvvv may include the following: 1) EVEX.vvvv encodes the first source register operand, specified in inverted (1s complement) form and is valid for instructions with 2 or more source operands; 2) EVEX.vvvv encodes the destination register operand, specified in 1s complement form for certain vector shifts; or 3) EVEX.vvvv does not encode any operand, the field is reserved and should contain **1111b**. Thus, EVEX.vvvv field **2620** encodes the 4 low-order bits of the first source register specifier stored in inverted (1s complement) form. Depending on the instruction, an extra different EVEX bit field is used to extend the specifier size to 32 registers.

EVEX.U **2568** Class field (EVEX byte 2, bit [2]—U)—If EVEX.U=0, it indicates class A or EVEX.U0; if EVEX.U=1, it indicates class B or EVEX.U1.

Prefix encoding field **2625** (EVEX byte 2, bits [1:0]-pp)—provides additional bits for the base operation field. In addition to providing support for the legacy SSE instructions in the EVEX prefix format, this also has the benefit of compacting the SIMD prefix (rather than requiring a byte to express the SIMD prefix, the EVEX prefix requires only 2 bits). In one embodiment, to support legacy SSE instructions that use a SIMD prefix (66H, F2H, F3H) in both the legacy format and in the EVEX prefix format, these legacy SIMD prefixes are encoded into the SIMD prefix encoding field; and at runtime are expanded into the legacy SIMD prefix prior to being provided to the decoder's PLA (so the PLA may execute both the legacy and EVEX format of these legacy instructions without modification). Although newer instructions could use the EVEX prefix encoding field's content directly as an opcode extension, certain embodiments expand in a similar fashion for consistency but allow for different meanings to be specified by these legacy SIMD prefixes. An alternative embodiment may redesign the PLA to support the 2-bit SIMD prefix encodings, and thus not require the expansion.

Alpha field **2552** (EVEX byte 3, bit [7]—EH; also known as EVEX.EH, EVEX.rs, EVEX.RL, EVEX.write mask control, and EVEX.N; also illustrated with α)—as previously described, this field is context specific.

Beta field **2554** (EVEX byte 3, bits [6:4]—SSS, also known as EVEX.S₂₋₀, EVEX.r₂₋₀, EVEX.rr1, EVEX.LL0, EVEX.LLB; also illustrated with $\beta\beta\beta$)—as previously described, this field is context specific.

REX' field **2510**—this is the remainder of the REX' field and is the EVEX.V' bit field (EVEX Byte 3, bit [3]—V') that may be used to encode either the upper 16 or lower 16 of the extended 32 register set. This bit is stored in bit inverted format. A value of 1 is used to encode the lower 16 registers. In other words, V'VVVV is formed by combining EVEX.V', EVEX.vvvv.

Write mask field **2570** (EVEX byte 3, bits [2:0]—kkk)—its content specifies the index of a register in the write mask registers as previously described. In one embodiment, the specific value EVEX.kkk=000 has a special behavior implying no write mask is used for the particular instruction (this may be implemented in a variety of ways including the use of a write mask hardwired to all ones or hardware that bypasses the masking hardware).

Real Opcode Field **2630** (Byte 4) is also known as the opcode byte. Part of the opcode is specified in this field.

MOD R/M Field **2640** (Byte 5) includes MOD field **2642**, Reg field **2644**, and R/M field **2646**. As previously described, the MOD field's **2642** content distinguishes

between memory access and non-memory access operations. The role of Reg field **2644** may be summarized to two situations: encoding either the destination register operand or a source register operand or be treated as an opcode extension and not used to encode any instruction operand. The role of R/M field **2646** may include the following: encoding the instruction operand that references a memory address or encoding either the destination register operand or a source register operand.

Scale, Index, Base (SIB) Byte (Byte 6)—As previously described, the content of SIB **2650** is used for memory address generation. SIB.xxx **2654** and SIB.bbb **2656**—the contents of these fields have been previously referred to with regard to the register indexes Xxxx and Bbbb.

Displacement field **2562A** (Bytes 7-10)—when MOD field **2642** contains 10, bytes 7-10 are the displacement field **2562A**, and it works the same as the legacy 32-bit displacement (disp32) and works at byte granularity.

Displacement factor field **2562B** (Byte 7)—when MOD field **2642** contains 01, byte 7 is the displacement factor field **2562B**. The location of this field is that same as that of the legacy x86 instruction set 8-bit displacement (disp8), which works at byte granularity. Since disp8 is sign extended, it may only address between -128 and 127 bytes offsets; in terms of 64-byte cache lines, disp8 uses 8 bits that may be set to only four really useful values -128, -64, 0, and 64; since a greater range is often needed, disp32 is used; however, disp32 requires 4 bytes. In contrast to disp8 and disp32, the displacement factor field **2562B** is a reinterpretation of disp8; when using displacement factor field **2562B**, the actual displacement is determined by the content of the displacement factor field multiplied by the size of the memory operand access (N). This type of displacement is referred to as disp8*N. This reduces the average instruction length (a single byte of used for the displacement but with a much greater range). Such compressed displacement assumes that the effective displacement is multiple of the granularity of the memory access, and hence, the redundant low-order bits of the address offset do not need to be encoded. In other words, the displacement factor field **2562B** substitutes the legacy x86 instruction set 8-bit displacement. Thus, the displacement factor field **2562B** is encoded the same way as an x86 instruction set 8-bit displacement (so no changes in the ModRM/SIB encoding rules) with the only exception that disp8 is overloaded to disp8*N. In other words, there are no changes in the encoding rules or encoding lengths but only in the interpretation of the displacement value by hardware (which needs to scale the displacement by the size of the memory operand to obtain a byte-wise address offset). Immediate field **2572** operates as previously described.

Full Opcode Field

FIG. **26B** is a block diagram illustrating the fields of the specific vector friendly instruction format **2600** that make up the full opcode field **2574** according to one embodiment. Specifically, the full opcode field **2574** includes the format field **2540**, the base operation field **2542**, and the data element width (W) field **2564**. The base operation field **2542** includes the prefix encoding field **2625**, the opcode map field **2615**, and the real opcode field **2630**.

Register Index Field

FIG. **26C** is a block diagram illustrating the fields of the specific vector friendly instruction format **2600** that make up the register index field **2544** according to one embodiment. Specifically, the register index field **2544** includes the REX **2605** field, the REX' **2610** field, the MODR/M.reg field

2644, the MODR/M.r/m field **2646**, the VVVV field **2620**, xxx field **2654**, and the bbb field **2656**.

Augmentation Operation Field

FIG. **26D** is a block diagram illustrating the fields of the specific vector friendly instruction format **2600** that make up the augmentation operation field **2550** according to one embodiment. When the class (U) field **2568** contains 0, it signifies EVEX.U0 (class A **2568A**); when it contains 1, it signifies EVEX.U1 (class B **2568B**). When U=0 and the MOD field **2642** contains 11 (signifying a no memory access operation), the alpha field **2552** (EVEX byte 3, bit [7]—EH) is interpreted as the rs field **2552A**. When the rs field **2552A** contains a 1 (round **2552A.1**), the beta field **2554** (EVEX byte 3, bits [6:4]—SSS) is interpreted as the round control field **2554A**. The round control field **2554A** includes a one-bit SAE field **2556** and a two-bit round operation field **2558**. When the rs field **2552A** contains a 0 (data transform **2552A.2**), the beta field **2554** (EVEX byte 3, bits [6:4]—SSS) is interpreted as a three-bit data transform field **2554B**. When U=0 and the MOD field **2642** contains 00, 01, or 10 (signifying a memory access operation), the alpha field **2552** (EVEX byte 3, bit [7]—EH) is interpreted as the eviction hint (EH) field **2552B** and the beta field **2554** (EVEX byte 3, bits [6:4]—SSS) is interpreted as a three bit data manipulation field **2554C**.

When U=1, the alpha field **2552** (EVEX byte 3, bit [7]—EH) is interpreted as the write mask control (Z) field **2552C**. When U=1 and the MOD field **2642** contains 11 (signifying a no memory access operation), part of the beta field **2554** (EVEX byte 3, bit [4]—S₀) is interpreted as the RL field **2557A**; when it contains a 1 (round **2557A.1**) the rest of the beta field **2554** (EVEX byte 3, bit [6-5]—S₂₋₁) is interpreted as the round operation field **2559A**, while when the RL field **2557A** contains a 0 (VSIZE **2557A.2**) the rest of the beta field **2554** (EVEX byte 3, bit [6-5]—S₂₋₁) is interpreted as the vector length field **2559B** (EVEX byte 3, bit [6-5]—L₁₋₀). When U=1 and the MOD field **2642** contains 00, 01, or 10 (signifying a memory access operation), the beta field **2554** (EVEX byte 3, bits [6:4]—SSS) is interpreted as the vector length field **2559B** (EVEX byte 3, bit [6-5]—L₁₋₀) and the broadcast field **2557B** (EVEX byte 3, bit [4]—B).

Exemplary Register Architecture

FIG. **27** is a block diagram of a register architecture **2700** according to one embodiment. In the embodiment illustrated, there are 32 vector registers **2710** that are 512 bits wide; these registers are referenced as zmm0 through zmm31. The lower order 256 bits of the lower 16 zmm registers are overlaid on registers ymm0-16. The lower order 128 bits of the lower 16 zmm registers (the lower order 128 bits of the ymm registers) are overlaid on registers xmm0-15. The specific vector friendly instruction format **2600** operates on these overlaid register file as illustrated in the below tables.

Adjustable Vector Length	Class	Operations	Registers
Instruction Templates that do not include the vector length field 2559B	A (FIG. 25A; U = 0)	2510, 2515, 2525, 2530	zmm registers (the vector length is 64 byte)
	B (FIG. 25B; U = 1)	2512	zmm registers (the vector length is 64 byte)
Instruction templates that do include the vector length field	B (FIG. 25B; U = 1)	2517, 2527	zmm, ymm, or xmm registers (the vector length is 64-byte,

-continued

Adjustable Vector Length	Class	Operations	Registers
2559B			32 byte, or 16 byte) depending on the vector length field 2559B

In other words, the vector length field **2559B** selects between a maximum length and one or more other shorter lengths, where each such shorter length is half the length of the preceding length; and instructions templates without the vector length field **2559B** operate on the maximum vector length. Further, in one embodiment, the class B instruction templates of the specific vector friendly instruction format **2600** operate on packed or scalar single/double-precision floating point data and packed or scalar integer data. Scalar operations are operations performed on the lowest order data element position in a zmm/ymm/xmm register; the higher order data element positions are either left the same as they were prior to the instruction or zeroed depending on the embodiment.

Write mask registers **2715**—in the embodiment illustrated, there are 8 write mask registers (k0 through k7), each 64 bits in size. In an alternate embodiment, the write mask registers **2715** are 16 bits in size. As previously described, in one embodiment, the vector mask register k0 may not be used as a write mask; when the encoding that would normally indicate k0 is used for a write mask, it selects a hardwired write mask of 0xFFFF, effectively disabling write masking for that instruction.

General-purpose registers **2725**—in the embodiment illustrated, there are sixteen 64-bit general-purpose registers that are used along with the existing x86 addressing modes to address memory operands. These registers are referenced by the names RAX, RBX, RCX, RDX, RBP, RSI, RDI, RSP, and R8 through R15.

Scalar floating point stack register file (x87 stack) **2745**, on which is aliased the MMX packed integer flat register file **2750**—in the embodiment illustrated, the x87 stack is an eight-element stack used to perform scalar floating-point operations on 32/64/80-bit floating point data using the x87 instruction set extension; while the MMX registers are used to perform operations on 64-bit packed integer data, as well as to hold operands for some operations performed between the MMX and XMM registers.

Alternative embodiments may use wider or narrower registers. Additionally, alternative embodiments may use more, less, or different register files and registers. Exemplary Core Architectures, Processors, and Computer Architectures

Processor cores may be implemented in different ways, for different purposes, and in different processors. For instance, implementations of such cores may include: 1) a general purpose in-order core intended for general-purpose computing; 2) a high performance general purpose out-of-order core intended for general-purpose computing; 3) a special purpose core intended primarily for graphics and/or scientific (throughput) computing. Implementations of different processors may include: 1) a CPU including one or more general purpose in-order cores intended for general-purpose computing and/or one or more general purpose out-of-order cores intended for general-purpose computing; and 2) a coprocessor including one or more special purpose cores intended primarily for graphics and/or scientific

(throughput). Such different processors lead to different computer system architectures, which may include: 1) the coprocessor on a separate chip from the CPU; 2) the coprocessor on a separate die in the same package as a CPU; 3) the coprocessor on the same die as a CPU (in which case, such a coprocessor is sometimes referred to as special purpose logic, such as integrated graphics and/or scientific (throughput) logic, or as special purpose cores); and 4) a system on a chip that may include on the same die the described CPU (sometimes referred to as the application core(s) or application processor(s)), the above described coprocessor, and additional functionality. Exemplary core architectures are described next, followed by descriptions of exemplary processors and computer architectures.

Exemplary Core Architectures

In-Order and Out-of-Order Core Block Diagram

FIG. 28A is a block diagram illustrating both an exemplary in-order pipeline and an exemplary register renaming, out-of-order issue/execution pipeline according to embodiments. FIG. 28B is a block diagram illustrating both an exemplary embodiment of an in-order architecture core and an exemplary register renaming, out-of-order issue/execution architecture core to be included in a processor according to embodiments. The solid lined boxes in FIGS. 28A-B illustrate the in-order pipeline and in-order core, while the optional addition of the dashed lined boxes illustrates the register renaming, out-of-order issue/execution pipeline and core. Given that the in-order aspect is a subset of the out-of-order aspect, the out-of-order aspect will be described.

In FIG. 28A, a processor pipeline 2800 includes a fetch stage 2802, a length decode stage 2804, a decode stage 2806, an allocation stage 2808, a renaming stage 2810, a scheduling (also known as a dispatch or issue) stage 2812, a register read/memory read stage 2814, an execute stage 2816, a write back/memory write stage 2818, an exception handling stage 2822, and a commit stage 2824.

FIG. 28B shows processor core 2890 including a front-end unit 2830 coupled to an execution engine unit 2850, and both are coupled to a memory unit 2870. The core 2890 may be a reduced instruction set computing (RISC) core, a complex instruction set computing (CISC) core, a very long instruction word (VLIW) core, or a hybrid or alternative core type. As yet another option, the core 2890 may be a special-purpose core, such as, for example, a network or communication core, compression engine, coprocessor core, general purpose computing graphics processing unit (GPGPU) core, graphics core, or the like.

The front-end unit 2830 includes a branch prediction unit 2832 coupled to an instruction cache unit 2834, which is coupled to an instruction translation lookaside buffer (TLB) 2836, which is coupled to an instruction fetch unit 2838, which is coupled to a decode unit 2840. The decode unit 2840 (or decoder) may decode instructions, and generate as an output one or more micro-operations, micro-code entry points, microinstructions, other instructions, or other control signals, which are decoded from, or which otherwise reflect, or are derived from, the original instructions. The decode unit 2840 may be implemented using various different mechanisms. Examples of suitable mechanisms include, but are not limited to, look-up tables, hardware implementations, programmable logic arrays (PLAs), microcode read only memories (ROMs), etc. In one embodiment, the core 2890 includes a microcode ROM or other medium that stores microcode for certain macroinstructions (e.g., in decode unit 2840 or otherwise within the front-end unit

2830). The decode unit 2840 is coupled to a rename/allocator unit 2852 in the execution engine unit 2850.

The execution engine unit 2850 includes the rename/allocator unit 2852 coupled to a retirement unit 2854 and a set of one or more scheduler unit(s) 2856. The scheduler unit(s) 2856 represents any number of different schedulers, including reservations stations, central instruction window, etc. The scheduler unit(s) 2856 is coupled to the physical register file(s) unit(s) 2858. Each of the physical register file(s) units 2858 represents one or more physical register files, different ones of which store one or more different data types, such as scalar integer, scalar floating point, packed integer, packed floating point, vector integer, vector floating point, status (e.g., an instruction pointer that is the address of the next instruction to be executed), etc. In one embodiment, the physical register file(s) unit 2858 comprises a vector registers unit, a write mask registers unit, and a scalar registers unit. These register units may provide architectural vector registers, vector mask registers, and general-purpose registers. The physical register file(s) unit(s) 2858 is overlapped by the retirement unit 2854 to illustrate various ways in which register renaming and out-of-order execution may be implemented (e.g., using a reorder buffer(s) and a retirement register file(s); using a future file(s), a history buffer(s), and a retirement register file(s); using a register maps and a pool of registers; etc.). The retirement unit 2854 and the physical register file(s) unit(s) 2858 are coupled to the execution cluster(s) 2860. The execution cluster(s) 2860 includes a set of one or more execution units 2862 and a set of one or more memory access units 2864. The execution units 2862 may perform various operations (e.g., shifts, addition, subtraction, multiplication) and on various types of data (e.g., scalar floating point, packed integer, packed floating point, vector integer, vector floating point). While some embodiments may include a number of execution units dedicated to specific functions or sets of functions, other embodiments may include only one execution unit or multiple execution units that all perform all functions. The scheduler unit(s) 2856, physical register file(s) unit(s) 2858, and execution cluster(s) 2860 are shown as being possibly plural because certain embodiments create separate pipelines for certain types of data/operations (e.g., a scalar integer pipeline, a scalar floating point/packed integer/packed floating point/vector integer/vector floating point pipeline, and/or a memory access pipeline that each have their own scheduler unit, physical register file(s) unit, and/or execution cluster—and in the case of a separate memory access pipeline, certain embodiments are implemented in which only the execution cluster of this pipeline has the memory access unit(s) 2864). It should also be understood that where separate pipelines are used, one or more of these pipelines may be out-of-order issue/execution and the rest in-order.

The set of memory access units 2864 is coupled to the memory unit 2870, which includes a data TLB unit 2872 coupled to a data cache unit 2874 coupled to a level 2 (L2) cache unit 2876. In one exemplary embodiment, the memory access units 2864 may include a load unit, a store address unit, and a store data unit, each of which is coupled to the data TLB unit 2872 in the memory unit 2870. The instruction cache unit 2834 is further coupled to a level 2 (L2) cache unit 2876 in the memory unit 2870. The L2 cache unit 2876 is coupled to one or more other levels of cache and eventually to a main memory.

By way of example, the exemplary register renaming, out-of-order issue/execution core architecture may implement the pipeline 2800 as follows: 1) the instruction fetch

2838 performs the fetch and length decoding stages **2802** and **2804**; 2) the decode unit **2840** performs the decode stage **2806**; 3) the rename/allocator unit **2852** performs the allocation stage **2808** and renaming stage **2810**; 4) the scheduler unit(s) **2856** performs the schedule stage **2812**; 5) the physical register file(s) unit(s) **2858** and the memory unit **2870** perform the register read/memory read stage **2814**; the execution cluster **2860** perform the execute stage **2816**; 6) the memory unit **2870** and the physical register file(s) unit(s) **2858** perform the write back/memory write stage **2818**; 7) various units may be involved in the exception handling stage **2822**; and 8) the retirement unit **2854** and the physical register file(s) unit(s) **2858** perform the commit stage **2824**.

The core **2890** may support one or more instructions sets (e.g., the x86 instruction set (with some extensions that have been added with newer versions); the MIPS instruction set of MIPS Technologies of Sunnyvale, CA; the ARM instruction set (with optional additional extensions such as NEON) of ARM Holdings of Sunnyvale, CA), including the instruction(s) described herein. In one embodiment, the core **2890** includes logic to support a packed data instruction set extension (e.g., AVX1, AVX2), thereby allowing the operations used by many multimedia applications to be performed using packed data.

It should be understood that the core may support multithreading (executing two or more parallel sets of operations or threads), and may do so in a variety of ways including time sliced multithreading, simultaneous multithreading (where a single physical core provides a logical core for each of the threads that physical core is simultaneously multithreading), or a combination thereof (e.g., time sliced fetching and decoding and simultaneous multithreading thereafter such as in the Intel® Hyperthreading technology).

While register renaming is described in the context of out-of-order execution, it should be understood that register renaming may be used in an in-order architecture. While the illustrated embodiment of the processor also includes separate instruction and data cache units **2834/2874** and a shared L2 cache unit **2876**, alternative embodiments may have a single internal cache for both instructions and data, such as, for example, a Level 1 (L1) internal cache, or multiple levels of internal cache. In some embodiments, the system may include a combination of an internal cache and an external cache that is external to the core and/or the processor. Alternatively, all of the cache may be external to the core and/or the processor.

Specific Exemplary In-Order Core Architecture

FIGS. **29A-B** illustrate a block diagram of a more specific exemplary in-order core architecture, which core would be one of several logic blocks (including other cores of the same type and/or different types) in a chip. The logic blocks communicate through a high-bandwidth interconnect network (e.g., a ring network) with some fixed function logic, memory I/O interfaces, and other necessary I/O logic, depending on the application.

FIG. **29A** is a block diagram of a single processor core, along with its connection to the on-die interconnect network **2902** and with its local subset of the Level 2 (L2) cache **2904**, according to embodiments. In one embodiment, an instruction decoder **2900** supports the x86 instruction set with a packed data instruction set extension. An L1 cache **2906** allows low-latency accesses to cache memory into the scalar and vector units. While in one embodiment (to simplify the design), a scalar unit **2908** and a vector unit **2910** use separate register sets (respectively, scalar registers **2912** and vector registers **2914**) and data transferred between them is written to memory and then read back in

from a level 1 (L1) cache **2906**, alternative embodiments may use a different approach (e.g., use a single register set or include a communication path that allow data to be transferred between the two register files without being written and read back).

The local subset of the L2 cache **2904** is part of a global L2 cache that is divided into separate local subsets, one per processor core. Each processor core has a direct access path to its own local subset of the L2 cache **2904**. Data read by a processor core is stored in its L2cache subset **2904** and may be accessed quickly, in parallel with other processor cores accessing their own local L2 cache subsets. Data written by a processor core is stored in its own L2 cache subset **2904** and is flushed from other subsets, if necessary. The ring network ensures coherency for shared data. The ring network is bi-directional to allow agents such as processor cores, L2 caches and other logic blocks to communicate with each other within the chip. Each ring data-path is 1012-bits wide per direction.

FIG. **29B** is an expanded view of part of the processor core in FIG. **29A** according to embodiments. FIG. **29B** includes an L1 data cache **2906A** part of the L1 cache **2904**, as well as more detail regarding the vector unit **2910** and the vector registers **2914**. Specifically, the vector unit **2910** is a 16-wide vector processing unit (VPU) (see the 16-wide ALU **2928**), which executes one or more of integer, single-precision float, and double-precision float instructions. The VPU supports swizzling the register inputs with swizzle unit **2920**, numeric conversion with numeric convert units **2922A-B**, and replication with replication unit **2924** on the memory input. Write mask registers **2926** allow predicating resulting vector writes.

FIG. **30** is a block diagram of a processor **3000** that may have more than one core, may have an integrated memory controller, and may have integrated graphics according to embodiments. The solid lined boxes in FIG. **30** illustrate a processor **3000** with a single core **3002A**, a system agent **3010**, a set of one or more bus controller units **3016**, while the optional addition of the dashed lined boxes illustrates an alternative processor **3000** with multiple cores **3002A-N**, a set of one or more integrated memory controller unit(s) **3014** in the system agent unit **3010**, and special purpose logic **3008**.

Thus, different implementations of the processor **3000** may include: 1) a CPU with the special purpose logic **3008** being integrated graphics and/or scientific (throughput) logic (which may include one or more cores), and the cores **3002A-N** being one or more general purpose cores (e.g., general purpose in-order cores, general purpose out-of-order cores, a combination of the two); 2) a coprocessor with the cores **3002A-N** being a large number of special purpose cores intended primarily for graphics and/or scientific (throughput); and 3) a coprocessor with the cores **3002A-N** being a large number of general purpose in-order cores. Thus, the processor **3000** may be a general-purpose processor, coprocessor or special-purpose processor, such as, for example, a network or communication processor, compression engine, graphics processor, GPGPU (general purpose graphics processing unit), a high-throughput many integrated core (MIC) coprocessor (including 30 or more cores), embedded processor, or the like. The processor may be implemented on one or more chips. The processor **3000** may be a part of and/or may be implemented on one or more substrates using any of a number of process technologies, such as, for example, BiCMOS, CMOS, or NMOS.

The memory hierarchy includes one or more levels of cache within the cores, a set or one or more shared cache

units **3006**, and external memory (not shown) coupled to the set of integrated memory controller units **3014**. The set of shared cache units **3006** may include one or more mid-level caches, such as level 2 (L2), level 3 (L3), level 4 (L4), or other levels of cache, a last level cache (LLC), and/or combinations thereof. While in one embodiment a ring based interconnect unit **3012** interconnects the special purpose logic **3008** (integrated graphics logic is an example of and is also referred to herein as special purpose logic), the set of shared cache units **3006**, and the system agent unit **3010**/integrated memory controller unit(s) **3014**, alternative embodiments may use any number of well-known techniques for interconnecting such units. In one embodiment, coherency is maintained between one or more cache units **3006** and cores **3002A-N**.

In some embodiments, one or more of the cores **3002A-N** are capable of multi-threading. The system agent **3010** includes those components coordinating and operating cores **3002A-N**. The system agent unit **3010** may include for example a power control unit (PCU) and a display unit. The PCU may be or include logic and components needed for regulating the power state of the cores **3002A-N** and the special purpose logic **3008**. The display unit is for driving one or more externally connected displays.

The cores **3002A-N** may be homogenous or heterogeneous in terms of architecture instruction set; that is, two or more of the cores **3002A-N** may be capable of execution the same instruction set, while others may be capable of executing only a subset of that instruction set or a different instruction set.

Exemplary Computer Architectures

FIGS. **31-34** are block diagrams of exemplary computer architectures. Other system designs and configurations known in the arts for laptops, desktops, handheld PCs, personal digital assistants, engineering workstations, servers, network devices, network hubs, switches, embedded processors, digital signal processors (DSPs), graphics devices, video game devices, set-top boxes, micro controllers, cell phones, portable media players, hand held devices, and various other electronic devices, are also suitable. In general, a huge variety of systems or electronic devices capable of incorporating a processor and/or other execution logic as disclosed herein are generally suitable.

Referring now to FIG. **31**, shown is a block diagram of a system **3100** in accordance with one embodiment of the present invention. The system **3100** may include one or more processors **3110**, **3115**, which are coupled to a controller hub **3120**. In one embodiment the controller hub **3120** includes a graphics memory controller hub (GMCH) **3190** and an Input/Output Hub (IOH) **3150** (which may be on separate chips); the GMCH **3190** includes memory and graphics controllers to which are coupled memory **3140** and a coprocessor **3145**; the IOH **3150** couples input/output (I/O) devices **3160** to the GMCH **3190**. Alternatively, one or both of the memory and graphics controllers are integrated within the processor (as described herein), the memory **3140** and the coprocessor **3145** are coupled directly to the processor **3110**, and the controller hub **3120** in a single chip with the IOH **3150**.

The optional nature of additional processors **3115** is denoted in FIG. **31** with broken lines. Each processor **3110**, **3115** may include one or more of the processing cores described herein and may be some version of the processor **3000**.

The memory **3140** may be, for example, dynamic random-access memory (DRAM), phase change memory (PCM), or a combination of the two. For at least one

embodiment, the controller hub **3120** communicates with the processor(s) **3110**, **3115** via a multi-drop bus, such as a frontside bus (FSB), point-to-point interface such as Quick-Path Interconnect (QPI), or similar connection **3195**.

In one embodiment, the coprocessor **3145** is a special-purpose processor, such as, for example, a high-throughput MIC processor, a network or communication processor, compression engine, graphics processor, GPGPU, embedded processor, or the like. In one embodiment, controller hub **3120** may include an integrated graphics accelerator.

There may be a variety of differences between the physical resources **3110**, **3115** in terms of a spectrum of metrics of merit including architectural, microarchitectural, thermal, power consumption characteristics, and the like.

In one embodiment, the processor **3110** executes instructions that control data processing operations of a general type. Embedded within the instructions may be coprocessor instructions. The processor **3110** recognizes these coprocessor instructions as being of a type that should be executed by the attached coprocessor **3145**. Accordingly, the processor **3110** issues these coprocessor instructions (or control signals representing coprocessor instructions) on a coprocessor bus or other interconnect, to coprocessor **3145**. Coprocessor(s) **3145** accept and execute the received coprocessor instructions.

Referring now to FIG. **32**, shown is a block diagram of a first more specific exemplary system **3200** in accordance with an embodiment of the present invention. As shown in FIG. **32**, multiprocessor system **3200** is a point-to-point interconnect system, and includes a first processor **3270** and a second processor **3280** coupled via a point-to-point interconnect **3250**. Each of processors **3270** and **3280** may be some version of the processor **3000**. In one embodiment, processors **3270** and **3280** are respectively processors **3110** and **3115**, while coprocessor **3238** is coprocessor **3145**. In another embodiment, processors **3270** and **3280** are respectively processor **3110** coprocessor **3145**.

Processors **3270** and **3280** are shown including integrated memory controller (IMC) units **3272** and **3282**, respectively. Processor **3270** also includes as part of its bus controller unit's point-to-point (P-P) interfaces **3276** and **3278**; similarly, second processor **3280** includes P-P interfaces **3286** and **3288**. Processors **3270**, **3280** may exchange information via a point-to-point (P-P) interface **3250** using P-P interface circuits **3278**, **3288**. As shown in FIG. **32**, IMCs **3272** and **3282** couple the processors to respective memories, namely a memory **3232** and a memory **3234**, which may be portions of main memory locally attached to the respective processors.

Processors **3270**, **3280** may each exchange information with a chipset **3290** via individual P-P interfaces **3252**, **3254** using point to point interface circuits **3276**, **3294**, **3286**, **3298**. Chipset **3290** may optionally exchange information with the coprocessor **3238** via a high-performance interface **3292**. In one embodiment, the coprocessor **3238** is a special-purpose processor, such as, for example, a high-throughput MIC processor, a network or communication processor, compression engine, graphics processor, GPGPU, embedded processor, or the like.

A shared cache (not shown) may be included in either processor or outside of both processors yet connected with the processors via P-P interconnect, such that either or both processors' local cache information may be stored in the shared cache if a processor is placed into a low power mode.

Chipset **3290** may be coupled to a first bus **3216** via an interface **3296**. In one embodiment, first bus **3216** may be a Peripheral Component Interconnect (PCI) bus, or a bus such

as a PCI Express bus or another third generation I/O interconnect bus, although the scope of the present invention is not so limited.

As shown in FIG. 32, various I/O devices 3214 may be coupled to first bus 3216, along with a bus bridge 3218 which couples first bus 3216 to a second bus 3220. In one embodiment, one or more additional processor(s) 3215, such as coprocessors, high-throughput MIC processors, GPGPU's, accelerators (such as, e.g., graphics accelerators or digital signal processing (DSP) units), field programmable gate arrays, or any other processor, are coupled to first bus 3216. In one embodiment, second bus 3220 may be a low pin count (LPC) bus. Various devices may be coupled to a second bus 3220 including, for example, a keyboard and/or mouse 3222, communication devices 3227 and a storage unit 3228 such as a disk drive or other mass storage device which may include instructions/code and data 3230, in one embodiment. Further, an audio I/O 3224 may be coupled to the second bus 3220. Note that other architectures are possible. For example, instead of the point-to-point architecture of FIG. 32, a system may implement a multi-drop bus or other such architecture.

Referring now to FIG. 33, shown is a block diagram of a second more specific exemplary system 3300 in accordance with an embodiment of the present invention. Like elements in FIGS. 32 and 33 bear like reference numerals, and certain aspects of FIG. 32 have been omitted from FIG. 33 in order to avoid obscuring other aspects of FIG. 33.

FIG. 33 illustrates that the processors 3270, 3280 may include integrated memory and I/O control logic ("CL") 3372 and 3382, respectively. Thus, the CL 3372, 3382 include integrated memory controller units and include I/O control logic. FIG. 33 illustrates that not only are the memories 3232, 3234 coupled to the CL 3372, 3382, but also that I/O devices 3314 are also coupled to the control logic 3372, 3382. Legacy I/O devices 3315 are coupled to the chipset 3290.

Referring now to FIG. 34, shown is a block diagram of a SoC 3400 in accordance with an embodiment of the present invention. Similar elements in FIG. 30 bear like reference numerals. Also, dashed lined boxes are optional features on more advanced SoCs. In FIG. 34, an interconnect unit(s) 3402 is coupled to: an application processor 3410 which includes a set of one or more cores 3002A-N, which include cache units 3004A-N, and shared cache unit(s) 3006; a system agent unit 3010; a bus controller unit(s) 3016; an integrated memory controller unit(s) 3014; a set or one or more coprocessors 3420 which may include integrated graphics logic, an image processor, an audio processor, and a video processor; an static random access memory (SRAM) unit 3430; a direct memory access (DMA) unit 3432; and a display unit 3440 for coupling to one or more external displays. In one embodiment, the coprocessor(s) 3420 include a special-purpose processor, such as, for example, a network or communication processor, compression engine, GPGPU, a high-throughput MIC processor, embedded processor, or the like.

Embodiments of the mechanisms disclosed herein may be implemented in hardware, software, firmware, or a combination of such implementation approaches. Embodiments may be implemented as computer programs or program code executing on programmable systems comprising at least one processor, a storage system (including volatile and non-volatile memory and/or storage elements), at least one input device, and at least one output device.

Program code, such as code 3230 illustrated in FIG. 32, may be applied to input instructions to perform the functions

described herein and generate output information. The output information may be applied to one or more output devices, in known fashion. For purposes of this application, a processing system includes any system that has a processor, such as, for example; a digital signal processor (DSP), a microcontroller, an application specific integrated circuit (ASIC), or a microprocessor.

The program code may be implemented in a high-level procedural or object-oriented programming language to communicate with a processing system. The program code may also be implemented in assembly or machine language, if desired. In fact, the mechanisms described herein are not limited in scope to any particular programming language. In any case, the language may be a compiled or interpreted language.

One or more aspects of at least one embodiment may be implemented by representative instructions stored on a machine-readable medium which represents various logic within the processor, which when read by a machine causes the machine to fabricate logic to perform the techniques described herein. Such representations, known as "IP cores," may be stored on a tangible, machine readable medium and supplied to various customers or manufacturing facilities to load into the fabrication machines that actually make the logic or processor.

Such machine-readable storage media may include, without limitation, non-transitory, tangible arrangements of articles manufactured or formed by a machine or device, including storage media such as hard disks, any other type of disk including floppy disks, optical disks, compact disk read-only memories (CD-ROMs), compact disk rewritables (CD-RWs), and magneto-optical disks, semiconductor devices such as read-only memories (ROMs), random access memories (RAMs) such as dynamic random access memories (DRAMs), static random access memories (SRAMs), erasable programmable read-only memories (EPROMs), flash memories, electrically erasable programmable read-only memories (EEPROMs), phase change memory (PCM), magnetic or optical cards, or any other type of media suitable for storing electronic instructions.

Accordingly, embodiments also include non-transitory, tangible machine-readable media containing instructions or containing design data, such as Hardware Description Language (HDL), which defines structures, circuits, apparatuses, processors and/or system features described herein. Such embodiments may also be referred to as program products.

Emulation (Including Binary Translation, Code Morphing, Etc.)

In some cases, an instruction converter may be used to convert an instruction from a source instruction set to a target instruction set. For example, the instruction converter may translate (e.g., using static binary translation, dynamic binary translation including dynamic compilation), morph, emulate, or otherwise convert an instruction to one or more other instructions to be processed by the core. The instruction converter may be implemented in software, hardware, firmware, or a combination thereof. The instruction converter may be on processor, off processor, or part on and part off processor.

FIG. 35 is a block diagram contrasting the use of a software instruction converter to convert binary instructions in a source instruction set to binary instructions in a target instruction set according to embodiments. In the illustrated embodiment, the instruction converter is a software instruction converter, although alternatively the instruction converter may be implemented in software, firmware, hardware,

or various combinations thereof. FIG. 35 shows a program in a high-level language 3502 may be compiled using an x86 compiler 3504 to generate x86 binary code 3506 that may be natively executed by a processor with at least one x86 instruction set core 3516. The processor with at least one x86 instruction set core 3516 represents any processor that may perform substantially the same functions as an Intel processor with at least one x86 instruction set core by compatibly executing or otherwise processing (1) a substantial portion of the instruction set of the Intel x86 instruction set core or (2) object code versions of applications or other software targeted to run on an Intel processor with at least one x86 instruction set core, in order to achieve substantially the same result as an Intel processor with at least one x86 instruction set core. The x86 compiler 3504 represents a compiler that is operable to generate x86 binary code 3506 (e.g., object code) that may, with or without additional linkage processing, be executed on the processor with at least one x86 instruction set core 3516. Similarly, FIG. 35 shows the program in the high level language 3502 may be compiled using an alternative instruction set compiler 3508 to generate alternative instruction set binary code 3510 that may be natively executed by a processor without at least one x86 instruction set core 3514 (e.g., a processor with cores that execute the MIPS instruction set of MIPS Technologies of Sunnyvale, CA and/or that execute the ARM instruction set of ARM Holdings of Sunnyvale, CA). The instruction converter 3512 is used to convert the x86 binary code 3506 into code that may be natively executed by the processor without an x86 instruction set core 3514. This converted code is not likely to be the same as the alternative instruction set binary code 3510 because an instruction converter capable of this is difficult to make; however, the converted code will accomplish the general operation and be made up of instructions from the alternative instruction set. Thus, the instruction converter 3512 represents software, firmware, hardware, or a combination thereof that, through emulation, simulation or any other process, allows a processor or other electronic device that does not have an x86 instruction set processor or core to execute the x86 binary code 3506.

What is claimed is:

1. A processor, comprising:

a plurality of registers to store a plurality of packed data elements including a first plurality of packed data elements of a first source matrix tile and a second plurality of packed data elements of a second source matrix tile, the first and second source matrix tiles comprising respective portions of a first source matrix and a second source matrix, and wherein each packed data element of the plurality of packed data elements has an element width;

a decoder to decode one or more instructions, at least one instruction of the one or more instructions including an opcode field configured to specify an opcode, a first source operand configured to indicate the first plurality of packed data elements, a second source operand configured to indicate the second plurality of packed data elements, and a destination operand configured to indicate a result matrix tile;

execution circuitry, in response to the one or more instructions, to transpose the first source matrix tile in accordance with a granularity equal to the element width to generate a first transposed source matrix tile comprising the first plurality of packed data elements and to multiply the first transposed source matrix tile and the second source matrix tile, the execution circuitry comprising:

a plurality of multipliers to multiply data elements of the first transposed source matrix tile and corresponding data elements of the second source matrix tile to produce a corresponding plurality of products; and

one or more accumulators to add groups of the products to generate corresponding result data elements in the result matrix tile.

2. The processor of claim 1, wherein to transpose the first source matrix tile, the execution circuitry is to transform each row of the first source matrix tile to a column of the first transposed source matrix tile.

3. The processor of claim 2, wherein each column of the first transposed source matrix tile has a width equal to the element width.

4. The processor of claim 3, wherein the element width comprises an 8-bit width or a 16-bit width and a result element width of each of the result data elements comprises a 32-bit width.

5. The processor of claim 4, wherein the first and second plurality of packed data elements comprise 16-bit floating point data elements and each of the result data elements comprise single-precision floating point data elements.

6. The processor of claim 5, wherein the 16-bit floating point data elements comprise bfloat16 data elements.

7. The processor of claim 1, further comprising:

a scheduler to schedule the one or more instructions for execution on the execution circuitry.

8. A method comprising:

storing in a plurality of registers a plurality of packed data elements including a first plurality of packed data elements of a first source matrix tile and a second plurality of packed data elements of a second source matrix tile, the first and second source matrix tiles comprising respective portions of a first source matrix and a second source matrix, and wherein each packed data element of the plurality of packed data elements has an element width;

decoding one or more instructions, at least one instruction of the one or more instructions including an opcode field configured to specify an opcode, a first source operand configured to indicate the first plurality of packed data elements, a second source operand configured to indicate the second plurality of packed data elements, and a destination operand configured to indicate a result matrix tile;

transposing, in response to the one or more instructions, the first source matrix tile in accordance with a granularity equal to the element width to generate a first transposed source matrix tile comprising the first plurality of packed data elements and to multiply the first transposed source matrix tile and the second source matrix tile, wherein multiplying the first transposed source matrix tile and the second source matrix tile comprises:

multiplying data elements of the first transposed source matrix tile and corresponding data elements of the second source matrix tile to produce a corresponding plurality of products; and

adding groups of the products to generate corresponding result data elements in the result matrix tile.

9. The method of claim 8, wherein transposing the first source matrix tile comprises transforming each row of the first source matrix tile to a column of the first transposed source matrix tile.

51

10. The method of claim 9, wherein each column of the first transposed source matrix tile has a width equal to the element width.

11. The method of claim 10, wherein the element width comprises an 8-bit width or a 16-bit width and a result element width of each of the result data elements comprises a 32-bit width.

12. The method of claim 11, wherein the first and second plurality of packed data elements comprise 16-bit floating point data elements and each of the result data elements comprise single-precision floating point data elements.

13. The method of claim 12, wherein the 16-bit floating point data elements comprise bfloat16 data elements.

14. The method of claim 8, further comprising:
scheduling the one or more instructions for execution on the execution circuitry.

15. A non-transitory machine-readable medium containing instructions which, when executed by a processor, are to cause the processor to respond by:

storing in a plurality of registers a plurality of packed data elements including a first plurality of packed data elements of a first source matrix tile and a second plurality of packed data elements of a second source matrix tile, the first and second source matrix tiles comprising respective portions of a first source matrix and a second source matrix, and wherein each packed data element of the plurality of packed data elements has an element width;

decoding one or more instructions, at least one instruction of the one or more instructions including an opcode field configured to specify an opcode, a first source operand configured to indicate the first plurality of packed data elements, a second source operand configured to indicate the second plurality of packed data elements, and a destination operand configured to indicate a result matrix tile;

52

transposing, in response to the one or more instructions, the first source matrix tile in accordance with a granularity equal to the element width to generate a first transposed source matrix tile comprising the first plurality of packed data elements and to multiply the first transposed source matrix tile and the second source matrix tile, wherein multiplying the first transposed source matrix tile and the second source matrix tile comprises:

multiplying data elements of the first transposed source matrix tile and corresponding data elements of the second source matrix tile to produce a corresponding plurality of products; and

adding groups of the products to generate corresponding result data elements in the result matrix tile.

16. The machine-readable medium of claim 15, wherein transposing the first source matrix tile comprises transforming each row of the first source matrix tile to a column of the first transposed source matrix tile.

17. The machine-readable medium of claim 16, wherein each column of the first transposed source matrix tile has a width equal to the element width.

18. The machine-readable medium of claim 17, wherein the element width comprises an 8-bit width or a 16-bit width and a result element width of each of the result data elements comprises a 32-bit width.

19. The machine-readable medium of claim 18, wherein the first and second plurality of packed data elements comprise 16-bit floating point data elements and each of the result data elements comprise single-precision floating point data elements.

20. The machine-readable medium of claim 19, wherein the 16-bit floating point data elements comprise bfloat16 data elements.

* * * * *